

Explicación y defensa del código

Programación Avanzada - Parcial 2do Cuatrimestre - Octubre 2025

Profesor: Macrino Matias · Universidad del Salvador – Sede Pilar

Aplicación web para una empresa de logística: ABM de camiones y choferes, carga de viajes con cálculo de tiempo y tanques, y un perfil de chofer que inicia y finaliza sus viajes. Este documento explica **cada línea** del código entregado y **defiende** las decisiones de diseño, organizado para responder las preguntas de una defensa oral.

Contenido

- **Parte I** – Visión general: qué se pidió, arquitectura, modelo de datos y cómo se cumple cada punto de evaluación.
- **Parte II** – Flujos principales de la aplicación, paso a paso.
- **Parte III** – Todo el código fuente explicado línea por línea y defendido.
- **Parte IV** – Preguntas frecuentes de defensa, supuestos y limitaciones reconocidas.

Parte I - Visión general

1. Qué pide el enunciado y dónde está resuelto

Requisito del enunciado	Dónde está resuelto
Choferes con nombre, apellido, DNI, fecha de nacimiento, categoría y solo celular; autorizados a uno o más camiones	Tabla chofer + chofer_camion; clases Persona y Chofer; ChoferServlet
Camiones con marca, modelo, dominio, toneladas máximas, litros del tanque y litros por km; manejados por uno o más choferes	Tabla camion; clase Camion; CamionServlet
Destinos únicamente: CABA, Córdoba, Corrientes, Neuquén, Formosa, La Rioja, Mendoza y La Plata	Enum Destino y tabla destino (con FK desde distancia y viaje)
Pantalla principal de inicio de sesión con usuario y contraseña	LoginServlet (web.xml) + login.jsp
Recordar al usuario sin volver a iniciar sesión	Cookie RECORDARME con token aleatorio guardado en sesion_recordada; AutenticacionFilter la restaura
Dos perfiles: administración y chofer	Enum Rol; AutenticacionFilter restringe /admin/* y /chofer/*
Administrador: cargar viajes, designar chofer, ABM de camiones y choferes	ViajeServlet, CamionServlet, ChoferServlet
Chofer: ver solo sus viajes, marcar inicio y marcar realizado	MisViajesServlet; procedure sp_finalizar_viaje
Buscar chofer por DNI; si no existe, informar en pantalla	ChoferDAO.buscarPorDni lanza ChoferNoEncontradoException; ViajeServlet la muestra con SweetAlert
Mostrar camiones que el chofer puede manejar y que no estén en viaje	Procedure sp_camiones_disponibles (CallableStatement)
Elegir origen y destino; calcular tiempo de viaje (200 km/día) y cantidad de tanques	Tabla distancia + CalculadoraViaje
Iniciar desde el path root con contexto (http://host:port/contexto/)	WAR llamado logistica y welcome-file index.jsp
Session y cookie; al cerrar sesión limpiar ambas	SesionUtil + LogoutServlet (invalida Session, expira cookies, borra token)
Entregables: código Maven con proyectos DAO y MVC, y script DDL	logistica-dao, logistica-web, db/01_ddl_logistica.sql

2. Arquitectura

El proyecto es un **Maven multi-módulo** con el POM padre y dos proyectos, tal como pide el profesor («separación de proyectos DAO y MVC»):

Módulo	Contiene	Depende de
logistica-dao (jar)	Dominio (modelo), excepciones, utilidades, interfaces DAO, DAOFactory, implementaciones JDBC, Singleton de conexión y tests	Solo el driver de MySQL y JUnit. No sabe nada de la web.
logistica-web (war)	Controladores (Servlets), filtros, vistas JSP + JSTL, CSS y JS	logistica-dao + APIs de Servlet/JSP (las provee Tomcat)

Flujo de una petición (MVC): el navegador llama a una URL → el AutenticacionFilter controla sesión y perfil → el **Controlador** (servlet) lee los parámetros, pide datos a los **DAO** (obtenidos de la DAOFactory) que consultan MySQL por JDBC y devuelven objetos del **Modelo** → el controlador deja esos objetos en la request y hace forward a una **Vista** (JSP) que los muestra con JSTL. Después de un POST siempre se

redirige (Post/Redirect/Get).

Capas y responsabilidades: el modelo contiene las reglas (validaciones, cálculo de viaje, qué camión puede manejar un chofer); el DAO solo traduce entre objetos y SQL; el controlador solo coordina; la vista solo muestra. Ninguna capa salta a otra.

3. Modelo de datos

Tabla	Descripción	Relaciones
categoría	Categorías A-D con toneladas máximas	1 categoría → N choferes
destino	Las 8 ciudades permitidas	Referenciada por distancia y viaje
distancia	Km entre cada par de ciudades (56 filas)	FK origen y destino → destino
camion	Datos del camión (dominio único)	N a N con chofer vía chofer_camion
chofer	Datos del chofer (DNI único)	FK categoría; N a N con camión
chofer_camion	Autorizaciones (clave compuesta)	FK chofer y camión con ON DELETE CASCADE
usuario	Acceso al sistema; rol ADMIN o CHOFER	0..1 chofer (UNIQUE)
sesion_recordada	Tokens de la cookie «recordarme»	FK usuario con cascade
viaje	Viaje con cálculo guardado y estado	FK chofer, camión, origen y destino

Además hay dos **stored procedures**: sp_camiones_disponibles y sp_finalizar_viaje, invocados con CallableStatement.

4. Cómo se cumple cada punto de evaluación

Punto	Cómo se cumple	Dónde verlo
01 – Patrones de diseño (DAO, Factory, MVC, Singleton)	DAO : interfaces + implementaciones JDBC. Factory : DAOFactory/JdbcDAOFactory. MVC : servlets (C), JSP (V), clases del dominio (M). Singleton : ConexionDB (holder idiom).	dao/*, dao/jdbc/*, controlador/*, WEB-INF/jsp
02 – Buenas prácticas de POO	Abstracción y herencia (Persona → Chofer, BaseServlet → servlets), encapsulamiento (atributos privados, lista de solo lectura), polimorfismo (validar() redefinido), clases inmutables (EstimacionViaje, Usuario), enums con comportamiento, constantes sin números mágicos, composición, comentarios y nombres claros.	modelo/*, BaseServlet
03 – Uso correcto de excepciones	Jerarquía propia (LogisticaException → DAOException, ValidacionException, ChoferNoEncontradoException). Se encadena la causa, nunca se traga un error, los recursos se cierran con try-with-resources/finally, y cada tipo recibe un tratamiento distinto en la capa web.	excepcion/*, jdbcUtil, servlets

Punto	Cómo se cumple	Dónde verlo
04 – Dominio coherente y estructura de BD	Tablas normalizadas con PK, FK, UNIQUE y CHECK; relación N a N explícita; las reglas críticas están también en la base.	01_ddl_logistica.sql
05 – Transacciones JDBC	Alta/modificación de chofer (3 tablas), alta de viaje (bloqueo FOR UPDATE + verificación + insert), iniciar viaje y guardar token: setAutoCommit(false), commit y rollback.	ChoferDAOJdbc, ViajeDAOJdbc, UsuarioDAOJdbc
06 – Statement, PreparedStatement y CallableStatement	Statement: listados fijos. PreparedStatement: todo lo que recibe datos. CallableStatement: camiones disponibles (procedure) y finalizar viaje (procedure con OUT).	CamionDAOJdbc, ChoferDAOJdbc, ViajeDAOJdbc, DistanciaDAOJdbc
07 – HTML, JS, Bootstrap y Sweetalert	Bootstrap 5 para diseño responsive, JS propio (app.js) y SweetAlert2 para avisos y confirmaciones, con degradación si no cargan.	encabezado.jspf, pie.jspf, js/app.js
08 – Servlets por anotaciones y web.xml	Login y Logout y el filtro de autenticación en web.xml; Inicio, Choferes, Camiones, Viajes, MisViajes y EncodingFilter con @WebServlet/@WebFilter.	web.xml, controlador/*
09 – JSP y tags JSP (JSTL)	Todas las vistas son JSP con c:, fmt: y fn:. Los scriptlets están prohibidos por configuración (scripting-invalid).	WEB-INF/jsp/*, web.xml
10 – Métodos HTTP	GET para consultar (listas, formularios, búsqueda y cálculo); POST para modificar (guardar, eliminar, iniciar, finalizar, login, logout); PRG tras cada POST; 400 ante acciones desconocidas; 403 por perfil.	Todos los servlets

Parte II - Flujos principales

Inicio de sesión y «recordarme»

- El usuario abre /logistica/: index.jsp redirige a /inicio; el filtro no ve sesión ni cookie y lo manda a /login.
- POST /login: se busca el usuario, se verifica la contraseña contra el hash PBKDF2, se cambia el id de sesión y se guarda el usuario en la Session.
- Si tildó «Recordarme»: se genera un token aleatorio, se guarda en la tabla sesion_recordada con vencimiento a 30 días y se envía en la cookie RECORDARME (HttpOnly, SameSite=Lax).
- Si luego se pierde la Session (cierra el navegador, expira), el filtro lee la cookie, valida el token en la base y reconstruye la Session: no pide la contraseña.

Cierre de sesión

- POST /logout: elimina el token de la base, invalida la Session, expira la cookie RECORDARME y JSESSIONID y redirige al login. El botón «Atrás» no muestra páginas privadas (Cache-Control: no-store).

Carga de un viaje (administrador)

- GET /admin/viajes?accion=nuevo: se ingresa el DNI. Si no existe: aviso «No existe ningún chofer con DNI ...».
- Si existe: se muestran los camiones que tiene autorizados, que entran en su categoría y que no están en un viaje ASIGNADO o EN_CURSO (procedure).
- Se eligen camión, origen y destino. «Calcular» (GET) muestra distancia, días (km/200, hacia arriba), litros y tanques (litros/capacidad, hacia arriba).
- «Confirmar» (POST): el servidor vuelve a buscar el chofer, verifica la disponibilidad del camión, recalcula y guarda en una transacción con el camión bloqueado. Se redirige al listado.

Trabajo del chofer

- GET /chofer/viajes: lista solo sus viajes. «Iniciar viaje» (POST) pasa ASIGNADO → EN_CURSO (solo si no tiene otro en curso). «Marcar como realizado» (POST) pasa EN_CURSO → FINALIZADO y el camión queda libre.

Parte III - Código fuente explicado línea por línea

A continuación está **todo el código del proyecto**. Cada recuadro gris muestra líneas reales del archivo con su número; el recuadro verde de abajo explica qué hacen esas líneas y por qué se escribieron así (la defensa). Los archivos siguen el orden de las capas: base de datos, dominio, acceso a datos (DAO), controladores, filtros y vistas.

Base de datos y configuración Maven

db / 01_ddl_logistica.sql

Para qué existe: Script DDL pedido como entregable 02. Crea la base **logistica** en MySQL 8 con todas las tablas, restricciones, datos maestros (categorías, destinos y la tabla de distancias del enunciado) y los dos stored procedures que usa Java con CallableStatement.

Defensa: El dominio está normalizado (3FN): cada dato vive en una sola tabla y se relaciona por claves foráneas. Las reglas importantes (DNI único, dominio único, km > 0, origen ≠ destino, estados válidos) están también como restricciones de la base, no solo en Java: si alguien insertara datos por fuera de la aplicación, la base igualmente los rechaza. Se usa InnoDB porque es el motor con transacciones y claves foráneas.

```
1 | -- =====
2 | -- Script DDL - Parcial Programacion Avanzada (2do Cuatrimestre 2025)
3 | -- Base de datos: MySQL 8.x (probado tambien en MariaDB 10.11)
4 | -- Ejecutar con: mysql -u root -p < 01_ddl_logistica.sql
5 | -- Incluye: tablas, restricciones, tabla de distancias y stored procedures
6 | -- (los procedures se usan desde Java con CallableStatement).
7 | -- =====
8 |
```

Líneas 1 a 8: Comentario de cabecera: indica cómo ejecutar el script y qué incluye.

```
9 | DROP DATABASE IF EXISTS logistica;
10 | CREATE DATABASE logistica CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
11 | USE logistica;
12 |
13 |
14 | -- Categoria de licencia del chofer: define las toneladas que puede llevar
15 | -- =====
```

Líneas 9 a 15: Borra la base si ya existía y la crea de nuevo con **utf8mb4** para guardar tildes, eñes y cualquier carácter. **USE logistica** selecciona la base para las sentencias siguientes. Hace que el script sea repetible.

```
16 | CREATE TABLE categoria (
17 |     codigo      CHAR(1) NOT NULL,
18 |     toneladas_max INT NOT NULL,
19 |     CONSTRAINT pk_categoria PRIMARY KEY (codigo),
20 |     CONSTRAINT ck_categoria_ton CHECK (toneladas_max > 0)
21 | ) ENGINE=InnoDB;
22 |
23 |
24 | -- Destinos permitidos (la empresa solo transporta a estas 8 ciudades)
25 | -- =====
```

Líneas 16 a 25: Tabla **categoria**: código de una letra (clave primaria) y las toneladas máximas que habilita. El CHECK impide toneladas ≤ 0. Es la regla del enunciado «categoría para saber qué camiones pueden transportar ciertas toneladas».

```
26 | CREATE TABLE destino (
27 |     codigo VARCHAR(20) NOT NULL,
28 |     nombre VARCHAR(40) NOT NULL,
29 |     CONSTRAINT pk_destino PRIMARY KEY (codigo)
30 | ) ENGINE=InnoDB;
31 |
32 |
33 | -- Tabla de distancias en km (se carga en ambos sentidos: A->B y B->A)
34 | -- =====
```

Líneas 26 a 34: Tabla **destino**: las 8 ciudades permitidas. El código coincide con el nombre del enum Java Destino. Las demás tablas la referencian con clave foránea, así es imposible guardar un destino inexistente.

```

35 | CREATE TABLE distancia (
36 |     origen VARCHAR(20) NOT NULL,
37 |     destino VARCHAR(20) NOT NULL,
38 |     km INT NOT NULL,
39 |     CONSTRAINT pk_distancia PRIMARY KEY (origen, destino),
40 |     CONSTRAINT fk_distancia_origen FOREIGN KEY (origen) REFERENCES destino (codigo),
41 |     CONSTRAINT fk_distancia_destino FOREIGN KEY (destino) REFERENCES destino (codigo),
42 |     CONSTRAINT ck_distancia_km CHECK (km > 0),
43 |     CONSTRAINT ck_distancia_dist CHECK (origen <> destino)
44 | ) ENGINE=InnoDB;
45 |
46 | -----
47 | -- Camiones
48 | -----

```

Líneas 35 a 48: Tabla **distancia**: km entre cada par de ciudades. La clave primaria es (origen, destino); dos claves foráneas apuntan a destino; los CHECK impiden $km \leq 0$ y $origen = destino$. Se carga en ambos sentidos para resolver con una sola consulta simple.

```

49 | CREATE TABLE camion (
50 |     id BIGINT NOT NULL AUTO_INCREMENT,
51 |     marca VARCHAR(40) NOT NULL,
52 |     modelo VARCHAR(40) NOT NULL,
53 |     dominio VARCHAR(10) NOT NULL,
54 |     toneladas_max DECIMAL(6,2) NOT NULL,
55 |     capacidad_tanque_litros DECIMAL(8,2) NOT NULL,
56 |     consumo_litros_km DECIMAL(6,3) NOT NULL,
57 |     CONSTRAINT pk_camion PRIMARY KEY (id),
58 |     CONSTRAINT uq_camion_dominio UNIQUE (dominio),
59 |     CONSTRAINT ck_camion_ton CHECK (toneladas_max > 0),
60 |     CONSTRAINT ck_camion_tanque CHECK (capacidad_tanque_litros > 0),
61 |     CONSTRAINT ck_camion_consumo CHECK (consumo_litros_km > 0)
62 | ) ENGINE=InnoDB;
63 |
64 | -----
65 | -- Choferes (datos personales + categoria + telefono celular)
66 | -----

```

Líneas 49 a 66: Tabla **camion** con los datos del enunciado: marca, modelo, dominio (UNIQUE: no hay dos camiones con la misma patente), toneladas máximas, capacidad del tanque y litros por km. DECIMAL en lugar de FLOAT para no perder precisión. Los CHECK exigen valores positivos.

```

67 | CREATE TABLE chofer (
68 |     id BIGINT NOT NULL AUTO_INCREMENT,
69 |     nombre VARCHAR(50) NOT NULL,
70 |     apellido VARCHAR(50) NOT NULL,
71 |     dni VARCHAR(8) NOT NULL,
72 |     fecha_nacimiento DATE NOT NULL,
73 |     categoria CHAR(1) NOT NULL,
74 |     telefono VARCHAR(15) NOT NULL,
75 |     CONSTRAINT pk_chofer PRIMARY KEY (id),
76 |     CONSTRAINT uq_chofer_dni UNIQUE (dni),
77 |     CONSTRAINT fk_chofer_categoria FOREIGN KEY (categoria) REFERENCES categoria (codigo)
78 | ) ENGINE=InnoDB;
79 |
80 | -----
81 | -- Relacion N a N: un chofer puede manejar varios camiones y un camion
82 | -- puede ser manejado por varios choferes.
83 | -----

```

Líneas 67 a 83: Tabla **chofer**: nombre, apellido, DNI (UNIQUE), fecha de nacimiento, categoría (FK a categoria) y teléfono celular como único dato de contacto, tal como pide el enunciado.

```

84 | CREATE TABLE chofer_camion (
85 |     chofer_id BIGINT NOT NULL,
86 |     camion_id BIGINT NOT NULL,
87 |     CONSTRAINT pk_chofer_camion PRIMARY KEY (chofer_id, camion_id),
88 |     CONSTRAINT fk_cc_chofer FOREIGN KEY (chofer_id) REFERENCES chofer (id) ON DELETE CASCADE,
89 |     CONSTRAINT fk_cc_camion FOREIGN KEY (camion_id) REFERENCES camion (id) ON DELETE CASCADE
90 | ) ENGINE=InnoDB;
91 |
92 | -----
93 | -- Usuarios del sistema. Perfil ADMIN (sin chofer) o CHOFER (ligado a un chofer)
94 | -----

```

Líneas 84 a 94: Tabla intermedia de la relación **muchos a muchos**: un chofer maneja uno o más camiones y un camión puede ser manejado por varios choferes. La clave primaria compuesta evita duplicados. ON DELETE CASCADE: al borrar un chofer o un camión se borran sus autorizaciones automáticamente.

```

95 | CREATE TABLE usuario (
96 |     id                BIGINT          NOT NULL AUTO_INCREMENT,
97 |     username          VARCHAR(30)     NOT NULL,
98 |     password_hash     VARCHAR(200)    NOT NULL,
99 |     rol                VARCHAR(10)     NOT NULL,
100 |     chofer_id         BIGINT          NULL,
101 |     CONSTRAINT pk_usuario PRIMARY KEY (id),
102 |     CONSTRAINT uq_usuario_username UNIQUE (username),
103 |     CONSTRAINT uq_usuario_chofer UNIQUE (chofer_id),
104 |     CONSTRAINT fk_usuario_chofer FOREIGN KEY (chofer_id) REFERENCES chofer (id) ON DELETE CASCADE,
105 |     CONSTRAINT ck_usuario_rol CHECK (
106 |         (rol = 'ADMIN' AND chofer_id IS NULL) OR (rol = 'CHOFER' AND chofer_id IS NOT NULL))
107 | ) ENGINE=InnoDB;
108 |
109 | -----
110 | -- Tokens de la cookie "recordarme" (sesion recordada)
111 | -----

```

Líneas 95 a 111: Tabla **usuario** para el inicio de sesión: username único, hash de la contraseña (nunca el texto), rol y, si es CHOFER, el chofer asociado (UNIQUE: un usuario por chofer). El CHECK final garantiza la coherencia: ADMIN no tiene chofer y CHOFER siempre tiene uno.

```

112 | CREATE TABLE sesion_recordada (
113 |     token             CHAR(64) NOT NULL,
114 |     usuario_id        BIGINT   NOT NULL,
115 |     expira             DATETIME NOT NULL,
116 |     CONSTRAINT pk_sesion_recordada PRIMARY KEY (token),
117 |     CONSTRAINT fk_sesion_usuario FOREIGN KEY (usuario_id) REFERENCES usuario (id) ON DELETE CASCADE
118 | ) ENGINE=InnoDB;
119 |
120 | -----
121 | -- Viajes. Estados: ASIGNADO -> EN_CURSO -> FINALIZADO
122 | -----

```

Líneas 112 a 122: Tabla de tokens de la cookie «recordarme». Guarda el token aleatorio, a qué usuario pertenece y cuándo vence. Permite «recordar» al usuario sin guardar su contraseña en el navegador, y revocar la sesión borrando la fila.

```

123 | CREATE TABLE viaje (
124 |     id                BIGINT          NOT NULL AUTO_INCREMENT,
125 |     chofer_id         BIGINT          NOT NULL,
126 |     camion_id         BIGINT          NOT NULL,
127 |     origen            VARCHAR(20)     NOT NULL,
128 |     destino           VARCHAR(20)     NOT NULL,
129 |     distancia_km      INT             NOT NULL,
130 |     dias_estimados    INT             NOT NULL,
131 |     litros_estimados  DECIMAL(10,2)  NOT NULL,
132 |     tanques           INT             NOT NULL,
133 |     estado            VARCHAR(12)     NOT NULL DEFAULT 'ASIGNADO',
134 |     fecha_carga       DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
135 |     fecha_inicio      DATETIME        NULL,
136 |     fecha_fin         DATETIME        NULL,
137 |     CONSTRAINT pk_viaje PRIMARY KEY (id),
138 |     CONSTRAINT fk_viaje_chofer FOREIGN KEY (chofer_id) REFERENCES chofer (id),
139 |     CONSTRAINT fk_viaje_camion FOREIGN KEY (camion_id) REFERENCES camion (id),
140 |     CONSTRAINT fk_viaje_origen FOREIGN KEY (origen) REFERENCES destino (codigo),
141 |     CONSTRAINT fk_viaje_destino FOREIGN KEY (destino) REFERENCES destino (codigo),
142 |     CONSTRAINT ck_viaje_estado CHECK (estado IN ('ASIGNADO', 'EN_CURSO', 'FINALIZADO')),
143 |     CONSTRAINT ck_viaje_od CHECK (origen <> destino),
144 |     CONSTRAINT ck_viaje_km CHECK (distancia_km > 0)
145 | ) ENGINE=InnoDB;
146 |

```

Líneas 123 a 146: Tabla **viaje**: chofer, camión, origen, destino y el resultado del cálculo (km, días, litros, tanques) guardado para tener historial fiel aunque después cambie el camión. El estado es ASIGNADO, EN_CURSO o FINALIZADO (CHECK). Las fechas de carga, inicio y fin permiten auditar. Las FK a chofer y camión no tienen cascade: no se puede borrar un chofer o camión con viajes.

```

147 | CREATE INDEX ix_viaje_chofer ON viaje (chofer_id, estado);
148 | CREATE INDEX ix_viaje_camion ON viaje (camion_id, estado);
149 |
150 | -----
151 | -- Datos maestros
152 | -----

```

Líneas 147 a 152: Índices para las consultas más frecuentes: viajes de un chofer por estado y viajes de un camión por estado (usada para saber si el camión está ocupado).

```

153 | INSERT INTO categoria (codigo, toneladas_max) VALUES
154 |     ('A', 10), ('B', 20), ('C', 30), ('D', 40);
155 |

```

Líneas 153 a 155: Datos maestros: categorías A (10 t), B (20 t), C (30 t) y D (40 t). Son valores que elegí como supuesto, porque el enunciado no fija cuántas toneladas corresponden a cada categoría.

```

156 | INSERT INTO destino (codigo, nombre) VALUES
157 |     ('CABA', 'CABA'),
158 |     ('CORDOBA', 'Córdoba'),
159 |     ('CORRIENTES', 'Corrientes'),
160 |     ('FORMOSA', 'Formosa'),
161 |     ('LA_PLATA', 'La Plata'),
162 |     ('LA_RIOJA', 'La Rioja'),
163 |     ('MENDOZA', 'Mendoza'),
164 |     ('NEUQUEN', 'Neuquén');
165 |
166 | -- Tabla de distancias del enunciado (km)

```

Líneas 156 a 166: Carga las 8 ciudades del enunciado: CABA, Córdoba, Corrientes, Formosa, La Plata, La Rioja, Mendoza y Neuquén.

```

167 | INSERT INTO distancia (origen, destino, km) VALUES
168 |     ('CABA', 'CORDOBA', 646),
169 |     ('CABA', 'CORRIENTES', 792),
170 |     ('CABA', 'FORMOSA', 933),
171 |     ('CABA', 'LA_PLATA', 53),
172 |     ('CABA', 'LA_RIOJA', 986),
173 |     ('CABA', 'MENDOZA', 985),
174 |     ('CABA', 'NEUQUEN', 989),
175 |     ('CORDOBA', 'CABA', 646),
176 |     ('CORDOBA', 'CORRIENTES', 677),
177 |     ('CORDOBA', 'FORMOSA', 824),
178 |     ('CORDOBA', 'LA_PLATA', 698),
179 |     ('CORDOBA', 'LA_RIOJA', 340),
180 |     ('CORDOBA', 'MENDOZA', 466),
181 |     ('CORDOBA', 'NEUQUEN', 907),
182 |     ('CORRIENTES', 'CABA', 792),
183 |     ('CORRIENTES', 'CORDOBA', 677),
184 |     ('CORRIENTES', 'FORMOSA', 157),
185 |     ('CORRIENTES', 'LA_PLATA', 830),
186 |     ('CORRIENTES', 'LA_RIOJA', 814),
187 |     ('CORRIENTES', 'MENDOZA', 1131),
188 |     ('CORRIENTES', 'NEUQUEN', 1534),
189 |     ('FORMOSA', 'CABA', 933),
190 |     ('FORMOSA', 'CORDOBA', 824),
191 |     ('FORMOSA', 'CORRIENTES', 157),
192 |     ('FORMOSA', 'LA_PLATA', 968),
193 |     ('FORMOSA', 'LA_RIOJA', 927),
194 |     ('FORMOSA', 'MENDOZA', 1269),
195 |     ('FORMOSA', 'NEUQUEN', 1690),
196 |     ('LA_PLATA', 'CABA', 53),
197 |     ('LA_PLATA', 'CORDOBA', 698),
198 |     ('LA_PLATA', 'CORRIENTES', 830),
199 |     ('LA_PLATA', 'FORMOSA', 968),
200 |     ('LA_PLATA', 'LA_RIOJA', 1038),
201 |     ('LA_PLATA', 'MENDOZA', 1029),
202 |     ('LA_PLATA', 'NEUQUEN', 1005),
203 |     ('LA_RIOJA', 'CABA', 986),
204 |     ('LA_RIOJA', 'CORDOBA', 340),
205 |     ('LA_RIOJA', 'CORRIENTES', 814),
206 |     ('LA_RIOJA', 'FORMOSA', 927),
207 |     ('LA_RIOJA', 'LA_PLATA', 1038),
208 |     ('LA_RIOJA', 'MENDOZA', 427),
209 |     ('LA_RIOJA', 'NEUQUEN', 1063),
210 |     ('MENDOZA', 'CABA', 985),
211 |     ('MENDOZA', 'CORDOBA', 466),
212 |     ('MENDOZA', 'CORRIENTES', 1131),
213 |     ('MENDOZA', 'FORMOSA', 1269),
214 |     ('MENDOZA', 'LA_PLATA', 1029),
215 |     ('MENDOZA', 'LA_RIOJA', 427),
216 |     ('MENDOZA', 'NEUQUEN', 676),
217 |     ('NEUQUEN', 'CABA', 989),
218 |     ('NEUQUEN', 'CORDOBA', 907),
219 |     ('NEUQUEN', 'CORRIENTES', 1534),
220 |     ('NEUQUEN', 'FORMOSA', 1690),
221 |     ('NEUQUEN', 'LA_PLATA', 1005),
222 |     ('NEUQUEN', 'LA_RIOJA', 1063),
223 |     ('NEUQUEN', 'MENDOZA', 676);
224 |
225 | -----
226 | -- Stored procedures (se invocan con CallableStatement)
227 | -----

```

Líneas 167 a 227: Carga la tabla de distancias del enunciado, 56 pares (8 × 7) en ambos sentidos. La tabla del PDF es simétrica y se verificó al generar el script.

```
228 | DELIMITER $$
229 |
230 | -- Camiones que el chofer tiene autorizados, que entran en su categoria
231 | -- y que NO estan en un viaje pendiente (ASIGNADO) ni en curso.
```

Líneas 228 a 231: Cambia el delimitador de ; a \$\$ para poder escribir procedimientos cuyo cuerpo contiene muchos ;.

```
232 | CREATE PROCEDURE sp_camiones_disponibles (IN p_chofer_id BIGINT)
233 | BEGIN
234 |     SELECT c.id, c.marca, c.modelo, c.dominio, c.toneladas_max,
235 |           c.capacidad_tanque_litros, c.consumo_litros_km
236 |     FROM camion c
237 |    JOIN chofer_camion cc ON cc.camion_id = c.id AND cc.chofer_id = p_chofer_id
238 |    JOIN chofer ch      ON ch.id = cc.chofer_id
239 |    JOIN categoria cat  ON cat.codigo = ch.categoria
240 |   WHERE c.toneladas_max <= cat.toneladas_max
241 |     AND NOT EXISTS (SELECT 1 FROM viaje v
242 |                     WHERE v.camion_id = c.id
243 |                           AND v.estado IN ('ASIGNADO', 'EN_CURSO'))
244 |   ORDER BY c.marca, c.modelo, c.dominio;
245 | END$$
246 |
247 | -- Marca un viaje como FINALIZADO. Solo si pertenece al chofer y esta EN_CURSO.
248 | -- Devuelve en p_filas la cantidad de filas modificadas (0 = no se pudo).
```

Líneas 232 a 248: Stored procedure 1 (se llama con CallableStatement). Recibe el id de un chofer y devuelve los camiones que: están autorizados para él (JOIN chofer_camion), entran en su categoría (toneladas ≤ máximo de la categoría) y no tienen un viaje ASIGNADO o EN_CURSO (NOT EXISTS). Es exactamente la regla «mostrar los camiones que ese chofer puede manejar y los que no se encuentren en viaje».

```
249 | CREATE PROCEDURE sp_finalizar_viaje (IN p_viaje_id BIGINT, IN p_chofer_id BIGINT, OUT p_filas INT)
250 | BEGIN
251 |     UPDATE viaje
252 |     SET estado = 'FINALIZADO', fecha_fin = NOW()
253 |     WHERE id = p_viaje_id AND chofer_id = p_chofer_id AND estado = 'EN_CURSO';
254 |     SET p_filas = ROW_COUNT();
255 | END$$
256 |
```

Líneas 249 a 256: Stored procedure 2. Recibe el viaje y el chofer (IN) y devuelve por un parámetro OUT cuántas filas modificó. El UPDATE solo afecta si el viaje es de ese chofer y está EN_CURSO; ROW_COUNT() permite a Java saber si funcionó (0 = no correspondía finalizarlo).

```
257 | DELIMITER ;
```

Línea 257: Restaura el delimitador normal.

db / 02_datos_iniciales.sql

Para qué existe: Datos de demostración para poder probar la aplicación apenas se instala: 4 camiones, 2 choferes, sus autorizaciones y 3 usuarios.

Defensa: Las contraseñas no están en texto plano: son hashes PBKDF2 generados con la misma clase PasswordUtil que usa la aplicación. Es un script aparte del DDL para poder crear la estructura sin datos de prueba en un entorno real.

```
1 | -- =====
2 | -- Datos iniciales de demostracion (ejecutar despues de 01_ddl_logistica.sql)
3 | -- Administrador:  admin / admin123
4 | -- Choferes:      30111222 / chofer123 (Juan Perez, categoria C)
5 | --               28555666 / chofer123 (Maria Gomez, categoria B)
6 | -- Las contraseñas estan guardadas con PBKDF2 (nunca en texto plano).
7 | -- =====
```

Líneas 1 a 7: Cabecera con los usuarios y contraseñas de prueba.

```
8 | USE logistica;
9 |
```

Líneas 8 a 9: Selecciona la base creada por el DDL.

```

10 | INSERT INTO camion (marca, modelo, dominio, toneladas_max, capacidad_tanque_litros, consumo_litros_km) VALUES
11 |     ('Scania', 'R450', 'AB123CD', 28.00, 600.00, 0.400),
12 |     ('Mercedes-Benz', 'Actros', 'AC456EF', 25.00, 500.00, 0.350),
13 |     ('Iveco', 'Tector', 'AA789GH', 12.00, 250.00, 0.250),
14 |     ('Volkswagen', 'Delivery', 'AD321IJ', 8.00, 150.00, 0.180);
15 |

```

Líneas 10 a 15: Cuatro camiones con distintas toneladas, tanques y consumos para poder probar categorías y cálculos.

```

16 | INSERT INTO chofer (nombre, apellido, dni, fecha_nacimiento, categoria, telefono) VALUES
17 |     ('Juan', 'Pérez', '30111222', '1985-03-14', 'C', '1155667788'),
18 |     ('María', 'Gómez', '28555666', '1982-11-02', 'B', '1133445566');
19 |

```

Líneas 16 a 19: Dos choferes: Juan Pérez (categoría C, hasta 30 t) y María Gómez (categoría B, hasta 20 t).

```

20 | -- Autorizaciones: Juan maneja los tres camiones grandes; Maria, los dos chicos
21 | INSERT INTO chofer_camion (chofer_id, camion_id) VALUES
22 |     (1, 1), (1, 2), (1, 3),
23 |     (2, 3), (2, 4);
24 |

```

Líneas 20 a 24: Relaciona cada chofer con los camiones que puede manejar (tabla N a N).

```

25 | INSERT INTO usuario (username, password_hash, rol, chofer_id) VALUES
26 |     ('admin', '65536$q7KabaZiTaaUIXU3GuIWyA==$Qv970io5LXoUGDKhrRQ1UNQnD9lu+00sGrfGKYzx3gs=', 'ADMIN', NULL),
27 |     ('30111222', '65536$K/tW0Ur6sCQq1mREstBWIw==$uf7wAX1zXDItyLTQapJ4kXidRl/xFZBbQbtQeCQUnzM=', 'CHOFER', 1),
28 |     ('28555666', '65536$K/tW0Ur6sCQq1mREstBWIw==$uf7wAX1zXDItyLTQapJ4kXidRl/xFZBbQbtQeCQUnzM=', 'CHOFER', 2);

```

Líneas 25 a 28: Usuario administrador (sin chofer) y un usuario por chofer, cuyo username es el DNI. Los hashes están en formato iteraciones\$sa\$hash.

pom.xml

Para qué existe: POM padre de Maven. Agrupa los dos proyectos pedidos por el profesor: **logistica-dao** y **logistica-web** (MVC).

Defensa: El enunciado pide «código fuente utilizando Maven con su respectiva separación de proyectos DAO y MVC». El padre con packaging pom permite compilar ambos con un solo **mvn package** y fija la versión de Java en un solo lugar.

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <!-- POM padre: agrupa los dos proyectos pedidos por el profesor (DAO y MVC). -->
3 | <project xmlns="http://maven.apache.org/POM/4.0.0"
4 |     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5 |     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

```

Líneas 1 a 5: Declaración XML y espacio de nombres del POM.

```

6 |     <modelVersion>4.0.0</modelVersion>
7 |
8 |     <groupId>ar.edu.usal.logistica</groupId>
9 |     <artifactId>logistica-parent</artifactId>
10 |     <version>1.0.0</version>
11 |     <packaging>pom</packaging>
12 |     <name>Logística - Parcial Programación Avanzada</name>
13 |

```

Líneas 6 a 13: Versión del modelo de POM (siempre 4.0.0) y coordenadas del proyecto: groupId, artifactId y versión. **packaging pom** indica que es un agregador sin código propio.

```

14 |     <modules>
15 |         <module>logistica-dao</module>
16 |         <module>logistica-web</module>
17 |     </modules>
18 |

```

Líneas 14 a 18: Lista los dos módulos hijos; Maven los compila en el orden correcto (primero DAO porque web depende de él).

```

19 | <properties>
20 |   <maven.compiler.release>11</maven.compiler.release>
21 |   <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
22 | </properties>
23 |

```

Líneas 19 a 23: Java 11 para todos los módulos y UTF-8 como codificación de los fuentes (tildes correctas).

```

24 |   <dependencyManagement>
25 |     <dependencies>
26 |       <dependency>
27 |         <groupId>ar.edu.usal.logistica</groupId>
28 |         <artifactId>logistica-dao</artifactId>
29 |         <version>${project.version}</version>
30 |       </dependency>
31 |     </dependencies>
32 |   </dependencyManagement>
33 | </project>

```

Líneas 24 a 33: Define la versión de logistica-dao una sola vez; el módulo web la referencia sin repetir la versión.

logistica-dao / pom.xml

Para qué existe: POM del proyecto DAO: contiene dominio, excepciones y acceso a datos JDBC. No depende de ninguna librería web.

Defensa: Separar DAO de MVC permite reutilizar la capa de datos con otra interfaz (por ejemplo una aplicación de escritorio) y probarla sin servidor web. El driver de MySQL es de alcance runtime: el código nunca lo nombra, solo JDBC estándar.

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <!-- Proyecto DAO: dominio (modelo), excepciones y acceso a datos JDBC. No conoce nada de la web. -->
3 | <project xmlns="http://maven.apache.org/POM/4.0.0"
4 |   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5 |   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
6 |   <modelVersion>4.0.0</modelVersion>
7 |

```

Líneas 1 a 7: Cabecera y declaración del proyecto hijo.

```

8 |   <parent>
9 |     <groupId>ar.edu.usal.logistica</groupId>
10 |     <artifactId>logistica-parent</artifactId>
11 |     <version>1.0.0</version>
12 |   </parent>
13 |

```

Líneas 8 a 13: Hereda del POM padre (versión de Java y codificación).

```

14 |   <artifactId>logistica-dao</artifactId>
15 |   <packaging>jar</packaging>
16 |   <name>Logistica - DAO</name>
17 |

```

Líneas 14 a 17: Nombre del módulo y empaquetado como **jar** (una biblioteca).

```

18 |   <dependencies>
19 |     <!-- Driver JDBC de MySQL: solo se necesita en ejecucion -->
20 |     <dependency>
21 |       <groupId>com.mysql</groupId>
22 |       <artifactId>mysql-connector-j</artifactId>
23 |       <version>8.4.0</version>
24 |       <scope>runtime</scope>
25 |     </dependency>
26 |     <dependency>
27 |       <groupId>org.junit.jupiter</groupId>
28 |       <artifactId>junit-jupiter</artifactId>
29 |       <version>5.10.2</version>
30 |       <scope>test</scope>
31 |     </dependency>
32 |   </dependencies>
33 | </project>

```

Líneas 18 a 33: Dependencias: **mysql-connector-j** (driver oficial de MySQL, runtime) y **Junit 5** (solo para tests).

logistica-web / pom.xml

Para qué existe: POM del proyecto MVC, empaquetado como **war** para desplegar en Tomcat.

Defensa: Las APIs de Servlet y JSP son provided: las aporta Tomcat y, si fueran dentro del WAR, chocarían. JSTL en cambio sí va en el WAR porque Tomcat no lo incluye. El finalName «logistica» produce la URL base `http://host:puerto/logistica/` que pide el enunciado (path root con contexto).

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <!-- Proyecto MVC: controladores (servlets), vistas (JSP + JSTL) y filtros. Depende del proyecto DAO. -->
3 | <project xmlns="http://maven.apache.org/POM/4.0.0"
4 |         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5 |         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
6 |     <modelVersion>4.0.0</modelVersion>
7 |
```

Líneas 1 a 7: Cabecera y proyecto hijo.

```
8 |     <parent>
9 |         <groupId>ar.edu.usal.logistica</groupId>
10 |         <artifactId>logistica-parent</artifactId>
11 |         <version>1.0.0</version>
12 |     </parent>
13 |
```

Líneas 8 a 13: Hereda del padre.

```
14 |         <artifactId>logistica-web</artifactId>
15 |         <packaging>war</packaging>
16 |         <name>Logistica - Web MVC</name>
17 |
```

Líneas 14 a 17: Módulo web: packaging **war**.

```
18 |     <dependencies>
19 |         <dependency>
20 |             <groupId>ar.edu.usal.logistica</groupId>
21 |             <artifactId>logistica-dao</artifactId>
22 |         </dependency>
23 |         <!-- Los que dice "provided" los aporta Tomcat 10.1 -->
24 |     </dependencies>
```

Líneas 18 a 24: Dependencia del módulo DAO (compile).

```
25 |         <groupId>jakarta.servlet</groupId>
26 |         <artifactId>jakarta.servlet-api</artifactId>
27 |         <version>6.0.0</version>
28 |         <scope>provided</scope>
29 |     </dependency>
30 |     <dependency>
31 |         <groupId>jakarta.servlet.jsp</groupId>
```

Líneas 25 a 31: Jakarta Servlet 6.0 (provided).

```
32 |         <artifactId>jakarta.servlet.jsp-api</artifactId>
33 |         <version>3.1.1</version>
34 |         <scope>provided</scope>
35 |     </dependency>
36 |     <!-- JSTL: Tomcat no lo trae, va dentro del WAR -->
37 |     <dependency>
```

Líneas 32 a 37: Jakarta JSP 3.1 (provided): necesaria para compilar las clases que usan JSP.

```
38 |         <groupId>org.glassfish.web</groupId>
39 |         <artifactId>jakarta.servlet.jsp.jstl</artifactId>
40 |         <version>3.0.1</version>
41 |     </dependency>
42 |     <dependency>
43 |         <groupId>jakarta.servlet.jsp.jstl</groupId>
```

Líneas 38 a 43: Implementación de JSTL 3.0 (las etiquetas `c:`, `fmt:`, `fn:`). Se excluye la API de EL porque Tomcat ya la trae y duplicarla causa errores.

```

44 |         <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
45 |         <version>3.0.0</version>
46 |         <exclusions>
47 |             <!-- La API de EL ya la trae Tomcat: dentro del WAR causaria conflictos -->
48 |             <exclusion>
49 |                 <groupId>jakarta.el</groupId>
50 |                 <artifactId>jakarta.el-api</artifactId>
51 |             </exclusion>
52 |         </exclusions>
53 |     </dependency>
54 | </dependencies>
55 |

```

Líneas 44 a 55: API de JSTL, con la misma exclusión de EL.

```

56 |     <build>
57 |         <!-- El WAR se llama "logistica": contexto http://host:puerto/logistica/ -->
58 |         <finalName>logistica</finalName>
59 |         <plugins>
60 |             <plugin>
61 |                 <groupId>org.apache.maven.plugins</groupId>
62 |                 <artifactId>maven-war-plugin</artifactId>
63 |                 <version>3.4.0</version>
64 |             </plugin>
65 |         </plugins>
66 |     </build>
67 | </project>

```

Líneas 56 a 67: Nombre final del WAR (**logistica.war**) → contexto /logistica, y versión del plugin que arma el WAR.

logistica-dao / src / main / resources / db.properties

Para qué existe: Parámetros de conexión a MySQL.

Defensa: Sacar usuario, clave y URL del código permite cambiarlos sin recompilar. Además ConexionDB acepta variables de entorno que tienen prioridad, práctica habitual para no dejar credenciales reales en el repositorio.

```

1 | # Configuración de la conexión a MySQL.
2 | # Se puede sobrescribir con variables de entorno DB_URL, DB_USER y DB_PASSWORD.

```

Líneas 1 a 2: Comentario con la posibilidad de sobrescribir por variables de entorno.

```

3 | db.driver=com.mysql.cj.jdbc.Driver

```

Línea 3: Clase del driver JDBC de MySQL.

```

4 | db.url=jdbc:mysql://localhost:3306/logistica?useSSL=false&allowPublicKeyRetrieval=true&connectionTimeZone=LOCAL&forceC
   |     onnectionTimeZoneToSession=true&characterEncoding=UTF-8

```

Línea 4: URL JDBC: servidor, puerto, base **logistica**; useSSL=false y allowPublicKeyRetrieval facilitan la conexión local con MySQL 8; connectionTimeZone=LOCAL y forceConnectionTimeZoneToSession hacen que fechas y NOW() de la base usen la misma zona horaria que Java.

```

5 | db.user=root

```

Línea 5: Usuario de la base (cambiar por el propio).

```

6 | db.password=root

```

Línea 6: Contraseña de la base (cambiar por la propia).

logistica-web / src / main / webapp / WEB-INF / web.xml

Para qué existe: Descriptor de despliegue. Cumple el punto «servlets configurados por anotaciones y web.xml»: acá se declaran LoginServlet y LogoutServlet y el filtro de autenticación; los demás servlets usan @WebServlet.

Defensa: Mostrar las dos formas de configuración es un requisito explícito del punto 08. Elegí web.xml para login/logout y seguridad porque son piezas de infraestructura que conviene ver centralizadas, y anotaciones para los controladores de negocio, que quedan autodocumentados con su URL.

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <!--
3 |     Descriptor de despliegue. Convive con las anotaciones:
4 |     - Login y Logout se declaran ACA (servlets por web.xml).
5 |     - Inicio, Choferes, Camiones, Viajes y MisViajes se declaran con @WebServlet.
6 |     - El filtro de autenticacion se declara aca; el de codificacion, con @WebFilter.
7 | -->
8 | <web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
9 |         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
10 |         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
11 |         version="6.0">
12 |

```

Líneas 1 a 12: Cabecera XML y raíz **web-app** versión 6.0 (Jakarta EE 10, Tomcat 10.1).

```

13 |     <display-name>Logistica</display-name>
14 |
15 |     <!-- ===== Servlets declarados por web.xml ===== -->
16 |     <servlet>

```

Líneas 13 a 16: Nombre de la aplicación y comentario sobre qué se declara acá y qué por anotaciones.

```

17 |         <servlet-name>LoginServlet</servlet-name>
18 |         <servlet-class>ar.edu.usal.logistica.web.controlador.LoginServlet</servlet-class>
19 |     </servlet>
20 |     <servlet-mapping>
21 |         <servlet-name>LoginServlet</servlet-name>
22 |         <url-pattern>/login</url-pattern>
23 |     </servlet-mapping>
24 |
25 |     <servlet>

```

Líneas 17 a 25: Declara el servlet de login (clase) y lo mapea a la URL **/login**.

```

26 |         <servlet-name>LogoutServlet</servlet-name>
27 |         <servlet-class>ar.edu.usal.logistica.web.controlador.LogoutServlet</servlet-class>
28 |     </servlet>
29 |     <servlet-mapping>
30 |         <servlet-name>LogoutServlet</servlet-name>
31 |         <url-pattern>/logout</url-pattern>
32 |     </servlet-mapping>
33 |
34 |     <!-- ===== Filtro de autenticacion y autorizacion por perfil ===== -->

```

Líneas 26 a 34: Declara el servlet de logout y lo mapea a **/logout**.

```

35 |     <filter>
36 |         <filter-name>AutenticacionFilter</filter-name>
37 |         <filter-class>ar.edu.usal.logistica.web.filtro.AutenticacionFilter</filter-class>
38 |     </filter>
39 |     <filter-mapping>
40 |         <filter-name>AutenticacionFilter</filter-name>
41 |         <url-pattern>/*</url-pattern>
42 |     </filter-mapping>
43 |
44 |     <!-- ===== Session: expira a los 30 minutos sin actividad; cookie de sesion segura ===== -->

```

Líneas 35 a 44: Declara el filtro de autenticación y lo aplica a **todas** las URLs (*). Por defecto actúa solo sobre requests externas, no sobre los forward internos a las JSP.

```

45 |     <session-config>
46 |         <session-timeout>30</session-timeout>
47 |         <cookie-config>
48 |             <http-only>true</http-only>
49 |         </cookie-config>
50 |         <tracking-mode>COOKIE</tracking-mode>
51 |     </session-config>
52 |
53 |     <!-- Pagina de entrada: http://host:puerto/logistica/ -->

```

Líneas 45 a 53: Session: expira a los 30 minutos de inactividad, cookie JSESSIONID **HttpOnly** (JavaScript no puede leerla) y seguimiento solo por cookie (nunca en la URL, evita filtrar el id de sesión).

```

54 |     <welcome-file-list>
55 |         <welcome-file>index.jsp</welcome-file>
56 |     </welcome-file-list>
57 |
58 |     <!-- ===== Paginas de error ===== -->

```

Líneas 54 a 58: Página de inicio del contexto: al abrir <http://host:puerto/logistica/> se carga index.jsp.

```

59 | <error-page>
60 |     <error-code>403</error-code>
61 |     <location>/WEB-INF/jsp/error.jsp</location>
62 | </error-page>
63 | <error-page>
64 |     <error-code>404</error-code>
65 |     <location>/WEB-INF/jsp/error.jsp</location>
66 | </error-page>
67 | <error-page>
68 |     <exception-type>java.lang.Throwable</exception-type>
69 |     <location>/WEB-INF/jsp/error.jsp</location>
70 | </error-page>
71 |
72 | <!-- ===== JSP: UTF-8 y sin scriptlets (todo con JSTL/EL) ===== -->

```

Líneas 59 a 72: Páginas de error amigables: 403 (perfil sin permiso), 404 y cualquier excepción no controlada (Throwable) van a la misma JSP, así el usuario nunca ve un stack trace.

```

73 | <jsp-config>
74 |     <jsp-property-group>
75 |         <url-pattern>*.jsp</url-pattern>
76 |         <url-pattern>*.jspf</url-pattern>
77 |         <page-encoding>UTF-8</page-encoding>
78 |         <scripting-invalid>true</scripting-invalid>
79 |     </jsp-property-group>
80 | </jsp-config>
81 | </web-app>

```

Líneas 73 a 81: Configuración de JSP: codificación UTF-8 y **scripting-invalid**: prohíbe scriptlets Java (<%>) en las vistas, obligando a usar JSTL y EL (punto 09).

Proyecto DAO - dominio (modelo) y excepciones

logistica-dao / src / main / java / ar / edu / usal / logica / excepcion / LogisticaException.java

Para qué existe: Excepción base (checked) de toda la aplicación.

Defensa: Punto 03 (uso correcto de excepciones). Hay una jerarquía propia: los errores técnicos (DAOException) y los de negocio (ValidacionException) se distinguen por tipo, y la capa web decide distinto para cada uno (avisar al usuario vs. mostrar error interno). Son checked para que el compilador obligue a tratarlas y nadie las ignore.

```
1 | package ar.edu.usal.logistica.excepcion;  
2 |
```

Líneas 1 a 2: Declara el paquete de las excepciones.

```
3 | /**  
4 |  * Excepcion base (checked) de la aplicacion. Obliga a quien llama a decidir  
5 |  * que hacer ante un error de negocio o de acceso a datos.  
6 |  */
```

Líneas 3 a 6: Comentario Javadoc: explica el propósito de la clase.

```
7 | public class LogisticaException extends Exception {  
8 |  
9 |     private static final long serialVersionUID = 1L;  
10 | }
```

Líneas 7 a 10: La clase hereda de **Exception** (no de RuntimeException): es una excepción controlada. serialVersionUID es el identificador de versión de serialización, buena práctica para toda Exception.

```
11 |     public LogisticaException(String mensaje) {  
12 |         super(mensaje);  
13 |     }  
14 | }
```

Líneas 11 a 14: Constructor con mensaje: lo pasa a la clase padre con super.

```
15 |     public LogisticaException(String mensaje, Throwable causa) {  
16 |         super(mensaje, causa);  
17 |     }  
18 | }
```

Líneas 15 a 18: Constructor con mensaje y **causa**: permite encadenar la excepción original (por ejemplo una SQLException) sin perder el stack trace.

logistica-dao / src / main / java / ar / edu / usal / logica / excepcion / DAOException.java

Para qué existe: Error técnico al acceder a datos.

Defensa: Envuelve la SQLException: la capa web no debe depender de java.sql. Si se cambiara JDBC por otra tecnología, los controladores no se enterarían. Conserva la causa original para el log.

```
1 | package ar.edu.usal.logistica.excepcion;  
2 |
```

Líneas 1 a 2: Paquete.

```
3 | /**  
4 |  * Error tecnico de acceso a datos (SQLException, driver, conexion, etc.).  
5 |  * Envuelve la excepcion original para no filtrar detalles de JDBC a la capa web.  
6 |  */
```

Líneas 3 a 6: Documentación de la clase.

```
7 | public class DAOException extends LogisticaException {  
8 |  
9 |     private static final long serialVersionUID = 1L;  
10 | }
```

Líneas 7 a 10: Hereda de LogisticaException y define el serialVersionUID.

```

11 |     public DAOException(String mensaje, Throwable causa) {
12 |         super(mensaje, causa);
13 |     }
14 |

```

Líneas 11 a 14: Constructor con mensaje y causa (el caso típico: envolver una SQLException).

```

15 |     public DAOException(String mensaje) {
16 |         super(mensaje);
17 |     }
18 | }

```

Líneas 15 a 18: Constructor solo con mensaje, para errores técnicos sin causa previa.

logistica-dao / src / main / java / ar / edu / usal / logica / excepcion / ValidacionException.java

Para qué existe: Error de regla de negocio o de datos cargados por el usuario.

Defensa: Su mensaje está escrito para el usuario final, por eso los controladores lo muestran tal cual con SweetAlert. Distinguirlo de DAOException evita mostrar detalles técnicos y permite responder con un aviso amable en lugar de una página de error.

```

1 | package ar.edu.usal.logistica.excepcion;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | /**
4 |  * Error de regla de negocio o de datos ingresados por el usuario
5 |  * (DNI duplicado, camion ocupado, categoria insuficiente, etc.).
6 |  * Su mensaje es apto para mostrarse en pantalla.
7 |  */

```

Líneas 3 a 7: Documentación: ejemplos de cuándo se lanza.

```

8 | public class ValidacionException extends LogisticaException {
9 |
10 |     private static final long serialVersionUID = 1L;
11 |

```

Líneas 8 a 11: Hereda de LogisticaException.

```

12 |     public ValidacionException(String mensaje) {
13 |         super(mensaje);
14 |     }
15 | }

```

Líneas 12 a 15: Único constructor: siempre lleva un mensaje legible.

logistica-dao / src / main / java / ar / edu / usal / logica / excepcion / ChoferNoEncontradoException.java

Para qué existe: Se lanza cuando se busca un chofer por DNI y no existe.

Defensa: El enunciado dice literalmente que, si el chofer no existe, «debe informarse en pantalla». Tener un tipo propio permite que el controlador lo atrape por separado y muestre exactamente ese aviso.

```

1 | package ar.edu.usal.logistica.excepcion;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | /**
4 |  * Se lanza cuando se busca un chofer por DNI y no existe.
5 |  * El enunciado pide informarlo en pantalla, por eso tiene su propio tipo.
6 |  */

```

Líneas 3 a 6: Documentación: conecta la clase con el requisito del enunciado.

```

7 | public class ChoferNoEncontradoException extends LogisticaException {
8 |
9 |     private static final long serialVersionUID = 1L;
10 |

```

Líneas 7 a 10: Hereda de LogisticaException.

```

11 |     public ChoferNoEncontradoException(String dni) {
12 |         super("No existe ningún chofer con DNI " + dni + ".");
13 |     }
14 | }

```

Líneas 11 a 14: El constructor recibe el DNI y arma el mensaje completo para el usuario.

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / Categoria.java

Para qué existe: Enum con las categorías de licencia A, B, C y D y las toneladas que habilita cada una.

Defensa: Un enum impide valores inválidos (no existe la categoría «Z») y agrupa dato y comportamiento: la regla «¿esta categoría admite este camión?» vive junto al dato, no dispersa en los controladores (encapsulamiento). Las toneladas por categoría son un supuesto mío, ya que el enunciado no las fija; se cambian en un solo lugar (y en la tabla categoría).

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | /**
4 |  * Categoría de licencia del chofer. Cada categoria habilita a manejar
5 |  * camiones hasta cierta cantidad de toneladas.
6 |  */

```

Líneas 3 a 6: Documentación del enum.

```

7 | public enum Categoria {
8 |     A(10), B(20), C(30), D(40);
9 |

```

Líneas 7 a 9: Las cuatro constantes del enum; cada una se construye con sus toneladas máximas (A=10, B=20, C=30, D=40).

```

10 |     private final int toneladasMaximas;
11 |
12 |     Categoria(int toneladasMaximas) {
13 |         this.toneladasMaximas = toneladasMaximas;
14 |     }
15 |

```

Líneas 10 a 15: Atributo privado y **final** (inmutable) guardado por cada constante, con su constructor.

```

16 |     public int getToneladasMaximas() {
17 |         return toneladasMaximas;
18 |     }
19 |

```

Líneas 16 a 19: Getter del atributo.

```

20 |     /** Indica si un camion de esas toneladas puede ser manejado con esta categoria. */
21 |     public boolean admite(double toneladas) {
22 |         return toneladas <= toneladasMaximas;
23 |     }
24 |

```

Líneas 20 a 24: Método de negocio **admite**: verdadero si las toneladas del camión no superan las de la categoría.

```

25 |     /** Descripcion para mostrar en listas desplegables. */
26 |     public String getDescripcion() {
27 |         return "Categoría " + name() + " (hasta " + toneladasMaximas + " t)";
28 |     }
29 | }

```

Líneas 25 a 29: Texto legible para las listas desplegables, por ejemplo «Categoría C (hasta 30 t)».

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / Destino.java

Para qué existe: Enum con las 8 ciudades a las que transporta la empresa.

Defensa: El enunciado dice «únicamente» estos destinos: un enum lo garantiza en el código y la tabla destino lo garantiza en la base. name() se guarda en la base y getNombre() se muestra en pantalla (con tildes).

```
1 | package ar.edu.usal.logistica.modelo;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | /**
4 |  * Ciudades a las que transporta la empresa (y unicas validas como origen/destino).
5 |  * El nombre del enum (name()) es el codigo que se guarda en la base de datos.
6 |  */
```

Líneas 3 a 6: Documentación.

```
7 | public enum Destino {
8 |     CABA("CABA"),
9 |     CORDOBA("Córdoba"),
10 |    CORRIENTES("Corrientes"),
11 |    FORMOSA("Formosa"),
12 |    LA_PLATA("La Plata"),
13 |    LA_RIOJA("La Rioja"),
14 |    MENDOZA("Mendoza"),
15 |    NEUQUEN("Neuquén");
16 |
```

Líneas 7 a 16: Las ocho ciudades con su nombre para mostrar.

```
17 |     private final String nombre;
18 |
19 |     Destino(String nombre) {
20 |         this.nombre = nombre;
21 |     }
22 |
23 |     public String getNombre() {
24 |         return nombre;
25 |     }
26 | }
```

Líneas 17 a 26: Atributo con el nombre visible, su constructor y su getter.

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / EstadoViaje.java

Para qué existe: Estados por los que pasa un viaje.

Defensa: Modela el ciclo de vida ASIGNADO → EN_CURSO → FINALIZADO del enunciado. El método ocupaCamion() deja la regla «un camión está libre solo cuando su viaje terminó» en un único lugar.

```
1 | package ar.edu.usal.logistica.modelo;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | /** Ciclo de vida de un viaje: ASIGNADO -> EN_CURSO -> FINALIZADO. */
4 | public enum EstadoViaje {
5 |     ASIGNADO("Asignado"),
6 |     EN_CURSO("En curso"),
7 |     FINALIZADO("Finalizado");
8 |
```

Líneas 3 a 8: Documentación y las tres constantes con su descripción visible.

```

9 |     private final String descripcion;
10 |
11 |     EstadoViaje(String descripcion) {
12 |         this.descripcion = descripcion;
13 |     }
14 |

```

Líneas 9 a 14: Atributo y constructor.

```

15 |     public String getDescripcion() {
16 |         return descripcion;
17 |     }
18 |

```

Líneas 15 a 18: Getter usado por las vistas JSP.

```

19 |     /** Un camion esta ocupado mientras su viaje no haya finalizado. */
20 |     public boolean ocupaCamion() {
21 |         return this != FINALIZADO;
22 |     }
23 | }

```

Líneas 19 a 23: Regla de negocio: todo estado distinto de FINALIZADO ocupa el camión.

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / Rol.java

Para qué existe: Perfiles del sistema: ADMIN y CHOFER.

Defensa: Son los dos perfiles del enunciado. Un enum evita comparar cadenas sueltas como «admin» y reduce errores de tipeo.

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | /** Perfiles del sistema. */
4 | public enum Rol {
5 |     ADMIN, CHOFER
6 | }

```

Líneas 3 a 6: El enum con los dos perfiles.

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / Persona.java

Para qué existe: Clase abstracta con los datos y validaciones comunes de una persona.

Defensa: Punto 02 (buenas prácticas de POO): **abstracción y herencia**. Chofer extiende Persona y agrega lo suyo. **Encapsulamiento:** atributos privados, acceso por getters, y la validación vive en la propia clase (validar()). Se normaliza con trim() en el constructor. El método validar() es polimórfico: Chofer lo redefine y llama a super.

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.excepcion.ValidacionException;
4 |
5 | import java.time.LocalDate;
6 | import java.time.Period;
7 | import java.time.format.DateTimeFormatter;
8 |

```

Líneas 3 a 8: Importaciones de las clases que usa este archivo: **ValidacionException, LocalDate, Period, DateTimeFormatter**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

9  | /**
10 |  * Clase abstracta con los datos comunes a cualquier persona del dominio.
11 |  * Hoy solo la especializa Chofer, pero deja el diseno listo para sumar
12 |  * otras personas (por ejemplo, un despachante) sin duplicar codigo.
13 |  */

```

Líneas 9 a 13: Documentación: por qué es abstracta.

```

14 | public abstract class Persona {
15 |
16 |     private static final DateTimeFormatter FORMATO = DateTimeFormatter.ofPattern("dd/MM/yyyy");
17 |
18 |     private Long id;
19 |     private String nombre;
20 |     private String apellido;
21 |     private String dni;
22 |     private LocalDate fechaNacimiento;
23 |     private String telefono;
24 |

```

Líneas 14 a 24: Clase **abstracta**: no se puede instanciar, solo heredar. Guarda un formateador de fecha estático y los atributos privados: id, nombre, apellido, dni, fecha de nacimiento y teléfono.

```

25 |     protected Persona(Long id, String nombre, String apellido, String dni,
26 |                       LocalDate fechaNacimiento, String telefono) {
27 |         this.id = id;
28 |         this.nombre = nombre == null ? null : nombre.trim();
29 |         this.apellido = apellido == null ? null : apellido.trim();
30 |         this.dni = dni == null ? null : dni.trim();
31 |         this.fechaNacimiento = fechaNacimiento;
32 |         this.telefono = telefono == null ? null : telefono.trim();
33 |     }
34 |

```

Líneas 25 a 34: Constructor **protected** (solo subclases). Guarda cada dato y aplica **trim()** a los textos, cuidando el null, para no almacenar espacios accidentales.

```

35 |     /** Valida los datos personales. Las subclases la extienden con sus propias reglas. */
36 |     public void validar() throws ValidacionException {

```

Líneas 35 a 36: Método validar(): lanza ValidacionException con un mensaje claro si algo está mal.

```

37 |         if (nombre == null || nombre.isEmpty() || nombre.length() > 50) {
38 |             throw new ValidacionException("El nombre es obligatorio (máximo 50 caracteres).");
39 |         }
40 |         if (apellido == null || apellido.isEmpty() || apellido.length() > 50) {
41 |             throw new ValidacionException("El apellido es obligatorio (máximo 50 caracteres).");
42 |         }

```

Líneas 37 a 42: Nombre y apellido obligatorios y de hasta 50 caracteres (coincide con la columna de la base).

```

43 |         if (dni == null || !dni.matches("\\d{7,8}")) {
44 |             throw new ValidacionException("El DNI debe tener 7 u 8 dígitos numéricos.");
45 |         }

```

Líneas 43 a 45: El DNI debe ser solo dígitos, 7 u 8 (expresión regular **\d{7,8}**).

```

46 |         if (fechaNacimiento == null) {
47 |             throw new ValidacionException("La fecha de nacimiento es obligatoria.");
48 |         }
49 |         if (getEdad() < 18) {
50 |             throw new ValidacionException("El chofer debe ser mayor de 18 años.");
51 |         }
52 |         if (getEdad() > 100) {
53 |             throw new ValidacionException("La fecha de nacimiento no es válida.");
54 |         }

```

Líneas 46 a 54: La fecha es obligatoria; además se exige ser mayor de 18 años (un chofer profesional) y una edad razonable (máx. 100) para detectar fechas absurdas.

```

55 |         if (telefono == null || !telefono.matches("\\d{8,15}")) {
56 |             throw new ValidacionException("El teléfono celular debe tener entre 8 y 15 dígitos, sin espacios ni
57 |                 guiones.");
58 |         }
59 |     }

```

Líneas 55 a 59: Teléfono celular: solo números, entre 8 y 15 dígitos. El enunciado aclara que solo interesa el celular.

```

60 |     public int getEdad() {
61 |         return Period.between(fechaNacimiento, LocalDate.now()).getYears();
62 |     }
63 |

```

Líneas 60 a 63: Calcula la edad con `java.time.Period` entre la fecha de nacimiento y hoy.

```

64 |     public String getNombreCompleto() {
65 |         return apellido + ", " + nombre;
66 |     }
67 |

```

Líneas 64 a 67: «Apellido, Nombre» para mostrar en listas.

```

68 |     public String getFechaNacimientoFormateada() {
69 |         return fechaNacimiento == null ? "" : fechaNacimiento.format(FORMATO);
70 |     }
71 |

```

Líneas 68 a 71: Fecha en formato dd/MM/yyyy para las vistas (JSP no formatea `LocalDate`).

```

72 |     public Long getId() { return id; }
73 |     public void setId(Long id) { this.id = id; }
74 |     public String getNombre() { return nombre; }
75 |     public String getApellido() { return apellido; }
76 |     public String getDni() { return dni; }
77 |     public LocalDate getFechaNacimiento() { return fechaNacimiento; }
78 |     public String getTelefono() { return telefono; }
79 | }

```

Líneas 72 a 79: Getters y setter de id. El id es **Long** (no long) porque puede ser null antes de guardar en la base. El resto de los atributos son de solo lectura (sin setters): objeto casi inmutable.

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / Camion.java

Para qué existe: Entidad Camión con sus reglas y cálculos propios.

Defensa: Encapsula el dato y la lógica: `litrosNecesarios(km)` es conocimiento del camión (su consumo), por eso está acá y no en el servlet. El dominio se normaliza a mayúsculas y sin espacios para que «ab 123 cd» y «AB123CD» sean la misma patente (y el UNIQUE de la base funcione).

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.excepcion.ValidacionException;
4 |
5 | import java.util.Locale;
6 |

```

Líneas 3 a 6: Importaciones de las clases que usa este archivo: **ValidacionException**, **Locale**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

7 | /** Camion de la flota. */
8 | public class Camion {
9 |
10 |     private Long id;
11 |     private String marca;
12 |     private String modelo;
13 |     private String dominio;
14 |     private double toneladasMaximas;
15 |     private double capacidadTanqueLitros;
16 |     private double consumoLitrosPorKm;
17 |

```

Líneas 7 a 17: Declaración de la clase y sus atributos privados: los datos pedidos en el enunciado (marca, modelo, dominio, toneladas máximas, litros del tanque y litros por km).

```

18 |     public Camion(Long id, String marca, String modelo, String dominio,
19 |                   double toneladasMaximas, double capacidadTanqueLitros, double consumoLitrosPorKm) {
20 |         this.id = id;
21 |         this.marca = marca == null ? null : marca.trim();
22 |         this.modelo = modelo == null ? null : modelo.trim();
23 |         // El dominio (patente) se normaliza siempre en mayusculas y sin espacios
24 |         this.dominio = dominio == null ? null : dominio.trim().toUpperCase(Locale.ROOT).replace(" ", "");
25 |         this.toneladasMaximas = toneladasMaximas;
26 |         this.capacidadTanqueLitros = capacidadTanqueLitros;
27 |         this.consumoLitrosPorKm = consumoLitrosPorKm;
28 |     }
29 |

```

Líneas 18 a 29: Constructor: guarda los datos, aplicando trim() y normalizando el dominio a mayúsculas sin espacios.

```

30 |     public void validar() throws ValidacionException {
31 |         if (marca == null || marca.isEmpty() || marca.length() > 40) {
32 |             throw new ValidacionException("La marca es obligatoria (máximo 40 caracteres).");
33 |         }
34 |         if (modelo == null || modelo.isEmpty() || modelo.length() > 40) {
35 |             throw new ValidacionException("El modelo es obligatorio (máximo 40 caracteres).");
36 |         }
37 |         // Formato argentino vigente (AB123CD) o anterior (ABC123)
38 |         if (dominio == null || !dominio.matches("[A-Z]{2}\\d{3}[A-Z]{2}|[A-Z]{3}\\d{3}")) {
39 |             throw new ValidacionException("El dominio debe tener formato AB123CD o ABC123.");
40 |         }
41 |         if (toneladasMaximas <= 0 || toneladasMaximas > 99) {
42 |             throw new ValidacionException("Las toneladas máximas deben ser mayores a 0 y menores a 100.");
43 |         }
44 |         if (capacidadTanqueLitros <= 0 || capacidadTanqueLitros > 9999) {
45 |             throw new ValidacionException("La capacidad del tanque debe ser mayor a 0 litros.");
46 |         }
47 |         if (consumoLitrosPorKm <= 0 || consumoLitrosPorKm > 99) {
48 |             throw new ValidacionException("El consumo por km debe ser mayor a 0 litros.");
49 |         }
50 |     }
51 |

```

Líneas 30 a 51: Validaciones de negocio: marca y modelo obligatorios (máx. 40), dominio con formato argentino AB123CD o ABC123 (expresión regular), toneladas entre 0 y 99, tanque y consumo positivos. Cada regla lanza una ValidacionException con mensaje claro.

```

52 |     /** Litros de combustible necesarios para recorrer una cantidad de km. */
53 |     public double litrosNecesarios(int km) {
54 |         return km * consumoLitrosPorKm;
55 |     }
56 |

```

Líneas 52 a 56: litrosNecesarios: km × consumo por km. Lo usa la calculadora de viajes.

```

57 |     /** Descripcion corta para listas: "Scania R450 (AB123CD) - 28 t". */
58 |     public String getDescripcion() {
59 |         return String.format(Locale.ROOT, "%s %s (%s) - %.1f t", marca, modelo, dominio, toneladasMaximas);
60 |     }
61 |

```

Líneas 57 a 61: Texto de una línea para listas: «Scania R450 (AB123CD) - 28.0 t». Usa Locale.ROOT para que el punto decimal no cambie según el idioma del servidor.

```

62 |     public Long getId() { return id; }
63 |     public void setId(Long id) { this.id = id; }
64 |     public String getMarca() { return marca; }
65 |     public String getModelo() { return modelo; }
66 |     public String getDominio() { return dominio; }
67 |     public double getToneladasMaximas() { return toneladasMaximas; }
68 |     public double getCapacidadTanqueLitros() { return capacidadTanqueLitros; }
69 |     public double getConsumoLitrosPorKm() { return consumoLitrosPorKm; }
70 | }

```

Líneas 62 a 70: Getters (y setId para asignar el id generado por la base).

logistica-dao / src / main / java / ar / edu / usal / logica / modelo / Chofer.java

Para qué existe: Chofer: una Persona con categoría y camiones autorizados.

Defensa: Herencia (extends Persona), **polimorfismo** (@Override de validar()), **composición** (tiene camiones autorizados) y **encapsulamiento**: la lista interna es privada y se expone como solo lectura (Collections.unmodifiableList); para agregar hay que pasar por autorizar(). Las reglas «puede manejar este camión» viven en el propio chofer.

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.excepcion.ValidacionException;
4 |
5 | import java.time.LocalDate;
6 | import java.util.ArrayList;
7 | import java.util.Collections;
8 | import java.util.List;
9 |

```

Líneas 3 a 9: Importaciones de las clases que usa este archivo: **ValidacionException**, **LocalDate**, **ArrayList**, **Collections**, **List**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

10 | /**
11 |  * Chofer de la empresa. Hereda los datos personales de Persona y agrega su
12 |  * categoria y los camiones que esta autorizado a manejar.
13 |  */

```

Líneas 10 a 13: Documentación de la clase.

```

14 | public class Chofer extends Persona {
15 |
16 |     private Categoria categoria;
17 |     // Lista interna modificable solo a traves de autorizar(); se expone de solo lectura
18 |     private final List<Camion> camionesAutorizados = new ArrayList<>();
19 |

```

Líneas 14 a 19: Hereda de Persona. Atributos propios: la categoría y la lista de camiones autorizados (**final**: la referencia no cambia, y es privada).

```

20 |     public Chofer(Long id, String nombre, String apellido, String dni,
21 |                   LocalDate fechaNacimiento, Categoria categoria, String telefono) {
22 |         super(id, nombre, apellido, dni, fechaNacimiento, telefono);
23 |         this.categoria = categoria;
24 |     }
25 |

```

Líneas 20 a 25: Constructor: pasa los datos personales a **super** y guarda la categoría.

```

26 | @Override
27 | public void validar() throws ValidacionException {
28 |     super.validar();
29 |     if (categoria == null) {
30 |         throw new ValidacionException("La categoría es obligatoria.");
31 |     }
32 |     for (Camion camion : camionesAutorizados) {
33 |         if (!categoria.admite(camion.getToneladasMaximas())) {
34 |             throw new ValidacionException("La categoría " + categoria + " no permite manejar el camión "
35 |                 + camion.getDescripcion() + ".");
36 |         }
37 |     }
38 | }
39 |

```

Líneas 26 a 39: validar() redefinido: primero las reglas de Persona (super.validar()), luego exige categoría y verifica que cada camión autorizado entre en la categoría: no se puede autorizar a un chofer categoría A a manejar un camión de 28 t.

```

40 | /** Agrega un camion a la lista de autorizados (sin repetir). */
41 | public void autorizar(Camion camion) {
42 |     if (!camionesAutorizados.contains(camion)
43 |         && camionesAutorizados.stream().noneMatch(c -> c.getId() != null &&
44 |             c.getId().equals(camion.getId()))) {
45 |         camionesAutorizados.add(camion);
46 |     }
47 | }

```

Líneas 40 a 47: autorizar(): agrega un camión a la lista solo si no estaba ya (evita duplicados por identidad o por id).

```

48 | /** Un chofer puede manejar un camion si esta autorizado y su categoria alcanza. */
49 | public boolean puedeManejar(Camion camion) {
50 |     boolean autorizado = camionesAutorizados.stream()
51 |         .anyMatch(c -> c.getId() != null && c.getId().equals(camion.getId()));
52 |     return autorizado && categoria.admite(camion.getToneladasMaximas());
53 | }
54 |

```

Líneas 48 a 54: puedeManejar(): verdadero si el camión está autorizado y además la categoría alcanza. Es la regla central que usa Viaje al validarse.

```

55 | public boolean estaAutorizadoPara(long camionId) {
56 |     return camionesAutorizados.stream().anyMatch(c -> c.getId() != null && c.getId() == camionId);
57 | }
58 |

```

Líneas 55 a 58: Consulta por id de camión.

```

59 | public Categoria getCategoria() { return categoria; }
60 | public List<Camion> getCamionesAutorizados() { return Collections.unmodifiableList(camionesAutorizados); }
61 | }

```

Líneas 59 a 61: Getters; la lista se entrega como vista de solo lectura para que nadie la modifique desde afuera.

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / Usuario.java

Para qué existe: Usuario que inicia sesión.

Defensa: Clase inmutable (todos los atributos final, sin setters): un usuario autenticado no debería cambiar mientras vive en la Session. Guarda el hash de la contraseña, nunca la contraseña.

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | /** Usuario que inicia sesion. Un usuario CHOFER esta ligado a un chofer concreto. */
4 | public class Usuario {
5 |
6 |     private final Long id;
7 |     private final String username;
8 |     private final String passwordHash;
9 |     private final Rol rol;
10 |    private final Long choferId;
11 |

```

Líneas 3 a 11: Atributos finales: id, username, hash de la contraseña, rol y (si es chofer) el id del chofer asociado.

```

12 |    public Usuario(Long id, String username, String passwordHash, Rol rol, Long choferId) {
13 |        this.id = id;
14 |        this.username = username;
15 |        this.passwordHash = passwordHash;
16 |        this.rol = rol;
17 |        this.choferId = choferId;
18 |    }
19 |

```

Líneas 12 a 19: Constructor que asigna todos los atributos.

```

20 |    public boolean esAdmin() {
21 |        return rol == Rol.ADMIN;
22 |    }
23 |

```

Líneas 20 a 23: Método de conveniencia usado por filtro y vistas (usuario.esAdmin()).

```

24 |    public Long getId() { return id; }
25 |    public String getUsername() { return username; }
26 |    public String getPasswordHash() { return passwordHash; }
27 |    public Rol getRol() { return rol; }
28 |    public Long getChoferId() { return choferId; }
29 | }

```

Líneas 24 a 29: Getters.

logistica-dao / src / main / java / ar / edu / usal / logistica / modelo / EstimacionViaje.java

Para qué existe: Resultado del cálculo de un viaje.

Defensa: Clase **immutable** de solo datos (atributos final, sin setters) con constructor, accesoros, equals, hashCode y toString. Como el proyecto usa Java 11 no se usa record. Agrupar los cuatro resultados evita pasar cuatro parámetros sueltos.

```

1 | package ar.edu.usal.logistica.modelo;
2 |
3 | import java.util.Objects;
4 |

```

Líneas 1 a 4: Paquete.

```

5 | /**
6 |  * Resultado del calculo de un viaje (objeto immutable).
7 |  * Los accesoros se llaman igual que el atributo (distanciaKm(), dias()...) y no getX(),
8 |  * porque así los usan las vistas JSP.
9 |  */

```

Líneas 5 a 9: Documentación de la clase y de cada parámetro del constructor.

```

10 | public final class EstimacionViaje {
11 |
12 |     private final int distanciaKm;
13 |     private final int dias;
14 |     private final double litrosTotales;
15 |     private final int tanques;
16 |
17 |     /**
18 |      * @param distanciaKm    kilometros entre origen y destino
19 |      * @param dias          dias de viaje (a 200 km por dia)
20 |      * @param litrosTotales combustible necesario para todo el recorrido
21 |      * @param tanques       cantidad de tanques a llenar (redondeado hacia arriba)
22 |      */
23 |     public EstimacionViaje(int distanciaKm, int dias, double litrosTotales, int tanques) {
24 |         this.distanciaKm = distanciaKm;
25 |         this.dias = dias;
26 |         this.litrosTotales = litrosTotales;
27 |         this.tanques = tanques;
28 |     }
29 |

```

Líneas 10 a 29: Clase final e inmutable con cuatro atributos: kilómetros, días, litros totales y tanques; se llenan una sola vez en el constructor.

```

30 |     public int distanciaKm() { return distanciaKm; }
31 |     public int dias() { return dias; }
32 |     public double litrosTotales() { return litrosTotales; }
33 |     public int tanques() { return tanques; }
34 |
35 |     @Override

```

Líneas 30 a 35: Accesores con el mismo nombre del atributo (los JSP los llaman como métodos en EL).

```

36 |     public boolean equals(Object otro) {
37 |         if (this == otro) {
38 |             return true;
39 |         }
40 |         if (!(otro instanceof EstimacionViaje)) {
41 |             return false;
42 |         }
43 |         EstimacionViaje e = (EstimacionViaje) otro;
44 |         return distanciaKm == e.distanciaKm && dias == e.dias
45 |             && Double.compare(litrosTotales, e.litrosTotales) == 0 && tanques == e.tanques;
46 |     }
47 |
48 |     @Override
49 |     public int hashCode() {
50 |         return Objects.hash(distanciaKm, dias, litrosTotales, tanques);
51 |     }
52 |
53 |     @Override
54 |     public String toString() {
55 |         return "EstimacionViaje[distanciaKm=" + distanciaKm + ", dias=" + dias
56 |             + ", litrosTotales=" + litrosTotales + ", tanques=" + tanques + "];"
57 |     }
58 | }

```

Líneas 36 a 58: equals, hashCode y toString escritos a mano: dos estimaciones con los mismos valores son iguales.

logistica-dao / src / main / java / ar / edu / usal / logica / modelo / CalculadoraViaje.java

Para qué existe: Lógica del cálculo que pide el enunciado: tiempo de viaje y cantidad de tanques.

Defensa: Es una clase sin estado y final con método estático: lógica pura, fácil de probar (ver los tests).

Tiempo: a 200 km por día, redondeado hacia arriba (646 km → 3,23 → 4 días; un viaje no termina a mitad de día). **Tanques:** litros totales = km × consumo; tanques = litros / capacidad, redondeado hacia arriba (hay que llenar el último tanque aunque sea parcial). Se calcula en el servidor y no en JavaScript, para que nadie pueda manipular el resultado.

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | /**
4 |  * Logica de calculo del viaje pedida en el enunciado:
5 |  * <ul>
6 |  *   <li>Tiempo: por dia se recorren 200 km, se redondea hacia arriba.</li>
7 |  *   <li>Tanques: litros totales (km x consumo por km) / capacidad del tanque, hacia arriba.</li>
8 |  * </ul>
9 |  * Es una clase sin estado, por eso su metodo es estatico y se puede probar sola.
10 | */

```

Líneas 3 a 10: Documentación con las dos fórmulas.

```

11 | public final class CalculadoraViaje {
12 |
13 |     /** Kilometros que recorre el camion por dia (dato del enunciado). */
14 |     public static final int KM_POR_DIA = 200;
15 |

```

Líneas 11 a 15: Clase **final** (no se hereda) y la constante KM_POR_DIA = 200, dato del enunciado, definida una sola vez (sin «números mágicos»).

```

16 |     private CalculadoraViaje() {
17 |         // Clase utilitaria: no se instancia
18 |     }
19 |

```

Líneas 16 a 19: Constructor privado: es una clase utilitaria, no se instancia.

```

20 |     public static EstimacionViaje calcular(Camion camion, int distanciaKm) {
21 |         if (distanciaKm <= 0) {
22 |             throw new IllegalArgumentException("La distancia debe ser mayor a cero.");
23 |         }

```

Líneas 20 a 23: Método principal. Primero valida que la distancia sea positiva (error de programación → `IllegalArgumentException`).

```

24 |         int dias = (int) Math.ceil((double) distanciaKm / KM_POR_DIA);

```

Línea 24: Días = techo(km / 200). El casteo a double evita la división entera que truncaría.

```

25 |         double litros = camion.litrosNecesarios(distanciaKm);
26 |         int tanques = (int) Math.ceil(litros / camion.getCapacidadTanqueLitros());
27 |         return new EstimacionViaje(distanciaKm, dias, litros, tanques);
28 |     }
29 | }

```

Líneas 25 a 29: Litros totales según el consumo del camión y tanques = techo(litros / capacidad del tanque). Devuelve todo en un `EstimacionViaje`.

logistica-dao / src / main / java / ar / edu / usal / logica / modelo / Viaje.java

Para qué existe: Viaje asignado a un chofer y un camión, con su estimación y su estado.

Defensa: Método fábrica estático nuevo(): centraliza la creación de un viaje aplicando el cálculo, con estado inicial ASIGNADO. `validar()` concentra las reglas de negocio (origen ≠ destino, chofer habilitado para ese camión). Los métodos `puedeIniciarse()`/`puedeFinalizarse()` evitan que las vistas repitan la lógica de estados.

```

1 | package ar.edu.usal.logistica.modelo;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.excepcion.ValidacionException;
4 |
5 | import java.time.LocalDateTime;
6 | import java.time.format.DateTimeFormatter;
7 |

```

Líneas 3 a 7: Importaciones de las clases que usa este archivo: **ValidacionException**, **LocalDateTime**, **DateTimeFormatter**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

8  | /** Viaje cargado por el administrador y asignado a un chofer y un camion. */
9  | public class Viaje {
10 |
11 |     private static final DateTimeFormatter FORMATO = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
12 |
13 |     private Long id;
14 |     private final Chofer chofer;
15 |     private final Camion camion;
16 |     private final Destino origen;
17 |     private final Destino destino;
18 |     private final EstimacionViaje estimacion;
19 |     private EstadoViaje estado;
20 |     private LocalDateTime fechaCarga;
21 |     private LocalDateTime fechaInicio;
22 |     private LocalDateTime fechaFin;
23 |

```

Líneas 8 a 23: Atributos: id, chofer, camión, origen, destino y estimación (todos **final**, inmutables), más estado y fechas (que cambian durante la vida del viaje).

```

24 |     public Viaje(Long id, Chofer chofer, Camion camion, Destino origen, Destino destino,
25 |                 EstimacionViaje estimacion, EstadoViaje estado,
26 |                 LocalDateTime fechaCarga, LocalDateTime fechaInicio, LocalDateTime fechaFin) {
27 |         this.id = id;
28 |         this.chofer = chofer;
29 |         this.camion = camion;
30 |         this.origen = origen;
31 |         this.destino = destino;
32 |         this.estimacion = estimacion;
33 |         this.estado = estado;
34 |         this.fechaCarga = fechaCarga;
35 |         this.fechaInicio = fechaInicio;
36 |         this.fechaFin = fechaFin;
37 |     }
38 |

```

Líneas 24 a 38: Constructor completo, usado por el DAO al leer viajes de la base.

```

39 |     /** Crea un viaje nuevo (estado ASIGNADO) aplicando el calculo de tiempo y tanques. */
40 |     public static Viaje nuevo(Chofer chofer, Camion camion, Destino origen, Destino destino, int distanciaKm) {
41 |         EstimacionViaje estimacion = CalculadoraViaje.calcular(camion, distanciaKm);
42 |         return new Viaje(null, chofer, camion, origen, destino, estimacion,
43 |                         EstadoViaje.ASIGNADO, LocalDateTime.now(), null, null);
44 |     }
45 |

```

Líneas 39 a 45: Método fábrica **nuevo**: calcula la estimación con CalculadoraViaje y crea el viaje ASIGNADO.

```

46 |     /** Reglas que debe cumplir un viaje antes de guardarse. */
47 |     public void validar() throws ValidacionException {
48 |         if (chofer == null || chofer.getId() == null) {
49 |             throw new ValidacionException("Debe seleccionar un chofer.");
50 |         }
51 |         if (camion == null || camion.getId() == null) {
52 |             throw new ValidacionException("Debe seleccionar un camión.");
53 |         }
54 |         if (origen == null || destino == null) {
55 |             throw new ValidacionException("Debe seleccionar origen y destino.");
56 |         }
57 |         if (origen == destino) {
58 |             throw new ValidacionException("El origen y el destino no pueden ser iguales.");
59 |         }
60 |         if (!chofer.puedeManejar(camion)) {
61 |             throw new ValidacionException("El chofer no está autorizado a manejar ese camión.");
62 |         }
63 |     }
64 |

```

Líneas 46 a 64: validar(): exige chofer, camión, origen y destino; origen distinto de destino; y que el chofer pueda manejar ese camión (autorizado + categoría).

```

65 |     public boolean puedeIniciarse() { return estado == EstadoViaje.ASIGNADO; }
66 |     public boolean puedeFinalizarse() { return estado == EstadoViaje.EN_CURSO; }
67 |

```

Líneas 65 a 67: Solo un viaje ASIGNADO puede iniciarse y solo uno EN_CURSO puede finalizarse.

```

68 |     public String getFechaCargaFormateada() { return formatear(fechaCarga); }
69 |     public String getFechaInicioFormateada() { return formatear(fechaInicio); }
70 |     public String getFechaFinFormateada() { return formatear(fechaFin); }
71 |
72 |     private static String formatear(LocalDateTime fecha) {
73 |         return fecha == null ? "-" : fecha.format(FORMATO);
74 |     }
75 |

```

Líneas 68 a 75: Fechas como texto dd/MM/yyyy HH:mm, o «-» si todavía no ocurrió.

```

76 |     public Long getId() { return id; }
77 |     public void setId(Long id) { this.id = id; }
78 |     public Chofer getChofer() { return chofer; }
79 |     public Camion getCamion() { return camion; }
80 |     public Destino getOrigen() { return origen; }
81 |     public Destino getDestino() { return destino; }
82 |     public EstimacionViaje getEstimacion() { return estimacion; }
83 |     public EstadoViaje getEstado() { return estado; }
84 |     public LocalDateTime getFechaCarga() { return fechaCarga; }
85 |     public LocalDateTime getFechaInicio() { return fechaInicio; }
86 |     public LocalDateTime getFechaFin() { return fechaFin; }
87 | }

```

Líneas 76 a 87: Getters.

logistica-dao / src / main / java / ar / edu / usal / logistica / util / PasswordUtil.java

Para qué existe: Hash y verificación seguros de contraseñas, y generación de tokens.

Defensa: Nunca se guardan contraseñas en texto plano. Se usa **PBKDF2-HMAC-SHA256** (incluido en el JDK) con **sal aleatoria** por usuario (dos usuarios con la misma clave tienen hashes distintos) y 65.536 iteraciones (lo vuelve lento de atacar por fuerza bruta). La comparación es en tiempo constante (MessageDigest.isEqual). SecureRandom es el generador criptográficamente seguro, necesario para sal y tokens.

```

1 | package ar.edu.usal.logistica.util;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import java.security.GeneralSecurityException;
4 | import java.security.MessageDigest;
5 | import java.security.SecureRandom;
6 | import java.util.Base64;
7 |
8 | import javax.crypto.SecretKeyFactory;
9 | import javax.crypto.spec.PBEKeySpec;
10 |

```

Líneas 3 a 10: Importaciones de las clases que usa este archivo: **GeneralSecurityException**, **MessageDigest**, **SecureRandom**, **Base64**, **SecretKeyFactory**, **PBEKeySpec**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

11 | /**
12 |  * Manejo seguro de contraseñas: nunca se guardan en texto plano.
13 |  * Se usa PBKDF2 con sal aleatoria. Formato guardado: iteraciones$sal$hash (Base64).
14 |  */

```

Líneas 11 a 14: Documentación del formato guardado: iteraciones\$sal\$hash.

```

15 | public final class PasswordUtil {
16 |
17 |     private static final int ITERACIONES = 65_536;
18 |     private static final int LONGITUD_CLAVE_BITS = 256;
19 |     private static final SecureRandom RANDOM = new SecureRandom();
20 |

```

Líneas 15 a 20: Constantes: iteraciones, tamaño de clave y un único SecureRandom compartido.

```

21 |     private PasswordUtil() {
22 |     }
23 |

```

Líneas 21 a 23: Constructor privado: clase utilitaria.

```

24 |     public static String hashear(String password) {
25 |         byte[] sal = new byte[16];
26 |         RANDOM.nextBytes(sal);
27 |         byte[] hash = pbkdf2(password, sal, ITERACIONES);
28 |         return ITERACIONES + "$" + Base64.getEncoder().encodeToString(sal)
29 |             + "$" + Base64.getEncoder().encodeToString(hash);
30 |     }
31 |

```

Líneas 24 a 31: Genera 16 bytes de sal aleatoria, calcula el hash y devuelve «iteraciones\$sal\$hash» con sal y hash en Base64.

```

32 |     public static boolean verificar(String password, String guardado) {
33 |         if (password == null || guardado == null) {
34 |             return false;
35 |         }
36 |         String[] partes = guardado.split("\\$");
37 |         if (partes.length != 3) {
38 |             return false;
39 |         }
40 |         byte[] sal = Base64.getDecoder().decode(partes[1]);
41 |         byte[] esperado = Base64.getDecoder().decode(partes[2]);
42 |         byte[] calculado = pbkdf2(password, sal, Integer.parseInt(partes[0]));
43 |         // Comparacion en tiempo constante para evitar ataques de timing
44 |         return MessageDigest.isEqual(esperado, calculado);
45 |     }
46 |

```

Líneas 32 a 46: Separa el valor guardado en sus tres partes, recalcula el hash con la misma sal e iteraciones y lo compara en tiempo constante (evita ataques por medición de tiempos). Devuelve false ante datos nulos o mal formados.

```

47 |     /** Token aleatorio de 64 caracteres hexadecimales para la cookie "recordarme". */
48 |     public static String generarToken() {
49 |         byte[] bytes = new byte[32];
50 |         RANDOM.nextBytes(bytes);
51 |         StringBuilder hex = new StringBuilder(bytes.length * 2);
52 |         for (byte b : bytes) {
53 |             hex.append(String.format("%02x", b));
54 |         }
55 |         return hex.toString();
56 |     }
57 |

```

Líneas 47 a 57: Token de 32 bytes aleatorios en hexadecimal (64 caracteres) para la cookie «recordarme»: imposible de adivinar.

```

58 |     private static byte[] pbkdf2(String password, byte[] sal, int iteraciones) {
59 |         try {
60 |             PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), sal, iteraciones, LONGITUD_CLAVE_BITS);
61 |             return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded();
62 |         } catch (GeneralSecurityException e) {
63 |             throw new IllegalStateException("No se pudo calcular el hash de la contraseña", e);
64 |         }
65 |     }
66 | }

```

Líneas 58 a 66: Implementación interna: arma la especificación (clave, sal, iteraciones, bits) y obtiene el hash con SecretKeyFactory.

logistica-dao / src / main / java / ar / edu / usal / logica / dao / ChoferDAO.java

Para qué existe: Contrato (interfaz) de acceso a datos de choferes.

Defensa: Patrón **DAO**: separa «qué se puede hacer» (interfaz) de «cómo se hace con JDBC» (implementación). Los controladores solo conocen esta interfaz, lo que baja el acoplamiento y permite cambiar la tecnología o testear con un falso. Cada método declara las excepciones que puede lanzar.

```

1 | package ar.edu.usal.logistica.dao;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.excepcion.ChoferNoEncontradoException;
4 | import ar.edu.usal.logistica.excepcion.DAOException;
5 | import ar.edu.usal.logistica.excepcion.ValidacionException;
6 | import ar.edu.usal.logistica.modelo.Chofer;
7 |
8 | import java.util.List;
9 |

```

Líneas 3 a 9: Importaciones de las clases que usa este archivo: **ChoferNoEncontradoException**, **DAOException**, **ValidacionException**, **Chofer**, **List**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

10 | /** Contrato de acceso a datos de choferes (y de su usuario de acceso al sistema). */
11 | public interface ChoferDAO {
12 |
13 |     /** Da de alta el chofer, sus camiones autorizados y su usuario, todo en una transaccion. */

```

Líneas 10 a 13: Interfaz ChoferDAO.

```

14 |         void insertar(Chofer chofer, String passwordInicial) throws DAOException, ValidacionException;
15 |
16 |         /** Modifica el chofer y reemplaza sus camiones autorizados, en una transaccion. */

```

Líneas 14 a 16: Alta del chofer + camiones autorizados + usuario en **una sola transacción**.

```

17 |         void actualizar(Chofer chofer) throws DAOException, ValidacionException;
18 |
19 |         /** Elimina el chofer, su usuario y sus autorizaciones (falla si tiene viajes). */

```

Líneas 17 a 19: Modificación del chofer y reemplazo de sus autorizaciones, también transaccional.

```

20 |         void eliminar(long id) throws DAOException, ValidacionException;
21 |

```

Líneas 20 a 21: Baja del chofer; falla con ValidacionException si tiene viajes.

```

22 |         Chofer buscarPorId(long id) throws DAOException;
23 |
24 |         /** @throws ChoferNoEncontradoException si no existe un chofer con ese DNI */
25 |         Chofer buscarPorDni(String dni) throws DAOException, ChoferNoEncontradoException;
26 |
27 |         List<Chofer> listarTodos() throws DAOException;
28 |     }

```

Líneas 22 a 28: Consultas: por id (null si no existe), por DNI (lanza ChoferNoEncontradoException) y listado completo.

logistica-dao / src / main / java / ar / edu / usal / logistica / dao / CamionDAO.java

Para qué existe: Contrato de acceso a datos de camiones.

Defensa: Incluye el ABM completo y una consulta especial, listarDisponiblesParaChofer, que resuelve el requisito «mostrar los camiones que ese chofer puede manejar y que no estén en viaje».

```

1 | package ar.edu.usal.logistica.dao;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.excepcion.DAOException;
4 | import ar.edu.usal.logistica.excepcion.ValidacionException;
5 | import ar.edu.usal.logistica.modelo.Camion;
6 |
7 | import java.util.List;
8 |

```

Líneas 3 a 8: Importaciones de las clases que usa este archivo: **DAOException**, **ValidacionException**, **Camion**, **List**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

9 | /** Contrato de acceso a datos de camiones. */
10 | public interface CamionDAO {
11 |
12 |     void insertar(Camion camion) throws DAOException, ValidacionException;
13 |
14 |     void actualizar(Camion camion) throws DAOException, ValidacionException;
15 |
16 |     void eliminar(long id) throws DAOException, ValidacionException;
17 |

```

Líneas 9 a 17: Interfaz CamionDAO con altas, bajas y modificaciones.

```

18 |     Camion buscarPorId(long id) throws DAOException;
19 |
20 |     List<Camion> listarTodos() throws DAOException;
21 |

```

Líneas 18 a 21: Consultas por id y listado.

```

22 |     /** Camiones que el chofer puede manejar y que no estan en viaje. */
23 |     List<Camion> listarDisponiblesParaChofer(long choferId) throws DAOException;
24 | }

```

Líneas 22 a 24: Camiones disponibles para un chofer (implementado con un stored procedure).

logistica-dao / src / main / java / ar / edu / usal / logica / dao / UsuarioDAO.java

Para qué existe: Contrato de acceso a usuarios y tokens de sesión recordada.

Defensa: Es lo mínimo necesario para el login y la cookie «recordarme».

```

1 | package ar.edu.usal.logistica.dao;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.excepcion.DAOException;
4 | import ar.edu.usal.logistica.modelo.Usuario;
5 |
6 | import java.time.LocalDateTime;
7 |

```

Líneas 3 a 7: Importaciones de las clases que usa este archivo: **DAOException**, **Usuario**, **LocalDateTime**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

8 | /** Contrato de acceso a datos de usuarios y de los tokens de "recordarme". */
9 | public interface UsuarioDAO {
10 |

```

Líneas 8 a 10: Interfaz UsuarioDAO.

```

11 |     /** @return el usuario o null si no existe */
12 |     Usuario buscarPorUsername(String username) throws DAOException;
13 |

```

Líneas 11 a 13: Buscar usuario por nombre (para el login).

```

14 |     void guardarToken(String token, long usuarioId, LocalDateTime expira) throws DAOException;
15 |

```

Líneas 14 a 15: Guardar un token de sesión recordada con su vencimiento.

```

16 |     /** @return el usuario dueño de un token vigente, o null si no existe o vencio */
17 |     Usuario buscarPorToken(String token) throws DAOException;
18 |

```

Líneas 16 a 18: Buscar el dueño de un token vigente.

```

19 |     void eliminarToken(String token) throws DAOException;
20 | }

```

Líneas 19 a 20: Eliminar un token (al cerrar sesión).

logistica-dao / src / main / java / ar / edu / usal / logica / dao / ViajeDAO.java

Para qué existe: Contrato de acceso a datos de viajes.

Defensa: iniciar() y finalizar() reciben siempre el id del chofer: la base verifica que el viaje le pertenezca, así un chofer no puede tocar viajes ajenos aunque manipule el formulario.

```
1 | package ar.edu.usal.logistica.dao;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | import ar.edu.usal.logistica.excepcion.DAOException;
4 | import ar.edu.usal.logistica.excepcion.ValidacionException;
5 | import ar.edu.usal.logistica.modelo.Viaje;
6 |
7 | import java.util.List;
8 |
```

Líneas 3 a 8: Importaciones de las clases que usa este archivo: **DAOException**, **ValidacionException**, **Viaje**, **List**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```
9 | /** Contrato de acceso a datos de viajes. */
10 | public interface ViajeDAO {
11 |
```

Líneas 9 a 11: Interfaz ViajeDAO.

```
12 |     /** Guarda el viaje verificando en la misma transaccion que el camion siga libre. */
13 |     void insertar(Viaje viaje) throws DAOException, ValidacionException;
14 |
```

Líneas 12 a 14: Alta del viaje, verificando disponibilidad del camión en la misma transacción.

```
15 |     List<Viaje> listarTodos() throws DAOException;
16 |
17 |     List<Viaje> listarPorChofer(long choferId) throws DAOException;
18 |
```

Líneas 15 a 18: Listado para el administrador y listado de un chofer.

```
19 |     /** El chofer marca que inicia un viaje ASIGNADO. */
20 |     void iniciar(long viajeId, long choferId) throws DAOException, ValidacionException;
21 |
22 |     /** El chofer marca como realizado un viaje EN_CURSO. */
23 |     void finalizar(long viajeId, long choferId) throws DAOException, ValidacionException;
24 | }
```

Líneas 19 a 24: Iniciar y finalizar, acciones del perfil chofer.

logistica-dao / src / main / java / ar / edu / usal / logistica / dao / DistanciaDAO.java

Para qué existe: Contrato de consulta de la tabla de distancias.

Defensa: La tabla del enunciado está en la base, no fija en el código: si cambia una distancia se actualiza un dato, no el programa.

```
1 | package ar.edu.usal.logistica.dao;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | import ar.edu.usal.logistica.excepcion.DAOException;
4 | import ar.edu.usal.logistica.excepcion.ValidacionException;
5 | import ar.edu.usal.logistica.modelo.Destino;
6 |
7 | import java.util.Map;
8 |
```

Líneas 3 a 8: Importaciones de las clases que usa este archivo: **DAOException**, **ValidacionException**, **Destino**, **Map**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```
9 | /** Contrato de acceso a la tabla de distancias entre ciudades. */
10 | public interface DistanciaDAO {
11 |
```

Líneas 9 a 11: Interfaz DistanciaDAO.

```

12 | /** @return kilometros entre ambas ciudades */
13 | int obtenerKm(Destino origen, Destino destino) throws DAOException, ValidacionException;
14 |

```

Líneas 12 a 14: Km entre dos ciudades.

```

15 | /** Tabla completa origen -> (destino -> km), para mostrarla en pantalla. */
16 | Map<Destino, Map<Destino, Integer>> obtenerTabla() throws DAOException;
17 | }

```

Líneas 15 a 17: La tabla entera para mostrarla en pantalla.

logistica-dao / src / main / java / ar / edu / usal / logica / dao / DAOFactory.java

Para qué existe: Fábrica abstracta de DAOs.

Defensa: Patrón **Factory** (punto 01). La capa web pide «dame el ChoferDAO» sin saber que es JDBC; para usar otra tecnología se crea otra fábrica y no se toca ningún controlador. Es el mismo principio de programar contra interfaces.

```

1 | package ar.edu.usal.logistica.dao;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.jdbc.JdbcDAOFactory;
4 |

```

Líneas 3 a 4: Importaciones de las clases que usa este archivo: **JdbcDAOFactory**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

5 | /**
6 |  * PATRON FACTORY: la capa web le pide los DAO a la fabrica y solo conoce
7 |  * interfaces. Si mañana se cambia JDBC por otra tecnologia, solo se agrega
8 |  * otra fabrica y los controladores no se modifican.
9 |  */

```

Líneas 5 a 9: Documentación del patrón.

```

10 | public abstract class DAOFactory {
11 |
12 |     /** Fabrica concreta en uso (hoy, la de JDBC). */
13 |     public static DAOFactory getInstance() {
14 |         return JdbcDAOFactory.getInstance();
15 |     }
16 | }

```

Líneas 10 a 16: Clase abstracta. **getInstance()** devuelve la fábrica concreta en uso (JDBC).

```

17 |     public abstract ChoferDAO getChoferDAO();
18 |
19 |     public abstract CamionDAO getCamionDAO();
20 |
21 |     public abstract UsuarioDAO getUsuarioDAO();
22 |
23 |     public abstract ViajeDAO getViajeDAO();
24 |
25 |     public abstract DistanciaDAO getDistanciaDAO();
26 | }

```

Líneas 17 a 26: Un método abstracto por cada DAO; cada fábrica concreta decide qué implementación entrega.

logistica-dao / src / test / java / ar / edu / usal / logica / modelo / CalculadoraViajeTest.java

Para qué existe: Pruebas unitarias (JUnit 5) del cálculo y de las reglas del dominio.

Defensa: Probar la lógica de negocio sin base de datos ni servidor demuestra el beneficio de separar el dominio del resto. Cada test usa ejemplos reales del enunciado (Córdoba 646 km).

```
1 | package ar.edu.usal.logistica.modelo;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | import static org.junit.jupiter.api.Assertions.assertEquals;
4 | import static org.junit.jupiter.api.Assertions.assertThrows;
5 | import static org.junit.jupiter.api.Assertions.assertTrue;
6 |
7 | import ar.edu.usal.logistica.excepcion.ValidacionException;
8 | import org.junit.jupiter.api.Test;
9 |
10 | import java.time.LocalDate;
11 |
```

Líneas 3 a 11: Importaciones de las clases que usa este archivo: **assertEquals**, **assertThrows**, **assertTrue**, **ValidacionException**, **Test**, **LocalDate**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```
12 | class CalculadoraViajeTest {
13 |
14 |     // Camion de 30 t, tanque de 400 l, consume 0,4 l/km
15 |     private final Camion camion = new Camion(1L, "Scania", "R450", "ab123cd", 30, 400, 0.4);
16 |
```

Líneas 12 a 16: Camión de prueba: 30 t, tanque de 400 l, consume 0,4 l/km.

```
17 |     @Test
18 |     void diasSeRedondeanHaciaArribaA200KmPorDia() {
19 |         assertEquals(1, CalculadoraViaje.calcular(camion, 200).dias());
20 |         assertEquals(2, CalculadoraViaje.calcular(camion, 201).dias());
21 |         // CABA - Córdoba: 646 km -> 4 días
22 |         assertEquals(4, CalculadoraViaje.calcular(camion, 646).dias());
23 |     }
24 |
25 |     @Test
```

Líneas 17 a 25: Días: 200 km = 1 día; 201 km = 2 días (redondeo hacia arriba); CABA-Córdoba 646 km = 4 días.

```
26 |     void tanquesSeCalculanConLitrosTotalesYSeRedondeanHaciaArriba() {
27 |         // 1000 km * 0,4 = 400 litros = exactamente 1 tanque
28 |         assertEquals(1, CalculadoraViaje.calcular(camion, 1000).tanques());
29 |         // 1001 km = 400,4 litros = 2 tanques
30 |         assertEquals(2, CalculadoraViaje.calcular(camion, 1001).tanques());
31 |         // Mendoza - CABA: 985 km = 394 litros = 1 tanque
32 |         assertEquals(1, CalculadoraViaje.calcular(camion, 985).tanques());
33 |     }
34 |
35 |     @Test
```

Líneas 26 a 35: Tanques: 1000 km × 0,4 = 400 l = exactamente 1 tanque; 1001 km = 400,4 l = 2 tanques; CABA-Mendoza 985 km = 394 l = 1 tanque.

```
36 |     void distanciaNoPositivaEsInvalida() {
37 |         assertThrows(IllegalArgumentException.class, () -> CalculadoraViaje.calcular(camion, 0));
38 |     }
39 |
40 |     @Test
```

Líneas 36 a 40: Una distancia 0 es un error: se espera `IllegalArgumentException`.

```
41 |     void elDominioSeNormalizaEnMayusculas() {
42 |         assertEquals("AB123CD", camion.getDominio());
43 |     }
44 |
45 |     @Test
```

Líneas 41 a 45: El dominio «ab123cd» se guarda en mayúsculas.

```

46 | void choferSoloManejaCamionesAutorizadosYDentroDeSuCategoria() throws ValidacionException {
47 |     Chofer chofer = new Chofer(1L, "Juan", "Pérez", "30111222", LocalDate.of(1990, 5, 10), Categoria.B,
48 |         "1144556677");
49 |     assertEquals(false, chofer.puedeManejar(camion)); // no autorizado
50 |     chofer.autorizar(camion);
51 |     assertEquals(false, chofer.puedeManejar(camion)); // categoria B llega a 20 t y el camion es de 30 t
52 |     assertThrows(ValidacionException.class, chofer::validar);
53 |
54 |     Chofer categoriaC = new Chofer(2L, "Ana", "Gómez", "30111223", LocalDate.of(1988, 1, 1), Categoria.C,
55 |         "1144556677");
56 |     categoriaC.autorizar(camion);
57 |     assertTrue(categoriaC.puedeManejar(camion));
58 |     categoriaC.validar();
59 | }
    @Test

```

Líneas 46 a 59: Un chofer categoría B no puede manejar un camión de 30 t aunque esté autorizado (validar() lanza excepción); con categoría C sí.

```

60 | void elViajeNoAdmiteMismoOrigenYDestino() {
61 |     Chofer chofer = new Chofer(2L, "Ana", "Gómez", "30111223", LocalDate.of(1988, 1, 1), Categoria.C,
62 |         "1144556677");
63 |     chofer.autorizar(camion);
64 |     Viaje viaje = Viaje.nuevo(chofer, camion, Destino.CABA, Destino.CABA, 10);
65 |     assertThrows(ValidacionException.class, viaje::validar);
66 | }

```

Líneas 60 a 66: Un viaje con el mismo origen y destino no es válido.

Proyecto DAO - acceso a datos JDBC

logistica-dao / src / main / java / ar / edu / usal / logistica / dao / jdbc / ConexionDB.java

Para qué existe: Entrega conexiones a MySQL y guarda la configuración.

Defensa: Patrón **Singleton** (punto 01): existe una única instancia, que lee db.properties una sola vez. Uso el holder idiom (clase interna estática) porque es seguro entre hilos sin necesidad de synchronized y carga la instancia recién cuando se usa. Cada DAO pide una conexión nueva y la cierra con try-with-resources: en una aplicación web varios hilos atienden a la vez y una conexión compartida sería insegura. En producción se reemplazaría por un pool de conexiones (DataSource), sin cambiar a los DAO.

```
1 | package ar.edu.usal.logistica.dao.jdbc;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | import ar.edu.usal.logistica.excepcion.DAOException;
4 |
5 | import java.io.IOException;
6 | import java.io.InputStream;
7 | import java.sql.Connection;
8 | import java.sql.DriverManager;
9 | import java.sql.SQLException;
10 | import java.util.Properties;
11 |
```

Líneas 3 a 11: Importaciones de las clases que usa este archivo: **DAOException**, **IOException**, **InputStream**, **Connection**, **DriverManager**, **SQLException**, **Properties**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```
12 | /**
13 |  * PATRON SINGLETON: existe una unica instancia que lee la configuracion una
14 |  * sola vez y entrega conexiones nuevas a quien las pide. Cada DAO abre su
15 |  * conexion, la usa y la cierra (try-with-resources).
16 |  */
```

Líneas 12 a 16: Documentación del patrón Singleton.

```
17 | public final class ConexionDB {
18 |
19 |     // Inicializacion perezosa y segura entre hilos (holder idiom)
20 |     private static final class Holder {
21 |         private static final ConexionDB INSTANCIA = new ConexionDB();
22 |     }
23 |
24 |     private final String url;
25 |     private final String usuario;
26 |     private final String clave;
27 |
```

Líneas 17 a 27: Clase final. La clase interna **Holder** crea la única instancia de forma perezosa y thread-safe. Atributos: url, usuario y clave.

```
28 |     private ConexionDB() {
29 |         Properties props = new Properties();
30 |         try (InputStream in = ConexionDB.class.getClassLoader().getResourceAsStream("db.properties")) {
31 |             if (in == null) {
32 |                 throw new IllegalStateException("No se encontró db.properties en el classpath");
33 |             }
34 |             props.load(in);
35 |             Class.forName(props.getProperty("db.driver"));
36 |         } catch (IOException | ClassNotFoundException e) {
37 |             throw new IllegalStateException("No se pudo inicializar la conexión a la base de datos", e);
38 |         }
39 |     }
```

Líneas 28 a 38: Constructor **privado**: nadie más puede crear instancias. Carga db.properties desde el classpath; si falta, falla con un mensaje claro. Class.forName carga el driver.

```

39 | // Las variables de entorno tienen prioridad (util para produccion sin tocar el codigo)
40 | this.url = valor("DB_URL", props.getProperty("db.url"));
41 | this.usuario = valor("DB_USER", props.getProperty("db.user"));
42 | this.clave = valor("DB_PASSWORD", props.getProperty("db.password"));
43 | }
44 |

```

Líneas 39 a 44: Si existen las variables DB_URL, DB_USER o DB_PASSWORD, tienen prioridad sobre el archivo.

```

45 | public static ConexionDB getInstance() {
46 |     return Holder.INSTANCE;
47 | }
48 |

```

Líneas 45 a 48: Punto de acceso global a la única instancia.

```

49 | /** Abre una conexion nueva. Quien la pide es responsable de cerrarla. */
50 | public Connection getConexion() throws DAOException {
51 |     try {
52 |         return DriverManager.getConnection(url, usuario, clave);
53 |     } catch (SQLException e) {
54 |         throw new DAOException("No se pudo conectar con la base de datos.", e);
55 |     }
56 | }
57 |

```

Líneas 49 a 57: getConexion(): pide una conexión nueva a DriverManager; si falla, lanza DAOException (error técnico encapsulado). Quien la pide debe cerrarla.

```

58 | private static String valor(String variableEntorno, String porDefecto) {
59 |     String env = System.getenv(variableEntorno);
60 |     return env != null && !env.isEmpty() ? env : porDefecto;
61 | }
62 |

```

Líneas 58 a 62: Auxiliar: devuelve la variable de entorno si existe y no está vacía; si no, el valor por defecto.

logistica-dao / src / main / java / ar / edu / usal / logica / dao / jdbc / JdbcDAOFactory.java

Para qué existe: Fábrica concreta que entrega los DAO JDBC.

Defensa: Implementa el patrón Factory. Es **final** y singleton (la fábrica no tiene estado); los DAO son clases de paquete (no públicas): desde afuera solo se accede por las interfaces, lo que refuerza el encapsulamiento.

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.CamionDAO;
4 | import ar.edu.usal.logistica.dao.ChoferDAO;
5 | import ar.edu.usal.logistica.dao.DAOFactory;
6 | import ar.edu.usal.logistica.dao.DistanciaDAO;
7 | import ar.edu.usal.logistica.dao.UsuarioDAO;
8 | import ar.edu.usal.logistica.dao.ViajeDAO;
9 |

```

Líneas 3 a 9: Importaciones de las clases que usa este archivo: **CamionDAO**, **ChoferDAO**, **DAOFactory**, **DistanciaDAO**, **UsuarioDAO**, **ViajeDAO**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

10 | /** Fabrica concreta que entrega las implementaciones JDBC de cada DAO. */
11 | public final class JdbcDAOFactory extends DAOFactory {
12 |
13 |     private static final JdbcDAOFactory INSTANCIA = new JdbcDAOFactory();
14 |
15 |     private JdbcDAOFactory() {
16 |     }
17 |
18 |     public static JdbcDAOFactory getInstance() {
19 |         return INSTANCIA;
20 |     }
21 |

```

Líneas 10 a 21: Clase final con su única instancia y constructor privado.

```

22 |     @Override
23 |     public ChoferDAO getChoferDAO() {
24 |         return new ChoferDAOJdbc();
25 |     }
26 |
27 |     @Override
28 |     public CamionDAO getCamionDAO() {
29 |         return new CamionDAOJdbc();
30 |     }
31 |
32 |     @Override
33 |     public UsuarioDAO getUsuarioDAO() {
34 |         return new UsuarioDAOJdbc();
35 |     }
36 |
37 |     @Override
38 |     public ViajeDAO getViajeDAO() {
39 |         return new ViajeDAOJdbc();
40 |     }
41 |
42 |     @Override
43 |     public DistanciaDAO getDistanciaDAO() {
44 |         return new DistanciaDAOJdbc();
45 |     }
46 | }

```

Líneas 22 a 46: Cada método getXxxDAO() devuelve una implementación nueva. Los DAO no guardan estado (abren y cierran su conexión en cada operación), por eso crear uno nuevo es barato y seguro.

logistica-dao / src / main / java / ar / edu / usal / logica / dao / jdbc / JdbcUtil.java

Para qué existe: Utilidades compartidas por los DAO: códigos de error y cierre seguro de transacciones.

Defensa: Evita repetir código (DRY). Traducir los códigos de error de MySQL (1062 duplicado, 1451/1452 clave foránea) permite dar mensajes de negocio claros («ya existe un chofer con ese DNI») en lugar de errores técnicos. Dejar que la base detecte el duplicado es más seguro que preguntar antes (evita condiciones de carrera entre dos usuarios).

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import java.sql.Connection;
4 | import java.sql.SQLException;
5 |

```

Líneas 3 a 5: Importaciones de las clases que usa este archivo: **Connection**, **SQLException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

6 | /** Utilidades compartidas por los DAO JDBC (codigos de error y manejo de transacciones). */
7 | final class JdbcUtil {
8 |
9 |     /**Codigo de MySQL: valor duplicado en una clave unica. */
10 |    static final int ERR_DUPLICADO = 1062;
11 |    /** Codigos de MySQL: violacion de clave foranea (hijo y padre). */
12 |    static final int ERR_FK_PADRE = 1451;
13 |    static final int ERR_FK_HIJO = 1452;
14 |
15 |    private JdbcUtil() {
16 |    }
17 |

```

Líneas 6 a 17: Clase de utilidades con los códigos de error de MySQL que nos interesan y constructor privado.

```

18 |    static boolean esDuplicado(SQLException e) {
19 |        return e.getErrorCode() == ERR_DUPLICADO;
20 |    }
21 |
22 |    static boolean esClaveForanea(SQLException e) {
23 |        return e.getErrorCode() == ERR_FK_PADRE || e.getErrorCode() == ERR_FK_HIJO;
24 |    }
25 |

```

Líneas 18 a 25: Predicados que reconocen un valor duplicado y una violación de clave foránea.

```

26 |    /** Deshace la transaccion sin ocultar la excepcion original si el rollback tambien falla. */
27 |    static void rollbackSilencioso(Connection con, Throwable causa) {
28 |        if (con == null) {
29 |            return;
30 |        }
31 |        try {
32 |            con.rollback();
33 |        } catch (SQLException ex) {
34 |            causa.addSuppressed(ex);
35 |        }
36 |    }
37 |

```

Líneas 26 a 37: rollbackSilencioso: intenta deshacer la transacción; si el rollback también falla, el error se agrega como suppressed de la causa original, sin ocultarla.

```

38 |    /** Devuelve la conexcion al modo normal (autocommit) y la cierra. */
39 |    static void cerrar(Connection con) {
40 |        if (con == null) {
41 |            return;
42 |        }
43 |        try {
44 |            con.setAutoCommit(true);
45 |            con.close();
46 |        } catch (SQLException ignorada) {
47 |            // Al cerrar no hay nada mas que hacer; el error original (si hubo) ya se informo
48 |        }
49 |    }
50 | }

```

Líneas 38 a 50: cerrar: devuelve la conexión a autocommit y la cierra. Se usa en **finally** para garantizar que nunca quede una conexión abierta.

logistica-dao / src / main / java / ar / edu / usal / logistica / dao / jdbc / Mapeadores.java

Para qué existe: Convierte filas del ResultSet en objetos Camion y Chofer.

Defensa: Un solo lugar para el mapeo objeto-relacional manual. El parámetro prefijo permite leer columnas con alias cuando una consulta une tablas (ch_nombre, c_marca).

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.modelo.Camion;
4 | import ar.edu.usal.logistica.modelo.Categoria;
5 | import ar.edu.usal.logistica.modelo.Chofer;
6 |
7 | import java.sql.ResultSet;
8 | import java.sql.SQLException;
9 |

```

Líneas 3 a 9: Importaciones de las clases que usa este archivo: **Camion, Categoria, Chofer, ResultSet, SQLException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

10 | /**
11 |  * Convierte filas de un ResultSet en objetos del dominio. Se centraliza aca
12 |  * para no repetir el mapeo en cada DAO. El prefijo permite leer columnas con
13 |  * alias cuando una consulta une varias tablas (ej: "ch_nombre", "c_marca").
14 |  */
15 | final class Mapeadores {
16 |
17 |     private Mapeadores() {
18 |     }
19 |

```

Líneas 10 a 19: Documentación.

```

20 |     static Camion camion(ResultSet rs, String prefijo) throws SQLException {
21 |         return new Camion(
22 |             rs.getLong(prefijo + "id"),
23 |             rs.getString(prefijo + "marca"),
24 |             rs.getString(prefijo + "modelo"),
25 |             rs.getString(prefijo + "dominio"),
26 |             rs.getDouble(prefijo + "toneladas_max"),
27 |             rs.getDouble(prefijo + "capacidad_tanque_litros"),
28 |             rs.getDouble(prefijo + "consumo_litros_km"));
29 |     }
30 |

```

Líneas 20 a 30: Crea un Camion leyendo las siete columnas (con el prefijo indicado).

```

31 |     static Chofer chofer(ResultSet rs, String prefijo) throws SQLException {
32 |         return new Chofer(
33 |             rs.getLong(prefijo + "id"),
34 |             rs.getString(prefijo + "nombre"),
35 |             rs.getString(prefijo + "apellido"),
36 |             rs.getString(prefijo + "dni"),
37 |             rs.getDate(prefijo + "fecha_nacimiento").toLocalDate(),
38 |             Categoria.valueOf(rs.getString(prefijo + "categoria")),
39 |             rs.getString(prefijo + "telefono"));
40 |     }
41 | }

```

Líneas 31 a 41: Crea un Chofer: la fecha SQL se convierte a LocalDate y el texto de la categoría al enum con Categoria.valueOf.

logistica-dao / src / main / java / ar / edu / usal / logistica / dao / jdbc / CamionDAOJdbc.java

Para qué existe: Implementación JDBC del DAO de camiones.

Defensa: Usa los **tres** mecanismos JDBC pedidos (punto 06): **Statement** en listarTodos (consulta fija, sin parámetros), **PreparedStatement** en alta, baja, modificación y búsqueda (parámetros con ? que evitan la inyección SQL y se compilan una vez) y **CallableStatement** en listarDisponiblesParaChofer (llama al stored procedure). Todos los recursos se cierran solos con try-with-resources (punto 02/05).

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.CamionDAO;
4 | import ar.edu.usal.logistica.excepcion.DAOException;
5 | import ar.edu.usal.logistica.excepcion.ValidacionException;
6 | import ar.edu.usal.logistica.modelo.Camion;
7 |
8 | import java.sql.CallableStatement;
9 | import java.sql.Connection;
10 | import java.sql.PreparedStatement;
11 | import java.sql.ResultSet;
12 | import java.sql.SQLException;
13 | import java.sql.Statement;
14 | import java.util.ArrayList;
15 | import java.util.List;
16 |

```

Líneas 3 a 16: Importaciones de las clases que usa este archivo: **CamionDAO**, **DAOException**, **ValidacionException**, **Camion**, **CallableStatement**, **Connection**, **PreparedStatement**, **ResultSet**, **SQLException**, **Statement**, **ArrayList**, **List**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

17 | /** Implementacion JDBC de CamionDAO. */
18 | class CamionDAOJdbc implements CamionDAO {
19 |
20 |     private static final String INSERT =
21 |         "INSERT INTO camion (marca, modelo, dominio, toneladas_max, capacidad_tanque_litros, consumo_litros_km) "
22 |         + "VALUES (?, ?, ?, ?, ?, ?)";
23 |     private static final String UPDATE =
24 |         "UPDATE camion SET marca = ?, modelo = ?, dominio = ?, toneladas_max = ?, "
25 |         + "capacidad_tanque_litros = ?, consumo_litros_km = ? WHERE id = ?";
26 |     private static final String DELETE = "DELETE FROM camion WHERE id = ?";
27 |     private static final String SELECT_POR_ID =
28 |         "SELECT id, marca, modelo, dominio, toneladas_max, capacidad_tanque_litros, consumo_litros_km "
29 |         + "FROM camion WHERE id = ?";
30 |     // Consulta fija, sin parametros: es el caso de uso de Statement
31 |     private static final String SELECT_TODOS =
32 |         "SELECT id, marca, modelo, dominio, toneladas_max, capacidad_tanque_litros, consumo_litros_km "
33 |         + "FROM camion ORDER BY marca, modelo, dominio";
34 |
35 |     @Override

```

Líneas 17 a 35: Clase de paquete (no pública) y las sentencias SQL como constantes: INSERT, UPDATE, DELETE y SELECT por id con parámetros ? y un SELECT fijo para el listado.

```

36 |     public void insertar(Camion camion) throws DAOException, ValidacionException {
37 |         camion.validar();
38 |         try (Connection con = ConexionDB.getInstance().getConexion();
39 |             PreparedStatement ps = con.prepareStatement(INSERT, Statement.RETURN_GENERATED_KEYS)) {
40 |             ps.setString(1, camion.getMarca());
41 |             ps.setString(2, camion.getModelo());
42 |             ps.setString(3, camion.getDominio());
43 |             ps.setDouble(4, camion.getToneladasMaximas());
44 |             ps.setDouble(5, camion.getCapacidadTanqueLitros());
45 |             ps.setDouble(6, camion.getConsumoLitrosPorKm());
46 |             ps.executeUpdate();
47 |             try (ResultSet claves = ps.getGeneratedKeys()) {
48 |                 if (claves.next()) {
49 |                     camion.setId(claves.getLong(1));
50 |                 }
51 |             }
52 |         } catch (SQLException e) {

```

Líneas 36 a 52: **PreparedStatement** para el alta. Primero valida el camión. RETURN_GENERATED_KEYS pide el id autoincremental que generó MySQL y se lo asigna al objeto.

```

53 |             throw traducir(e, camion, "No se pudo guardar el camión.");
54 |         }
55 |     }
56 |
57 |     @Override

```

Líneas 53 a 57: Si la base rechaza la operación, traducir() decide: dominio duplicado → mensaje de negocio.

```

58 |     public void actualizar(Camion camion) throws DAOException, ValidacionException {
59 |         camion.validar();
60 |         try (Connection con = ConexionDB.getInstance().getConexion();
61 |             PreparedStatement ps = con.prepareStatement(UPDATE)) {
62 |             ps.setString(1, camion.getMarca());
63 |             ps.setString(2, camion.getModelo());
64 |             ps.setString(3, camion.getDominio());
65 |             ps.setDouble(4, camion.getToneladasMaximas());
66 |             ps.setDouble(5, camion.getCapacidadTanqueLitros());
67 |             ps.setDouble(6, camion.getConsumoLitrosPorKm());
68 |             ps.setLong(7, camion.getId());
69 |             if (ps.executeUpdate() == 0) {
70 |                 throw new ValidacionException("El camión que intenta modificar no existe.");
71 |             }
72 |         } catch (SQLException e) {
73 |             throw traducir(e, camion, "No se pudo modificar el camión.");
74 |         }
75 |     }
76 |
77 |     @Override

```

Líneas 58 a 77: UPDATE por id. Si no modificó ninguna fila, el camión no existe y se informa.

```

78 |     public void eliminar(long id) throws DAOException, ValidacionException {
79 |         try (Connection con = ConexionDB.getInstance().getConexion();
80 |             PreparedStatement ps = con.prepareStatement(DELETE)) {
81 |             ps.setLong(1, id);
82 |             if (ps.executeUpdate() == 0) {
83 |                 throw new ValidacionException("El camión que intenta eliminar no existe.");
84 |             }
85 |         } catch (SQLException e) {
86 |             if (JdbcUtil.esClaveForanea(e)) {
87 |                 throw new ValidacionException("No se puede eliminar el camión porque tiene viajes registrados.");
88 |             }
89 |             throw new DAOException("No se pudo eliminar el camión.", e);
90 |         }
91 |     }
92 |
93 |     @Override

```

Líneas 78 a 93: DELETE por id. Si el camión tiene viajes, la clave foránea de la base lo impide y se traduce a «tiene viajes registrados».

```

94 |     public Camion buscarPorId(long id) throws DAOException {
95 |         try (Connection con = ConexionDB.getInstance().getConexion();
96 |             PreparedStatement ps = con.prepareStatement(SELECT_POR_ID)) {
97 |             ps.setLong(1, id);
98 |             try (ResultSet rs = ps.executeQuery()) {
99 |                 return rs.next() ? Mapeadores.camion(rs, "") : null;
100 |             }
101 |         } catch (SQLException e) {
102 |             throw new DAOException("No se pudo consultar el camión.", e);
103 |         }
104 |     }
105 |
106 |     @Override

```

Líneas 94 a 106: SELECT por id: devuelve el camión o null.

```

107 |     public List<Camion> listarTodos() throws DAOException {
108 |         List<Camion> camiones = new ArrayList<>();
109 |         try (Connection con = ConexionDB.getInstance().getConexion();
110 |             Statement st = con.createStatement();
111 |             ResultSet rs = st.executeQuery(SELECT_TODOS)) {
112 |             while (rs.next()) {
113 |                 camiones.add(Mapeadores.camion(rs, ""));
114 |             }
115 |         } catch (SQLException e) {
116 |             throw new DAOException("No se pudo listar los camiones.", e);
117 |         }
118 |         return camiones;
119 |     }
120 |
121 |     @Override

```

Líneas 107 a 121: Acá se usa **Statement**: la consulta es fija y no recibe datos del usuario, por eso no hay riesgo de inyección.

```

122 |     public List<Camion> listarDisponiblesParaChofer(long choferId) throws DAOException {
123 |         List<Camion> camiones = new ArrayList<>();
124 |         // La regla de disponibilidad vive en el stored procedure sp_camiones_disponibles
125 |         try (Connection con = ConexionDB.getInstance().getConexion();
126 |             CallableStatement cs = con.prepareCall("{call sp_camiones_disponibles(?)}")) {
127 |             cs.setLong(1, choferId);
128 |             try (ResultSet rs = cs.executeQuery()) {
129 |                 while (rs.next()) {
130 |                     camiones.add(Mapeadores.camion(rs, ""));
131 |                 }
132 |             }
133 |         } catch (SQLException e) {
134 |             throw new DAOException("No se pudieron consultar los camiones disponibles.", e);
135 |         }
136 |         return camiones;
137 |     }
138 | }

```

Líneas 122 a 138: Usa **CallableStatement** con la sintaxis {call sp_camiones_disponibles(?)}. La regla de disponibilidad queda en la base, en un solo lugar.

```

139 |     private DAOException traducir(SQLException e, Camion camion, String mensajeGenerico) throws ValidacionException {
140 |         if (JdbcUtil.esDuplicado(e)) {
141 |             throw new ValidacionException("Ya existe un camión con el dominio " + camion.getDominio() + ".");
142 |         }
143 |         return new DAOException(mensajeGenerico, e);
144 |     }
145 | }

```

Líneas 139 a 145: Traduce el error de duplicado en ValidacionException; cualquier otro error pasa a DAOException.

logistica-dao / src / main / java / ar / edu / usal / logica / dao / jdbc / UsuarioDAOJdbc.java

Para qué existe: Implementación JDBC de usuarios y tokens de sesión recordada.

Defensa: guardarToken usa una transacción porque son dos sentencias que deben ir juntas (limpiar tokens vencidos + insertar el nuevo). Las fechas se comparan con la hora de Java para no depender de la hora del servidor de base.

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.UsuarioDAO;
4 | import ar.edu.usal.logistica.excepcion.DAOException;
5 | import ar.edu.usal.logistica.modelo.Rol;
6 | import ar.edu.usal.logistica.modelo.Usuario;
7 |
8 | import java.sql.Connection;
9 | import java.sql.PreparedStatement;
10 | import java.sql.ResultSet;
11 | import java.sql.SQLException;
12 | import java.sql.Timestamp;
13 | import java.time.LocalDateTime;
14 |

```

Líneas 3 a 14: Importaciones de las clases que usa este archivo: **UsuarioDAO**, **DAOException**, **Rol**, **Usuario**, **Connection**, **PreparedStatement**, **ResultSet**, **SQLException**, **Timestamp**, **LocalDateTime**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

15 | /** Implementacion JDBC de UsuarioDAO. */
16 | class UsuarioDAOJdbc implements UsuarioDAO {
17 |
18 |     private static final String SELECT_POR_USERNAME =
19 |         "SELECT id, username, password_hash, rol, chofer_id FROM usuario WHERE username = ?";
20 |     private static final String SELECT_POR_TOKEN =
21 |         "SELECT u.id, u.username, u.password_hash, u.rol, u.chofer_id FROM usuario u "
22 |         + "JOIN sesion_recordada s ON s.usuario_id = u.id WHERE s.token = ? AND s.expira > ?";
23 |
24 |     @Override

```

Líneas 15 a 24: Consultas: usuario por nombre y usuario por token vigente (JOIN con sesion_recordada y expira > ahora).

```

25 | public Usuario buscarPorUsername(String username) throws DAOException {
26 |     try (Connection con = ConexionDB.getInstance().getConexion()) {
27 |         PreparedStatement ps = con.prepareStatement(SELECT_POR_USERNAME) {
28 |             ps.setString(1, username);
29 |             try (ResultSet rs = ps.executeQuery()) {
30 |                 return rs.next() ? mapear(rs) : null;
31 |             }
32 |         } catch (SQLException e) {
33 |             throw new DAOException("No se pudo consultar el usuario.", e);
34 |         }
35 |     }
36 |
37 |     @Override

```

Líneas 25 a 37: PreparedStatement: busca por username; devuelve null si no existe.

```

38 | public void guardarToken(String token, long usuarioId, LocalDateTime expira) throws DAOException {
39 |     try (Connection con = ConexionDB.getInstance().getConexion()) {
40 |         // Dos sentencias que deben ir juntas: limpieza de tokens vencidos + alta del nuevo
41 |         con.setAutoCommit(false);
42 |         try (PreparedStatement limpiar = con.prepareStatement("DELETE FROM sesion_recordada WHERE expira < ?");
43 |             PreparedStatement alta = con.prepareStatement(
44 |                 "INSERT INTO sesion_recordada (token, usuario_id, expira) VALUES (?, ?, ?)")) {
45 |             limpiar.setTimestamp(1, Timestamp.valueOf(LocalDateTime.now()));
46 |             limpiar.executeUpdate();
47 |             alta.setString(1, token);
48 |             alta.setLong(2, usuarioId);
49 |             alta.setTimestamp(3, Timestamp.valueOf(expira));
50 |             alta.executeUpdate();
51 |             con.commit();
52 |         } catch (SQLException e) {
53 |             JdbcUtil.rollbackSilencioso(con, e);
54 |             throw e;
55 |         }
56 |     } catch (SQLException e) {
57 |         throw new DAOException("No se pudo recordar la sesión.", e);
58 |     }
59 | }
60 |
61 | @Override

```

Líneas 38 a 61: Transacción: desactiva autocommit, borra los tokens vencidos, inserta el nuevo y confirma con commit; ante error hace rollback.

```

62 | public Usuario buscarPorToken(String token) throws DAOException {
63 |     try (Connection con = ConexionDB.getInstance().getConexion()) {
64 |         PreparedStatement ps = con.prepareStatement(SELECT_POR_TOKEN) {
65 |             ps.setString(1, token);
66 |             ps.setTimestamp(2, Timestamp.valueOf(LocalDateTime.now()));
67 |             try (ResultSet rs = ps.executeQuery()) {
68 |                 return rs.next() ? mapear(rs) : null;
69 |             }
70 |         } catch (SQLException e) {
71 |             throw new DAOException("No se pudo validar la sesión recordada.", e);
72 |         }
73 |     }
74 |
75 |     @Override

```

Líneas 62 a 75: Devuelve el usuario dueño de un token que todavía no venció.

```

76 | public void eliminarToken(String token) throws DAOException {
77 |     try (Connection con = ConexionDB.getInstance().getConexion()) {
78 |         PreparedStatement ps = con.prepareStatement("DELETE FROM sesion_recordada WHERE token = ?") {
79 |             ps.setString(1, token);
80 |             ps.executeUpdate();
81 |         } catch (SQLException e) {
82 |             throw new DAOException("No se pudo cerrar la sesión recordada.", e);
83 |         }
84 |     }
85 |

```

Líneas 76 a 85: Borra el token (logout): ya no se puede reutilizar aunque alguien lo copie.

```

86 |     private Usuario mapear(ResultSet rs) throws SQLException {
87 |         long choferId = rs.getLong("chofer_id");
88 |         Long choferIdONulo = rs.isNull() ? null : choferId;
89 |         return new Usuario(rs.getLong("id"), rs.getString("username"), rs.getString("password_hash"),
90 |             Rol.valueOf(rs.getString("rol")), choferIdONulo);
91 |     }
92 | }

```

Líneas 86 a 92: Convierte la fila en Usuario. chofer_id puede ser NULL (administrador): se usa wasNull() para distinguir null de 0.

logistica-dao / src / main / java / ar / edu / usal / logica / dao / jdbc / DistanciaDAOJdbc.java

Para qué existe: Consulta de la tabla de distancias.

Defensa: Si no hay distancia (o origen = destino) se informa con una ValidacionException clara; nunca se asume un valor por defecto que generaría cálculos erróneos.

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.DistanciaDAO;
4 | import ar.edu.usal.logistica.excepcion.DAOException;
5 | import ar.edu.usal.logistica.excepcion.ValidacionException;
6 | import ar.edu.usal.logistica.modelo.Destino;
7 |
8 | import java.sql.Connection;
9 | import java.sql.PreparedStatement;
10 | import java.sql.ResultSet;
11 | import java.sql.SQLException;
12 | import java.sql.Statement;
13 | import java.util.EnumMap;
14 | import java.util.Map;
15 |

```

Líneas 3 a 15: Importaciones de las clases que usa este archivo: **DistanciaDAO**, **DAOException**, **ValidacionException**, **Destino**, **Connection**, **PreparedStatement**, **ResultSet**, **SQLException**, **Statement**, **EnumMap**, **Map**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

16 | /** Implementacion JDBC de DistanciaDAO. */
17 | class DistanciaDAOJdbc implements DistanciaDAO {
18 |
19 |     @Override

```

Líneas 16 a 19: Clase de paquete.

```

20 |     public int obtenerKm(Destino origen, Destino destino) throws DAOException, ValidacionException {
21 |         if (origen == null || destino == null || origen == destino) {
22 |             throw new ValidacionException("Debe elegir un origen y un destino distintos.");
23 |         }
24 |         try (Connection con = ConexionDB.getInstance().getConexion();
25 |             PreparedStatement ps = con.prepareStatement("SELECT km FROM distancia WHERE origen = ? AND destino =
26 |                 ?")) {
27 |             ps.setString(1, origen.name());
28 |             ps.setString(2, destino.name());
29 |             try (ResultSet rs = ps.executeQuery()) {
30 |                 if (rs.next()) {
31 |                     return rs.getInt("km");
32 |                 }
33 |             } catch (SQLException e) {
34 |                 throw new DAOException("No se pudo consultar la distancia.", e);
35 |             }

```

Líneas 20 a 35: Valida que haya origen y destino distintos. PreparedStatement con dos parámetros; si hay fila devuelve los km.

```

36 |         throw new ValidacionException("No hay distancia cargada entre " + origen.getNombre()
37 |             + " y " + destino.getNombre() + ".");
38 |     }
39 |
40 |     @Override

```

Líneas 36 a 40: Si la consulta no devolvió fila, avisa que no hay distancia cargada.

```

41 |     public Map<Destino, Map<Destino, Integer>> obtenerTabla() throws DAOException {
42 |         Map<Destino, Map<Destino, Integer>> tabla = new EnumMap<>(Destino.class);
43 |         for (Destino d : Destino.values()) {
44 |             tabla.put(d, new EnumMap<>(Destino.class));
45 |         }
46 |         try (Connection con = ConexionDB.getInstance().getConexion();
47 |             Statement st = con.createStatement();
48 |             ResultSet rs = st.executeQuery("SELECT origen, destino, km FROM distancia")) {
49 |             while (rs.next()) {
50 |                 tabla.get(Destino.valueOf(rs.getString("origen")))
51 |                     .put(Destino.valueOf(rs.getString("destino")), rs.getInt("km"));
52 |             }
53 |         } catch (SQLException e) {
54 |             throw new DAOException("No se pudo leer la tabla de distancias.", e);
55 |         }
56 |         return tabla;
57 |     }
58 | }

```

Líneas 41 a 58: Devuelve la tabla completa como Map de Map con EnumMap (eficiente para enums). Usa **Statement** porque la consulta es fija.

logistica-dao / src / main / java / ar / edu / usal / logistica / dao / jdbc / ChoferDAOJdbc.java

Para qué existe: Implementación JDBC del DAO de choferes, con transacciones.

Defensa: Punto 05 (transacciones JDBC): dar de alta un chofer toca tres tablas (chofer, chofer_camion y usuario). Con **setAutoCommit(false)** + commit/rollback, o se guarda todo o no se guarda nada: nunca queda un chofer sin usuario ni autorizaciones a medias. La consulta de lectura usa LEFT JOIN y agrupa las filas en Java: una sola consulta en vez de una por chofer (evita el problema N+1).

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.ChoferDAO;
4 | import ar.edu.usal.logistica.excepcion.ChoferNoEncontradoException;
5 | import ar.edu.usal.logistica.excepcion.DAOException;
6 | import ar.edu.usal.logistica.excepcion.ValidacionException;
7 | import ar.edu.usal.logistica.modelo.Camion;
8 | import ar.edu.usal.logistica.modelo.Chofer;
9 | import ar.edu.usal.logistica.util.PasswordUtil;
10 |
11 | import java.sql.Connection;
12 | import java.sql.Date;
13 | import java.sql.PreparedStatement;
14 | import java.sql.ResultSet;
15 | import java.sql.SQLException;
16 | import java.sql.Statement;
17 | import java.util.ArrayList;
18 | import java.util.LinkedHashMap;
19 | import java.util.List;
20 | import java.util.Map;
21 |

```

Líneas 3 a 21: Importaciones de las clases que usa este archivo: **ChoferDAO**, **ChoferNoEncontradoException**, **DAOException**, **ValidacionException**, **Camion**, **Chofer**, **PasswordUtil**, **Connection**, **Date**, **PreparedStatement**, **ResultSet**, **SQLException**, **Statement**, **ArrayList**, **LinkedHashMap**, **List**, **Map**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

22 | /** Implementacion JDBC de ChoferDAO. Las altas y modificaciones tocan varias tablas: usan transaccion. */
23 | class ChoferDAOJdbc implements ChoferDAO {
24 |
25 |     // Un chofer con todos sus camiones autorizados en una sola consulta (LEFT JOIN)
26 |     private static final String SELECT_BASE =
27 |         "SELECT ch.id AS ch_id, ch.nombre AS ch_nombre, ch.apellido AS ch_apellido, ch.dni AS ch_dni, "
28 |         + "ch.fecha_nacimiento AS ch_fecha_nacimiento, ch.categoria AS ch_categoria, "
29 |         + "ch.telefono AS ch_telefono, "
30 |         + "c.id AS c_id, c.marca AS c_marca, c.modelo AS c_modelo, c.dominio AS c_dominio, "
31 |         + "c.toneladas_max AS c_toneladas_max, c.capacidad_tanque_litros AS c_capacidad_tanque_litros, "
32 |         + "c.consumo_litros_km AS c_consumo_litros_km "
33 |         + "FROM chofer ch "
34 |         + "LEFT JOIN chofer_camion cc ON cc.chofer_id = ch.id "
35 |         + "LEFT JOIN camion c ON c.id = cc.camion_id ";
36 |     private static final String ORDEN = " ORDER BY ch.apellido, ch.nombre, c.marca, c.modelo";
37 |
38 |     private static final String INSERT_CHOFER =
39 |         "INSERT INTO chofer (nombre, apellido, dni, fecha_nacimiento, categoria, telefono) VALUES (?, ?, ?, ?, ?, "
40 |         + "?)";
41 |     private static final String UPDATE_CHOFER =
42 |         "UPDATE chofer SET nombre = ?, apellido = ?, dni = ?, fecha_nacimiento = ?, categoria = ?, telefono = ? "
43 |         + "WHERE id = ?";
44 |     private static final String INSERT_AUTORIZACION = "INSERT INTO chofer_camion (chofer_id, camion_id) VALUES (?, "
45 |         + "?)";
46 |     private static final String INSERT_USUARIO =
47 |         "INSERT INTO usuario (username, password_hash, rol, chofer_id) VALUES (?, ?, 'CHOFER', ?)";
48 |
49 |     @Override

```

Líneas 22 a 47: Consulta base con LEFT JOIN: chofer + camiones autorizados (alias ch_ y c_). LEFT JOIN para que también aparezcan los choferes sin camiones. Siguen los INSERT/UPDATE de cada tabla.

```

48 |     public void insertar(Chofer chofer, String passwordInicial) throws DAOException, ValidacionException {
49 |         chofer.validar();
50 |         if (passwordInicial == null || passwordInicial.length() < 6) {
51 |             throw new ValidacionException("La contraseña inicial debe tener al menos 6 caracteres.");
52 |         }
53 |         Connection con = null;
54 |         try {
55 |             con = ConexionDB.getInstance().getConexion();
56 |             con.setAutoCommit(false); // Inicio de la transaccion: o se guarda todo o no se guarda nada
57 |

```

Líneas 48 a 57: Valida el chofer y que la contraseña inicial tenga al menos 6 caracteres. Abre la conexión y empieza la transacción.

```

58 |             try (PreparedStatement ps = con.prepareStatement(INSERT_CHOFER, Statement.RETURN_GENERATED_KEYS)) {
59 |                 cargarDatos(ps, chofer);
60 |                 ps.executeUpdate();
61 |                 try (ResultSet claves = ps.getGeneratedKeys()) {
62 |                     claves.next();
63 |                     chofer.setId(claves.getLong(1));
64 |                 }
65 |             }

```

Líneas 58 a 65: Paso 1: inserta el chofer y recupera su id generado.

```

66 |             guardarAutorizaciones(con, chofer);

```

Línea 66: Paso 2: inserta las autorizaciones.

```

67 |             // El usuario del chofer se llama igual que su DNI
68 |             try (PreparedStatement ps = con.prepareStatement(INSERT_USUARIO)) {
69 |                 ps.setString(1, chofer.getDni());
70 |                 ps.setString(2, PasswordUtil.hashear(passwordInicial));
71 |                 ps.setLong(3, chofer.getId());
72 |                 ps.executeUpdate();
73 |             }

```

Líneas 67 a 73: Paso 3: crea el usuario con username = DNI y la contraseña **hasheada**.

```

74 |             con.commit();
75 |         } catch (SQLException e) {
76 |             JdbcUtil.rollbackSilencioso(con, e);

```

Líneas 74 a 76: Si los tres pasos salieron bien, **confirma** (commit). Las líneas del catch que siguen hacen rollback si algo falló.

```

77 |         chofer.setId(null);
78 |         throw traducir(e, chofer, "No se pudo guardar el chofer.");
79 |     } finally {
80 |         JdbcUtil.cerrar(con);
81 |     }
82 | }
83 |
84 | @Override

```

Líneas 77 a 84: Si algo falló, **deshace todo** (rollback), limpia el id y traduce el error. El finally siempre cierra la conexión.

```

85 |     public void actualizar(Chofer chofer) throws DAOException, ValidacionException {
86 |         chofer.validar();
87 |         Connection con = null;
88 |         try {
89 |             con = ConexionDB.getInstance().getConexion();
90 |             con.setAutoCommit(false);
91 |
92 |             try (PreparedStatement ps = con.prepareStatement(UPDATE_CHOFER)) {
93 |                 cargarDatos(ps, chofer);
94 |                 ps.setLong(7, chofer.getId());
95 |                 if (ps.executeUpdate() == 0) {
96 |                     throw new ValidacionException("El chofer que intenta modificar no existe.");
97 |                 }
98 |             }
99 |             // Se reemplazan las autorizaciones: se borran las viejas y se cargan las nuevas
100 |            try (PreparedStatement ps = con.prepareStatement("DELETE FROM chofer_camion WHERE chofer_id = ?")) {
101 |                ps.setLong(1, chofer.getId());
102 |                ps.executeUpdate();
103 |            }
104 |            guardarAutorizaciones(con, chofer);
105 |            // El usuario sigue el DNI del chofer si este cambio
106 |            try (PreparedStatement ps = con.prepareStatement("UPDATE usuario SET username = ? WHERE chofer_id = ?")) {
107 |                ps.setString(1, chofer.getDni());
108 |                ps.setLong(2, chofer.getId());
109 |                ps.executeUpdate();
110 |            }
111 |            con.commit();
112 |        } catch (SQLException e) {
113 |            JdbcUtil.rollbackSilencioso(con, e);
114 |            throw traducir(e, chofer, "No se pudo modificar el chofer.");
115 |        } catch (ValidacionException e) {
116 |            JdbcUtil.rollbackSilencioso(con, e);
117 |            throw e;
118 |        } finally {
119 |            JdbcUtil.cerrar(con);
120 |        }
121 |    }
122 |
123 | @Override

```

Líneas 85 a 123: También transaccional: actualiza el chofer, borra sus autorizaciones anteriores, inserta las nuevas y sincroniza el username con el DNI. Si no existe, lanza ValidacionException y hace rollback.

```

124 |     public void eliminar(long id) throws DAOException, ValidacionException {
125 |         // Las FK con ON DELETE CASCADE borran usuario, autorizaciones y tokens; si hay viajes la BD lo impide
126 |         try (Connection con = ConexionDB.getInstance().getConexion();
127 |             PreparedStatement ps = con.prepareStatement("DELETE FROM chofer WHERE id = ?")) {
128 |             ps.setLong(1, id);
129 |             if (ps.executeUpdate() == 0) {
130 |                 throw new ValidacionException("El chofer que intenta eliminar no existe.");
131 |             }
132 |         } catch (SQLException e) {
133 |             if (JdbcUtil.esClaveForanea(e)) {
134 |                 throw new ValidacionException("No se puede eliminar el chofer porque tiene viajes registrados.");
135 |             }
136 |             throw new DAOException("No se pudo eliminar el chofer.", e);
137 |         }
138 |     }
139 |
140 | @Override

```

Líneas 124 a 140: Un solo DELETE: las claves foráneas con ON DELETE CASCADE eliminan usuario y autorizaciones. Si hay viajes, la base lo impide y se informa con un mensaje claro.

```

141 |     public Chofer buscarPorId(long id) throws DAOException {
142 |         try (Connection con = ConexionDB.getInstance().getConexion();
143 |             PreparedStatement ps = con.prepareStatement(SELECT_BASE + "WHERE ch.id = ?" + ORDEN)) {
144 |             ps.setLong(1, id);
145 |             try (ResultSet rs = ps.executeQuery()) {
146 |                 List<Chofer> choferes = leer(rs);
147 |                 return choferes.isEmpty() ? null : choferes.get(0);
148 |             }
149 |         } catch (SQLException e) {
150 |             throw new DAOException("No se pudo consultar el chofer.", e);
151 |         }
152 |     }
153 |
154 |     @Override

```

Líneas 141 a 154: Búsqueda por id usando la consulta base.

```

155 |     public Chofer buscarPorDni(String dni) throws DAOException, ChoferNoEncontradoException {
156 |         try (Connection con = ConexionDB.getInstance().getConexion();
157 |             PreparedStatement ps = con.prepareStatement(SELECT_BASE + "WHERE ch.dni = ?" + ORDEN)) {
158 |             ps.setString(1, dni == null ? "" : dni.trim());
159 |             try (ResultSet rs = ps.executeQuery()) {
160 |                 List<Chofer> choferes = leer(rs);
161 |                 if (choferes.isEmpty()) {
162 |                     throw new ChoferNoEncontradoException(dni);
163 |                 }
164 |                 return choferes.get(0);
165 |             }
166 |         } catch (SQLException e) {
167 |             throw new DAOException("No se pudo consultar el chofer.", e);
168 |         }
169 |     }
170 |
171 |     @Override

```

Líneas 155 a 171: Búsqueda por DNI; si no hay resultado lanza **ChoferNoEncontradoException** (requisito del enunciado).

```

172 |     public List<Chofer> listarTodos() throws DAOException {
173 |         try (Connection con = ConexionDB.getInstance().getConexion();
174 |             Statement st = con.createStatement();
175 |             ResultSet rs = st.executeQuery(SELECT_BASE + ORDEN)) {
176 |             return leer(rs);
177 |         } catch (SQLException e) {
178 |             throw new DAOException("No se pudo listar los choferes.", e);
179 |         }
180 |     }
181 |
182 |     /** Agrupa las filas del LEFT JOIN: varias filas del mismo chofer = varios camiones autorizados. */

```

Líneas 172 a 182: Listado con **Statement** (consulta fija).

```

183 |     private List<Chofer> leer(ResultSet rs) throws SQLException {
184 |         Map<Long, Chofer> choferes = new LinkedHashMap<>();
185 |         while (rs.next()) {
186 |             long id = rs.getLong("ch_id");
187 |             Chofer chofer = choferes.get(id);
188 |             if (chofer == null) {
189 |                 chofer = Mapeadores.chofer(rs, "ch_");
190 |                 choferes.put(id, chofer);
191 |             }
192 |             rs.getLong("c_id");
193 |             if (!rs.wasNull()) { // si el chofer no tiene camiones, las columnas del camion vienen en NULL
194 |                 chofer.autorizar(Mapeadores.camion(rs, "c_"));
195 |             }
196 |         }
197 |         return new ArrayList<>(choferes.values());
198 |     }
199 |

```

Líneas 183 a 199: Agrupa las filas del LEFT JOIN: varias filas del mismo chofer son varios camiones. Un LinkedHashMap conserva el orden y evita duplicar choferes. Si c_id es NULL, el chofer no tiene camiones.

```

200 |     private void cargarDatos(PreparedStatement ps, Chofer chofer) throws SQLException {
201 |         ps.setString(1, chofer.getNombre());
202 |         ps.setString(2, chofer.getApellido());
203 |         ps.setString(3, chofer.getDni());
204 |         ps.setDate(4, Date.valueOf(chofer.getFechaNacimiento()));
205 |         ps.setString(5, chofer.getCategoria().name());
206 |         ps.setString(6, chofer.getTelefono());
207 |     }
208 |

```

Líneas 200 a 208: Carga los seis parámetros comunes de INSERT y UPDATE (evita repetirlos).

```

209 |     private void guardarAutorizaciones(Connection con, Chofer chofer) throws SQLException {
210 |         try (PreparedStatement ps = con.prepareStatement(INSERT_AUTORIZACION)) {
211 |             for (Camion camion : chofer.getCamionesAutorizados()) {
212 |                 ps.setLong(1, chofer.getId());
213 |                 ps.setLong(2, camion.getId());
214 |                 ps.addBatch();
215 |             }
216 |             ps.executeBatch();
217 |         }
218 |     }
219 |

```

Líneas 209 a 219: Inserta todas las autorizaciones con un **batch** (una sola ida a la base).

```

220 |     private DAOException traducir(SQLException e, Chofer chofer, String mensajeGenerico) throws ValidacionException {
221 |         if (JdbcUtil.esDuplicado(e)) {
222 |             throw new ValidacionException("Ya existe un chofer con DNI " + chofer.getDni() + ".");
223 |         }
224 |         if (JdbcUtil.esClaveForanea(e)) {
225 |             throw new ValidacionException("Alguno de los camiones seleccionados ya no existe.");
226 |         }
227 |         return new DAOException(mensajeGenerico, e);
228 |     }
229 |

```

Líneas 220 a 229: Traduce DNI duplicado y camión inexistente a mensajes de negocio.

logistica-dao / src / main / java / ar / edu / usal / logica / dao / jdbc / ViajeDAOJdbc.java

Para qué existe: Implementación JDBC del DAO de viajes.

Defensa: Es el DAO con más reglas de concurrencia. **insertar** bloquea la fila del camión con SELECT ... FOR UPDATE: si dos administradores asignan el mismo camión a la vez, el segundo espera y luego ve que ya está ocupado (no hay doble asignación). **iniciar** usa transacción para verificar que el chofer no tenga otro viaje en curso. **finalizar** usa un stored procedure con parámetro OUT (CallableStatement). Todas las actualizaciones incluyen chofer_id en el WHERE: un chofer no puede modificar viajes ajenos.

```

1 | package ar.edu.usal.logistica.dao.jdbc;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.ViajeDAO;
4 | import ar.edu.usal.logistica.excepcion.DAOException;
5 | import ar.edu.usal.logistica.excepcion.ValidacionException;
6 | import ar.edu.usal.logistica.modelo.Destino;
7 | import ar.edu.usal.logistica.modelo.EstadoViaje;
8 | import ar.edu.usal.logistica.modelo.EstimacionViaje;
9 | import ar.edu.usal.logistica.modelo.Viaje;
10 |
11 | import java.sql.CallableStatement;
12 | import java.sql.Connection;
13 | import java.sql.PreparedStatement;
14 | import java.sql.ResultSet;
15 | import java.sql.SQLException;
16 | import java.sql.Statement;
17 | import java.sql.Timestamp;
18 | import java.sql.Types;
19 | import java.time.LocalDateTime;
20 | import java.util.ArrayList;
21 | import java.util.List;
22 |

```

Líneas 3 a 22: Importaciones de las clases que usa este archivo: **ViajeDAO**, **DAOException**, **ValidacionException**, **Destino**, **EstadoViaje**, **EstimacionViaje**, **Viaje**, **CallableStatement**, **Connection**, **PreparedStatement**, **ResultSet**, **SQLException**, **Statement**, **Timestamp**, **Types**, **LocalDateTime**, **ArrayList**, **List**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

23 | /** Implementacion JDBC de ViajeDAO. */
24 | class ViajeDAOJdbc implements ViajeDAO {
25 |
26 |     private static final String SELECT_BASE =
27 |         "SELECT v.id, v.origen, v.destino, v.distancia_km, v.dias_estimados, v.litros_estimados, v.tanques, "
28 |         + "v.estado, v.fecha_carga, v.fecha_inicio, v.fecha_fin, "
29 |         + "ch.id AS ch_id, ch.nombre AS ch_nombre, ch.apellido AS ch_apellido, ch.dni AS ch_dni, "
30 |         + "ch.fecha_nacimiento AS ch_fecha_nacimiento, ch.categoria AS ch_categoria, "
31 |         + "ch.telefono AS ch_telefono, "
32 |         + "c.id AS c_id, c.marca AS c_marca, c.modelo AS c_modelo, c.dominio AS c_dominio, "
33 |         + "c.toneladas_max AS c_toneladas_max, c.capacidad_tanque_litros AS c_capacidad_tanque_litros, "
34 |         + "c.consumo_litros_km AS c_consumo_litros_km "
35 |         + "FROM viaje v JOIN chofer ch ON ch.id = v.chofer_id JOIN camion c ON c.id = v.camion_id ";
36 |     private static final String ORDEN = " ORDER BY v.fecha_carga DESC, v.id DESC";
37 |
38 |     private static final String INSERT =
39 |         "INSERT INTO viaje (chofer_id, camion_id, origen, destino, distancia_km, dias_estimados, "
40 |         + "litros_estimados, tanques, estado) VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'ASIGNADO')";
41 |
42 |     @Override

```

Líneas 23 a 42: Consulta base con JOIN de viaje, chofer y camión (alias ch_ y c_), orden por fecha descendente y el INSERT del viaje.

```

43 |     public void insertar(Viaje viaje) throws DAOException, ValidacionException {
44 |         viaje.validar();
45 |         Connection con = null;
46 |         try {
47 |             con = ConexionDB.getInstance().getConexion();
48 |             con.setAutoCommit(false);
49 |

```

Líneas 43 a 49: Valida el viaje (regla de dominio) y abre la transacción.

```

50 |         // Se bloquea la fila del camion: si dos administradores cargan viajes a la vez, el segundo espera
51 |         try (PreparedStatement ps = con.prepareStatement("SELECT id FROM camion WHERE id = ? FOR UPDATE")) {
52 |             ps.setLong(1, viaje.getCamion().getId());
53 |             try (ResultSet rs = ps.executeQuery()) {
54 |                 if (!rs.next()) {
55 |                     throw new ValidacionException("El camión seleccionado ya no existe.");
56 |                 }
57 |             }
58 |         }

```

Líneas 50 a 58: Bloquea la fila del camión (FOR UPDATE) y verifica que exista.

```

59 | // Con el camion bloqueado se verifica que siga libre
60 | try (PreparedStatement ps = con.prepareStatement(
61 |     "SELECT COUNT(*) FROM viaje WHERE camion_id = ? AND estado IN ('ASIGNADO', 'EN_CURSO')")) {
62 |     ps.setLong(1, viaje.getCamion().getId());
63 |     try (ResultSet rs = ps.executeQuery()) {
64 |         rs.next();
65 |         if (rs.getInt(1) > 0) {
66 |             throw new ValidacionException("El camión ya está asignado a otro viaje.");
67 |         }
68 |     }
69 | }

```

Líneas 59 a 69: Con el camión bloqueado, cuenta los viajes ASIGNADO/EN_CURSO: si hay alguno, lanza ValidacionException.

```

70 | try (PreparedStatement ps = con.prepareStatement(INSERT, Statement.RETURN_GENERATED_KEYS)) {
71 |     EstimacionViaje e = viaje.getEstimacion();
72 |     ps.setLong(1, viaje.getChofer().getId());
73 |     ps.setLong(2, viaje.getCamion().getId());
74 |     ps.setString(3, viaje.getOrigen().name());
75 |     ps.setString(4, viaje.getDestino().name());
76 |     ps.setInt(5, e.distanciaKm());
77 |     ps.setInt(6, e.dias());
78 |     ps.setDouble(7, e.litrosTotales());
79 |     ps.setInt(8, e.tanques());
80 |     ps.executeUpdate();
81 |     try (ResultSet claves = ps.getGeneratedKeys()) {
82 |         claves.next();
83 |         viaje.setId(claves.getLong(1));
84 |     }
85 | }

```

Líneas 70 a 85: Inserta el viaje con los resultados del cálculo y recupera su id.

```

86 | con.commit();
87 | } catch (SQLException e) {
88 |     JdbcUtil.rollbackSilencioso(con, e);

```

Líneas 86 a 88: Confirma la transacción (commit). Si alguna línea anterior falló, el catch de abajo hace rollback.

```

89 |     throw new DAOException("No se pudo guardar el viaje.", e);
90 | } catch (ValidacionException e) {
91 |     JdbcUtil.rollbackSilencioso(con, e);
92 |     throw e;
93 | } finally {
94 |     JdbcUtil.cerrar(con);
95 | }
96 | }
97 |
98 | @Override

```

Líneas 89 a 98: Ante error técnico o de negocio hace rollback y propaga la excepción adecuada; finally cierra la conexión.

```

99 | public List<Viaje> listarTodos() throws DAOException {
100 |     try (Connection con = ConexionDB.getInstance().getConexion();
101 |         PreparedStatement ps = con.prepareStatement(SELECT_BASE + ORDEN);
102 |         ResultSet rs = ps.executeQuery()) {
103 |         return leer(rs);
104 |     } catch (SQLException e) {
105 |         throw new DAOException("No se pudo listar los viajes.", e);
106 |     }
107 | }
108 |
109 | @Override

```

Líneas 99 a 109: Lista todos los viajes (vista del administrador) con PreparedStatement.

```

110 | public List<Viaje> listarPorChofer(long choferId) throws DAOException {
111 |     try (Connection con = ConexionDB.getInstance().getConexion();
112 |         PreparedStatement ps = con.prepareStatement(SELECT_BASE + "WHERE v.chofer_id = ?" + ORDEN)) {
113 |         ps.setLong(1, choferId);
114 |         try (ResultSet rs = ps.executeQuery()) {
115 |             return leer(rs);
116 |         }
117 |     } catch (SQLException e) {
118 |         throw new DAOException("No se pudo listar los viajes del chofer.", e);
119 |     }
120 | }
121 |
122 | @Override

```

Líneas 110 a 122: Lista solo los viajes del chofer indicado (vista del chofer).

```

123 | public void iniciar(long viajeId, long choferId) throws DAOException, ValidacionException {
124 |     Connection con = null;
125 |     try {
126 |         con = ConexionDB.getInstance().getConexion();
127 |         con.setAutoCommit(false);
128 |

```

Líneas 123 a 128: Transacción para iniciar un viaje.

```

129 |         // Un chofer no puede tener dos viajes en curso a la vez
130 |         try (PreparedStatement ps = con.prepareStatement(
131 |             "SELECT COUNT(*) FROM viaje WHERE chofer_id = ? AND estado = 'EN_CURSO' FOR UPDATE")) {
132 |             ps.setLong(1, choferId);
133 |             try (ResultSet rs = ps.executeQuery()) {
134 |                 rs.next();
135 |                 if (rs.getInt(1) > 0) {
136 |                     throw new ValidacionException("Ya tenés un viaje en curso. Finalizalo antes de iniciar
137 |                         ↳ otro.");
138 |                 }
139 |             }
140 |             try (PreparedStatement ps = con.prepareStatement(

```

Líneas 129 a 140: Regla: un chofer no puede tener dos viajes en curso.

```

141 |                 "UPDATE viaje SET estado = 'EN_CURSO', fecha_inicio = NOW() "
142 |                     + "WHERE id = ? AND chofer_id = ? AND estado = 'ASIGNADO'")) {
143 |                 ps.setLong(1, viajeId);
144 |                 ps.setLong(2, choferId);
145 |                 if (ps.executeUpdate() == 0) {
146 |                     throw new ValidacionException("El viaje no existe, no es tuyo o ya fue iniciado.");
147 |                 }
148 |             }
149 |             con.commit();
150 |         } catch (SQLException e) {
151 |             JdbcUtil.rollbackSilencioso(con, e);
152 |             throw new DAOException("No se pudo iniciar el viaje.", e);
153 |         } catch (ValidacionException e) {
154 |             JdbcUtil.rollbackSilencioso(con, e);
155 |             throw e;
156 |         } finally {
157 |             JdbcUtil.cerrar(con);
158 |         }
159 |     }
160 | }
161 | @Override

```

Líneas 141 a 161: El UPDATE solo afecta un viaje del chofer y en estado ASIGNADO; si no modificó ninguna fila, informa el motivo.

```

162 | public void finalizar(long viajeId, long choferId) throws DAOException, ValidacionException {
163 |     // Invocacion del stored procedure con dos parametros IN y uno OUT
164 |     try (Connection con = ConexionDB.getInstance().getConexion();
165 |          CallableStatement cs = con.prepareCall("{call sp_finalizar_viaje(?, ?, ?)}")) {
166 |         cs.setLong(1, viajeId);
167 |         cs.setLong(2, choferId);
168 |         cs.registerOutParameter(3, Types.INTEGER);
169 |         cs.execute();
170 |         if (cs.getInt(3) == 0) {
171 |             throw new ValidacionException("El viaje no existe, no es tuyo o todavía no fue iniciado.");
172 |         }
173 |     } catch (SQLException e) {
174 |         throw new DAOException("No se pudo finalizar el viaje.", e);
175 |     }
176 | }
177 |

```

Líneas 162 a 177: CallableStatement: llama a `sp_finalizar_viaje` con dos parámetros de entrada y registra el tercero como **OUT** (entero). Si el procedimiento modificó 0 filas, el viaje no correspondía.

```

178 | private List<Viaje> leer(ResultSet rs) throws SQLException {
179 |     List<Viaje> viajes = new ArrayList<>();
180 |     while (rs.next()) {
181 |         EstimacionViaje estimacion = new EstimacionViaje(rs.getInt("distancia_km"), rs.getInt("dias_estimados"),
182 |                                                             rs.getDouble("litros_estimados"), rs.getInt("tanques"));
183 |         viajes.add(new Viaje(
184 |             rs.getLong("id"),
185 |             Mapeadores.chofer(rs, "ch_"),
186 |             Mapeadores.camion(rs, "c_"),
187 |             Destino.valueOf(rs.getString("origen")),
188 |             Destino.valueOf(rs.getString("destino")),
189 |             estimacion,
190 |             EstadoViaje.valueOf(rs.getString("estado")),
191 |             aFecha(rs.getTimestamp("fecha_carga")),
192 |             aFecha(rs.getTimestamp("fecha_inicio")),
193 |             aFecha(rs.getTimestamp("fecha_fin"))));
194 |     }
195 |     return viajes;
196 | }
197 |

```

Líneas 178 a 197: Convierte cada fila en un `Viaje` completo: estimación, chofer, camión, enums y fechas.

```

198 | private LocalDateTime aFecha(Timestamp timestamp) {
199 |     return timestamp == null ? null : timestamp.toLocalDateTime();
200 | }
201 |

```

Líneas 198 a 201: Convierte `Timestamp` a `LocalDateTime` cuidando el `null` (fechas de inicio/fin aún vacías).

Proyecto MVC - controladores, filtros y utilidades

logistica-web / src / main / java / ar / edu / usal / logistica / web / SesionUtil.java

Para qué existe: Centraliza el manejo de Session y de la cookie «recordarme».

Defensa: Punto del enunciado: «hacer uso de Session y cookie cuando lo considere necesario» y «al cierre de sesión se deberán limpiar tanto la Session como la cookie». La **Session** guarda al usuario autenticado durante la navegación; la **cookie** RECORDARME guarda un token aleatorio (no la contraseña) para volver a entrar sin loguearse. La cookie es **HttpOnly** (JavaScript no puede robarla ante un XSS), **SameSite=Lax** (no se envía en POST desde otros sitios) y **Secure** cuando hay HTTPS. Tener todo en una clase garantiza que login, logout y filtro usen las mismas reglas (mismo path, mismo nombre).

```
1 | package ar.edu.usal.logistica.web;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | import ar.edu.usal.logistica.modelo.Usuario;
4 |
5 | import jakarta.servlet.http.Cookie;
6 | import jakarta.servlet.http.HttpServletRequest;
7 | import jakarta.servlet.http.HttpServletResponse;
8 | import jakarta.servlet.http.HttpSession;
9 |
```

Líneas 3 a 9: Importaciones de las clases que usa este archivo: **Usuario**, **Cookie**, **HttpServletRequest**, **HttpServletResponse**, **HttpSession**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```
10 | /**
11 |  * Centraliza el manejo de la Session y de la cookie "recordarme" para que
12 |  * el filtro, el login y el logout usen exactamente las mismas reglas.
13 |  */
```

Líneas 10 a 13: Documentación.

```
14 | public final class SesionUtil {
15 |
16 |     /** Nombre del atributo de Session donde vive el usuario autenticado. */
17 |     public static final String ATRIBUTO_USUARIO = "usuario";
18 |     /** Nombre de la cookie con el token de sesion recordada. */
19 |     public static final String COOKIE_RECORDARME = "RECORDARME";
20 |     /** Dias que se recuerda la sesion. */
21 |     public static final int DIAS_RECORDAR = 30;
22 |
23 |     private SesionUtil() {
24 |     }
25 | }
```

Líneas 14 a 25: Constantes compartidas: nombre del atributo de Session, nombre de la cookie y cantidad de días que se recuerda (30). Constructor privado: clase utilitaria.

```
26 |     public static Usuario usuarioActual(HttpServletRequest req) {
27 |         HttpSession session = req.getSession(false);
28 |         return session == null ? null : (Usuario) session.getAttribute(ATRIBUTO_USUARIO);
29 |     }
30 | }
```

Líneas 26 a 30: Devuelve el usuario de la Session, o null si no hay sesión. **getSession(false)** no crea una sesión nueva si no existe.

```

31 | /** Devuelve el valor de la cookie "recordarme" o null si no vino en la request. */
32 | public static String tokenRecordado(HttpServletRequest req) {
33 |     Cookie[] cookies = req.getCookies();
34 |     if (cookies == null) {
35 |         return null;
36 |     }
37 |     for (Cookie cookie : cookies) {
38 |         if (COOKIE_RECORDARME.equals(cookie.getName())) {
39 |             return cookie.getValue();
40 |         }
41 |     }
42 |     return null;
43 | }
44 |

```

Líneas 31 a 44: Recorre las cookies de la request y devuelve el token de RECORDARME, o null.

```

45 | /** Crea la cookie persistente. HttpOnly: JavaScript no puede leerla; SameSite=Lax: no viaja en POST de otros
46 |     ↳ sitios. */
47 | public static void guardarCookie(HttpServletRequest req, HttpServletResponse resp, String token) {
48 |     resp.addCookie(crearCookie(req, token, DIAS_RECORDAR * 24 * 60 * 60));
49 | }

```

Líneas 45 a 49: Crea la cookie de sesión recordada con vida de 30 días.

```

50 | /** Borra del navegador la cookie de recordarme y la de la Session (JSESSIONID). */
51 | public static void limpiarCookies(HttpServletRequest req, HttpServletResponse resp) {
52 |     resp.addCookie(crearCookie(req, "", 0));
53 |     Cookie jsession = new Cookie("JSESSIONID", "");
54 |     jsession.setPath(rutaCookie(req));
55 |     jsession.setMaxAge(0);
56 |     jsession.setHttpOnly(true);
57 |     resp.addCookie(jsession);
58 | }
59 |

```

Líneas 50 a 59: Expira (maxAge 0) la cookie de recordarme y también JSESSIONID: cumple con limpiar las cookies al cerrar sesión.

```

60 | private static Cookie crearCookie(HttpServletRequest req, String valor, int segundos) {
61 |     Cookie cookie = new Cookie(COOKIE_RECORDARME, valor);
62 |     cookie.setHttpOnly(true);
63 |     cookie.setSecure(req.isSecure());
64 |     cookie.setPath(rutaCookie(req));
65 |     cookie.setMaxAge(segundos);
66 |     cookie.setAttribute("SameSite", "Lax");
67 |     return cookie;
68 | }
69 |

```

Líneas 60 a 69: Arma la cookie con sus protecciones: HttpOnly, Secure si la conexión es HTTPS, path del contexto y SameSite=Lax.

```

70 | private static String rutaCookie(HttpServletRequest req) {
71 |     String contexto = req.getContextPath();
72 |     return contexto.isEmpty() ? "/" : contexto;
73 | }
74 | }

```

Líneas 70 a 74: El path de la cookie es el contexto de la aplicación (/logistica) para que solo viaje hacia esta app.

logistica-web / src / main / java / ar / edu / usal / logica / web / filtro / EncodingFilter.java

Para qué existe: Filtro que fuerza UTF-8 en todas las requests.

Defensa: Sin esto, los nombres con tilde o eñe (Pérez, Muñoz) llegarían deformados desde los formularios. Está declarado con la anotación @WebFilter (contraste con el filtro de autenticación, declarado en web.xml).

```

1 | package ar.edu.usal.logistica.web.filtro;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import jakarta.servlet.Filter;
4 | import jakarta.servlet.FilterChain;
5 | import jakarta.servlet.ServletException;
6 | import jakarta.servlet.ServletRequest;
7 | import jakarta.servlet.ServletResponse;
8 | import jakarta.servlet.annotation.WebFilter;
9 |
10 | import java.io.IOException;
11 |

```

Líneas 3 a 11: Importaciones de las clases que usa este archivo: **Filter**, **FilterChain**, **ServletException**, **ServletRequest**, **ServletResponse**, **WebFilter**, **IOException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

12 | /** Fuerza UTF-8 en todas las requests para que tildes y enies lleguen bien desde los formularios. */
13 | @WebFilter(filterName = "EncodingFilter", urlPatterns = "/*")
14 | public class EncodingFilter implements Filter {
15 |
16 |     @Override

```

Líneas 12 a 16: @WebFilter lo registra para todas las URLs (/*)

```

17 |         public void doFilter(ServletRequest req, ServletResponse resp, FilterChain chain)
18 |             throws IOException, ServletException {
19 |             req.setCharacterEncoding("UTF-8");
20 |             resp.setCharacterEncoding("UTF-8");
21 |             chain.doFilter(req, resp);
22 |         }
23 |     }

```

Líneas 17 a 23: Establece la codificación de request y response en UTF-8 y continúa con **chain.doFilter**, que pasa al siguiente filtro o servlet.

logistica-web / src / main / java / ar / edu / usal / logistica / web / filtro / AutenticacionFilter.java

Para qué existe: Filtro de seguridad: autenticación, sesión recordada y autorización por perfil.

Defensa: Seguridad centralizada en un único punto: ningún servlet puede olvidarse de controlar el acceso. Esto es **autenticación** (¿quién es?) y **autorización** (¿puede entrar acá?): /admin/* solo ADMIN, /chofer/* solo CHOFER; un chofer que escribe una URL de admin recibe 403. Las páginas privadas se envían con Cache-Control: no-store: al cerrar sesión, el botón «Atrás» no muestra datos privados. Las JSP están bajo WEB-INF, por lo que solo se llega a ellas a través de un controlador.

```

1 | package ar.edu.usal.logistica.web.filtro;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.DAOFactory;
4 | import ar.edu.usal.logistica.excepcion.DAOException;
5 | import ar.edu.usal.logistica.modelo.Usuario;
6 | import ar.edu.usal.logistica.web.SesionUtil;
7 |
8 | import jakarta.servlet.Filter;
9 | import jakarta.servlet.FilterChain;
10 | import jakarta.servlet.ServletException;
11 | import jakarta.servlet.ServletRequest;
12 | import jakarta.servlet.ServletResponse;
13 | import jakarta.servlet.http.HttpServletRequest;
14 | import jakarta.servlet.http.HttpServletResponse;
15 | import jakarta.servlet.http.HttpSession;
16 |
17 | import java.io.IOException;
18 |

```

Líneas 3 a 18: Importaciones de las clases que usa este archivo: **DAOFactory**, **DAOException**, **Usuario**, **SesionUtil**, **Filter**, **FilterChain**, **ServletException**, **ServletRequest**, **ServletResponse**, **HttpServletRequest**, **HttpServletResponse**, **HttpSession**, **IOException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

19 | /**
20 |  * Filtro de seguridad (declarado en web.xml). Se ejecuta antes de cualquier servlet y:
21 |  * 1) deja pasar solo las paginas publicas (login y recursos estaticos);
22 |  * 2) si no hay usuario en Session, intenta recuperarlo con la cookie "recordarme";
23 |  * 3) verifica que el perfil alcance: /admin/* solo ADMIN, /chofer/* solo CHOFER.
24 | */

```

Líneas 19 a 24: Documentación: las tres tareas del filtro.

```

25 | public class AutenticacionFilter implements Filter {
26 |
27 |     @Override

```

Líneas 25 a 27: Implementa la interfaz Filter.

```

28 |     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
29 |         throws IOException, ServletException {
30 |         HttpServletRequest req = (HttpServletRequest) request;
31 |         HttpServletResponse resp = (HttpServletResponse) response;
32 |         String ruta = req.getRequestURI().substring(req.getContextPath().length());
33 |

```

Líneas 28 a 33: Obtiene la ruta pedida sin el contexto (por ejemplo /admin/choferes).

```

34 |         if (esPublica(ruta)) {
35 |             chain.doFilter(req, resp);
36 |             return;
37 |         }
38 |

```

Líneas 34 a 38: Las rutas públicas (login y archivos estáticos) pasan sin controles.

```

39 |         Usuario usuario = SesionUtil.usuarioActual(req);
40 |         if (usuario == null) {
41 |             usuario = recuperarDesdeCookie(req);
42 |         }
43 |         if (usuario == null) {

```

Líneas 39 a 43: Busca el usuario en la Session; si no hay, intenta recuperarlo con la cookie «recordarme».

```

44 |             resp.sendRedirect(req.getContextPath() + "/login");
45 |             return;
46 |         }
47 |

```

Líneas 44 a 47: Si sigue sin haber usuario, redirige a /login.

```

48 |         // Las paginas privadas no deben quedar en cache: al cerrar sesion, "Atras" no las muestra
49 |         resp.setHeader("Cache-Control", "no-store, no-cache, must-revalidate");
50 |         resp.setHeader("Pragma", "no-cache");
51 |         resp.setDateHeader("Expires", 0);
52 |

```

Líneas 48 a 52: Encabezados anti-cache para las páginas privadas.

```

53 |         boolean rutaAdmin = ruta.equals("/admin") || ruta.startsWith("/admin/");
54 |         boolean rutaChofer = ruta.equals("/chofer") || ruta.startsWith("/chofer/");
55 |         if ((rutaAdmin && !usuario.esAdmin()) || (rutaChofer && usuario.esAdmin())) {
56 |             resp.sendError(HttpServletResponse.SC_FORBIDDEN);
57 |             return;
58 |         }
59 |         chain.doFilter(req, resp);
60 |     }
61 |

```

Líneas 53 a 61: Autorización por perfil: un ADMIN no entra a /chofer y un CHOFER no entra a /admin; en ese caso responde 403 (prohibido). Si todo está bien, sigue la cadena.

```

62 |     private boolean esPublica(String ruta) {
63 |         return ruta.equals("/login") || ruta.startsWith("/css/") || ruta.startsWith("/js/");
64 |     }
65 |

```

Líneas 62 a 65: Rutas públicas: /login, /css/ y /js/.

```

66 | /** Si la cookie trae un token vigente, se reconstruye la Session sin pedir contrasena. */
67 | private Usuario recuperarDesdeCookie(HttpServletRequest req) throws ServletException {
68 |     String token = SesionUtil.tokenRecordado(req);
69 |     if (token == null || token.isEmpty()) {
70 |         return null;
71 |     }
72 |     try {
73 |         Usuario usuario = DAOFactory.getInstance().getUsuarioDAO().buscarPorToken(token);
74 |         if (usuario != null) {
75 |             HttpSession session = req.getSession(true);
76 |             session.setAttribute(SesionUtil.ATRIBUTO_USUARIO, usuario);
77 |         }
78 |         return usuario;
79 |     } catch (DAOException e) {
80 |         throw new ServletException(e);
81 |     }
82 | }
83 | }

```

Líneas 66 a 83: Si hay token y es vigente en la base, reconstruye la Session con el usuario (no pide la contraseña). Si el token no existe o venció, devuelve null.

logistica-web / src / main / java / ar / edu / usal / logistica / web / controlador / BaseServlet.java

Para qué existe: Clase base (herencia) de todos los controladores.

Defensa: Punto 02: **herencia** para no repetir código. Concentra forward, redirect, mensajes y lectura de parámetros. Aplica el patrón **PRG (Post/Redirect/Get)**: después de un POST siempre se redirige, así que recargar la página (F5) no repite la operación. Las vistas están en WEB-INF, por lo que solo se alcanzan vía forward desde un controlador (**MVC**: el controlador decide, la vista solo muestra).

```

1 | package ar.edu.usal.logistica.web.controlador;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.web.SesionUtil;
4 |
5 | import jakarta.servlet.ServletException;
6 | import jakarta.servlet.http.HttpServlet;
7 | import jakarta.servlet.http.HttpServletRequest;
8 | import jakarta.servlet.http.HttpServletResponse;
9 |
10 | import java.io.IOException;
11 |

```

Líneas 3 a 11: Importaciones de las clases que usa este archivo: **SesionUtil**, **ServletException**, **HttpServlet**, **HttpServletRequest**, **HttpServletResponse**, **IOException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

12 | /**
13 |  * Clase base de los controladores (herencia): junta las utilidades que todos
14 |  * necesitan para no repetir código. Las vistas JSP viven en WEB-INF, por lo que
15 |  * nadie puede abrirlas directamente: siempre se pasa por un controlador.
16 |  */

```

Líneas 12 a 16: Documentación.

```

17 | public abstract class BaseServlet extends HttpServlet {
18 |
19 |     private static final long serialVersionUID = 1L;
20 |

```

Líneas 17 a 20: Clase abstracta que extiende HttpServlet.

```

21 | /** Reenvia (forward) a una vista JSP. */
22 | protected void vista(HttpServletRequest req, HttpServletResponse resp, String jsp)
23 |     throws ServletException, IOException {
24 |     req.getRequestDispatcher("/WEB-INF/jsp/" + jsp).forward(req, resp);
25 | }
26 |

```

Líneas 21 a 26: vista(): forward interno a una JSP de WEB-INF (el navegador no cambia de URL).

```

27 |     /** Redirige dentro de la aplicacion (patron Post/Redirect/Get). */
28 |     protected void redirigir(HttpServletRequest req, HttpServletResponse resp, String ruta) throws IOException {
29 |         resp.sendRedirect(req.getContextPath() + ruta);
30 |     }
31 |

```

Líneas 27 a 31: redirigir(): respuesta 302 con la URL completa (contexto + ruta).

```

32 |     /** Mensaje para la proxima pagina (sobrevive a un redirect porque vive en la Session). */
33 |     protected void mensajeFlash(HttpServletRequest req, String tipo, String texto) {
34 |         req.getSession().setAttribute("flashTipo", tipo);
35 |         req.getSession().setAttribute("flashTexto", texto);
36 |     }
37 |

```

Líneas 32 a 37: mensajeFlash(): guarda el mensaje en la Session para mostrarlo en la siguiente página tras un redirect (luego la JSP lo borra).

```

38 |     /** Mensaje para la pagina que se esta por mostrar ahora mismo (forward). */
39 |     protected void mensajeAhora(HttpServletRequest req, String tipo, String texto) {
40 |         req.setAttribute("alertaTipo", tipo);
41 |         req.setAttribute("alertaTexto", texto);
42 |     }
43 |

```

Líneas 38 a 43: mensajeAhora(): mensaje en la request, para cuando se muestra la vista directamente (forward).

```

44 |     protected Long parametroLong(HttpServletRequest req, String nombre) {
45 |         String valor = req.getParameter(nombre);
46 |         if (valor == null || valor.isBlank()) {
47 |             return null;
48 |         }
49 |         try {
50 |             return Long.valueOf(valor.trim());
51 |         } catch (NumberFormatException e) {
52 |             return null;
53 |         }
54 |     }
55 |

```

Líneas 44 a 55: Lee un parámetro numérico de forma segura: devuelve null si falta o no es un número, sin lanzar excepción.

```

56 |     protected String parametro(HttpServletRequest req, String nombre) {
57 |         String valor = req.getParameter(nombre);
58 |         return valor == null ? "" : valor.trim();
59 |     }
60 |

```

Líneas 56 a 60: Lee un parámetro de texto: nunca null y sin espacios sobrantes.

```

61 |     protected Long choferIdDeSesion(HttpServletRequest req) {
62 |         return SesionUtil.usuarioActual(req).getChoferId();
63 |     }
64 | }

```

Líneas 61 a 64: El id del chofer se toma siempre de la Session (nunca de un parámetro que el usuario podría modificar).

logistica-web / src / main / java / ar / edu / usal / logistica / web / controlador / LoginServlet.java

Para qué existe: Pantalla principal de inicio de sesión (declarado en web.xml).

Defensa: **doGet** solo muestra el formulario; **doPost** procesa las credenciales (punto 10: el método HTTP correcto; las credenciales nunca viajan en la URL). Mismo mensaje si falla el usuario o la clave: no se revela qué usuarios existen. **changeSessionId()** tras autenticar evita la fijación de sesión. La contraseña se compara contra el hash PBKDF2. Con «recordarme» se crea un token aleatorio guardado en la base y en una cookie, cumpliendo «si un usuario ya inició sesión, el mismo sea recordado sin volver a iniciar sesión».

```

1 | package ar.edu.usal.logistica.web.controlador;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.DAOFactory;
4 | import ar.edu.usal.logistica.dao.UsuarioDAO;
5 | import ar.edu.usal.logistica.excepcion.DAOException;
6 | import ar.edu.usal.logistica.modelo.Usuario;
7 | import ar.edu.usal.logistica.util.PasswordUtil;
8 | import ar.edu.usal.logistica.web.SesionUtil;
9 |
10 | import jakarta.servlet.ServletException;
11 | import jakarta.servlet.http.HttpServletRequest;
12 | import jakarta.servlet.http.HttpServletResponse;
13 | import jakarta.servlet.http.HttpSession;
14 |
15 | import java.io.IOException;
16 | import java.time.LocalDateTime;
17 |

```

Líneas 3 a 17: Importaciones de las clases que usa este archivo: **DAOFactory**, **UsuarioDAO**, **DAOException**, **Usuario**, **PasswordUtil**, **SesionUtil**, **ServletException**, **HttpServletRequest**, **HttpServletResponse**, **HttpSession**, **IOException**, **LocalDateTime**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

18 | /**
19 |  * Pantalla principal de inicio de sesion. Esta clase se declara en web.xml
20 |  * (no usa @WebServlet). GET muestra el formulario; POST autentica.
21 |  */

```

Líneas 18 a 21: Documentación.

```

22 | public class LoginServlet extends BaseServlet {
23 |
24 |     private static final long serialVersionUID = 1L;
25 |
26 |     @Override

```

Líneas 22 a 26: Hereda de BaseServlet.

```

27 |         protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
28 |             if (SesionUtil.usuarioActual(req) != null) {
29 |                 redirigir(req, resp, "/inicio");
30 |                 return;
31 |             }
32 |             vista(req, resp, "login.jsp");
33 |         }
34 |
35 |         @Override

```

Líneas 27 a 35: GET: si ya hay sesión, va al inicio; si no, muestra login.jsp.

```

36 |         protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
37 |             String username = parametro(req, "username");
38 |             String password = req.getParameter("password");
39 |             boolean recordar = "on".equals(req.getParameter("recordar"));
40 |
41 |             try {

```

Líneas 36 a 41: POST: lee usuario, contraseña y si tildó «recordarme».

```

42 |                 UsuarioDAO usuarioDAO = DAOFactory.getInstance().getUsuarioDAO();
43 |                 Usuario usuario = usuarioDAO.buscarPorUsername(username);
44 |                 // Mismo mensaje si falla el usuario o la clave: no se revela cual de los dos es incorrecto
45 |                 if (usuario == null || !PasswordUtil.verificar(password, usuario.getPasswordHash())) {
46 |                     req.setAttribute("username", username);
47 |                     mensajeAhora(req, "error", "Usuario o contraseña incorrectos.");
48 |                     vista(req, resp, "login.jsp");
49 |                     return;
50 |                 }
51 |

```

Líneas 42 a 51: Busca el usuario por nombre y verifica la contraseña contra el hash. Si falla, vuelve al formulario conservando el usuario escrito y con un mensaje genérico.

```

52 |         // Se cambia el id de Session al autenticar para evitar la fijacion de sesion
53 |         HttpSession session = req.getSession(false);
54 |         if (session != null) {
55 |             req.changeSessionId();
56 |         } else {
57 |             session = req.getSession(true);
58 |         }
59 |         session.setAttribute(SesionUtil.ATRIBUTO_USUARIO, usuario);
60 |

```

Líneas 52 a 60: Si ya existía una Session, le cambia el id (seguridad); si no, la crea. Guarda el usuario en la Session.

```

61 |         if (recordar) {
62 |             String token = PasswordUtil.generarToken();
63 |             usuarioDAO.guardarToken(token, usuario.getId(),
64 |                                     LocalDate.now().plusDays(SesionUtil.DIAS_RECORDAR));
65 |             SesionUtil.guardarCookie(req, resp, token);
66 |         }
67 |

```

Líneas 61 a 65: «Recordarme»: genera el token, lo guarda con vencimiento en 30 días y lo envía en la cookie.

```

66 |         redirigir(req, resp, "/inicio");
67 |

```

Línea 66: Redirige al inicio (PRG).

```

67 |         } catch (DAOException e) {
68 |             throw new ServletException(e);
69 |         }
70 |     }
71 | }

```

Líneas 67 a 71: Un error técnico se envuelve en ServletException y lo atiende la página de error de web.xml.

logistica-web / src / main / java / ar / edu / usal / logica / web / controlador / LogoutServlet.java

Para qué existe: Cierre de sesión (declarado en web.xml).

Defensa: Cumple «al cierre de sesión se deberán limpiar tanto la Session como la cookie»: borra el token de la base (queda inutilizable aunque alguien lo haya copiado), **invalida la Session** y expira las cookies. Solo acepta **POST** porque cierra sesión (cambia el estado): un GET podría dispararse con un simple enlace o imagen de otro sitio. El bloque finally garantiza que la sesión se cierre incluso si falla el borrado del token.

```

1 | package ar.edu.usal.logistica.web.controlador;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.DAOFactory;
4 | import ar.edu.usal.logistica.excepcion.DAOException;
5 | import ar.edu.usal.logistica.web.SesionUtil;
6 |
7 | import jakarta.servlet.ServletException;
8 | import jakarta.servlet.http.HttpServletRequest;
9 | import jakarta.servlet.http.HttpServletResponse;
10 | import jakarta.servlet.http.HttpSession;
11 |
12 | import java.io.IOException;
13 |

```

Líneas 3 a 13: Importaciones de las clases que usa este archivo: **DAOFactory**, **DAOException**, **SesionUtil**, **ServletException**, **HttpServletRequest**, **HttpServletResponse**, **HttpSession**, **IOException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

14 | /**
15 |  * Cierre de sesion (declarado en web.xml). Limpia la Session y las cookies,
16 |  * como pide el enunciado. Solo acepta POST porque cambia el estado del sistema.
17 |  */

```

Líneas 14 a 17: Documentación.

```

18 | public class LogoutServlet extends BaseServlet {
19 |
20 |     private static final long serialVersionUID = 1L;
21 |
22 |     @Override

```

Líneas 18 a 22: Hereda de BaseServlet.

```

23 |     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
24 |         // Una URL tipeada no debe cerrar la sesion: se vuelve al inicio
25 |         redirigir(req, resp, "/inicio");
26 |     }
27 |
28 |     @Override

```

Líneas 23 a 28: Un GET (URL tipeada) no cierra la sesión: vuelve al inicio.

```

29 |     protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
30 |         try {
31 |             // El token recordado tambien se elimina de la base para que no pueda reutilizarse
32 |             String token = SesionUtil.tokenRecordado(req);
33 |             if (token != null) {
34 |                 DAOFactory.getInstance().getUsuarioDAO().eliminarToken(token);
35 |             }
36 |         } catch (DAOException e) {
37 |             throw new ServletException(e);

```

Líneas 29 a 37: POST: elimina el token recordado de la base si la cookie lo traía.

```

38 |         } finally {
39 |             HttpSession sesion = req.getSession(false);
40 |             if (sesion != null) {
41 |                 sesion.invalidate(); // destruye la Session y todos sus atributos
42 |             }
43 |             SesionUtil.limpiarCookies(req, resp); // borra del navegador la cookie de sesion y la de recordarme
44 |         }
45 |         redirigir(req, resp, "/login");
46 |     }
47 | }

```

Líneas 38 a 47: Siempre: invalida la Session (destruye todos sus atributos) y expira las cookies del navegador. Luego redirige al login.

logistica-web / src / main / java / ar / edu / usal / logica / web / controlador / InicioServlet.java

Para qué existe: Punto de entrada tras autenticarse.

Defensa: Decide el destino según el perfil: el administrador va a Viajes y el chofer a Mis viajes. Declarado con @WebServlet (configuración por anotación).

```

1 | package ar.edu.usal.logistica.web.controlador;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.modelo.Usuario;
4 | import ar.edu.usal.logistica.web.SesionUtil;
5 |
6 | import jakarta.servlet.annotation.WebServlet;
7 | import jakarta.servlet.http.HttpServletRequest;
8 | import jakarta.servlet.http.HttpServletResponse;
9 |
10 | import java.io.IOException;
11 |

```

Líneas 3 a 11: Importaciones de las clases que usa este archivo: **Usuario, SesionUtil, WebServlet, HttpServletRequest, HttpServletResponse, IOException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

12 | /** Punto de entrada despues de autenticarse: deriva a cada perfil a su pantalla. */
13 | @WebServlet(name = "InicioServlet", urlPatterns = "/inicio")
14 | public class InicioServlet extends BaseServlet {
15 |
16 |     private static final long serialVersionUID = 1L;
17 |
18 |     @Override

```

Líneas 12 a 18: @WebServlet registra el servlet en /inicio.

```

19 |     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
20 |         Usuario usuario = SesionUtil.usuarioActual(req);
21 |         redirigir(req, resp, usuario.esAdmin() ? "/admin/viajes" : "/chofer/viajes");
22 |     }
23 | }

```

Líneas 19 a 23: Redirige según el rol del usuario de la Session.

logistica-web / src / main / java / ar / edu / usal / logistica / web / controlador / CamionServlet.java

Para qué existe: Controlador del ABM de camiones (solo administrador).

Defensa: **GET** solo consulta (lista, formulario vacío, formulario de edición); **POST** modifica datos (guardar, eliminar): uso correcto de métodos HTTP (punto 10). Los navegadores no envían DELETE/PUT desde un formulario, por eso la baja es un POST con accion=eliminar. Ante un error de validación se vuelve al formulario **conservando lo escrito** y se muestra el mensaje; tras un éxito se redirige (PRG). El controlador no contiene SQL ni reglas: delega en el DAO y en el modelo.

```

1 | package ar.edu.usal.logistica.web.controlador;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.CamionDAO;
4 | import ar.edu.usal.logistica.dao.DAOFactory;
5 | import ar.edu.usal.logistica.excepcion.DAOException;
6 | import ar.edu.usal.logistica.excepcion.ValidacionException;
7 | import ar.edu.usal.logistica.modelo.Camion;
8 |
9 | import jakarta.servlet.ServletException;
10 | import jakarta.servlet.annotation.WebServlet;
11 | import jakarta.servlet.http.HttpServletRequest;
12 | import jakarta.servlet.http.HttpServletResponse;
13 |
14 | import java.io.IOException;
15 | import java.util.HashMap;
16 | import java.util.Map;
17 |

```

Líneas 3 a 17: Importaciones de las clases que usa este archivo: **CamionDAO**, **DAOFactory**, **DAOException**, **ValidacionException**, **Camion**, **ServletException**, **WebServlet**, **HttpServletRequest**, **HttpServletResponse**, **IOException**, **HashMap**, **Map**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

18 | /**
19 |  * ABM de camiones (solo administrador). GET consulta (lista, formulario);
20 |  * POST modifica datos (guardar, eliminar).
21 |  */
22 | @WebServlet(name = "CamionServlet", urlPatterns = "/admin/camiones")

```

Líneas 18 a 22: Documentación.

```

23 | public class CamionServlet extends BaseServlet {
24 |
25 |     private static final long serialVersionUID = 1L;
26 |
27 |     private final CamionDAO camionDAO = DAOFactory.getInstance().getCamionDAO();
28 |
29 |     @Override

```

Líneas 23 a 29: El DAO se obtiene de la **fábrica** y se guarda en un atributo; es seguro compartirlo porque no tiene estado.

```

30 | protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
31 |     String accion = parametro(req, "accion");
32 |     try {
33 |         if ("nuevo".equals(accion)) {
34 |             req.setAttribute("valores", new HashMap<String, String>());
35 |             vista(req, resp, "admin/camiones/form.jsp");
36 |         } else if ("editar".equals(accion)) {
37 |             Camion camion = buscarCamion(req);
38 |             if (camion == null) {
39 |                 mensajeFlash(req, "error", "El camión solicitado no existe.");
40 |                 redirigir(req, resp, "/admin/camiones");
41 |                 return;
42 |             }
43 |             req.setAttribute("valores", aMapa(camion));
44 |             vista(req, resp, "admin/camiones/form.jsp");
45 |         } else {
46 |             req.setAttribute("camiones", camionDAO.listarTodos());
47 |             vista(req, resp, "admin/camiones/lista.jsp");
48 |         }
49 |     } catch (DAOException e) {
50 |         throw new ServletException(e);
51 |     }
52 | }
53 |
54 | @Override

```

Líneas 30 a 54: GET: según el parámetro accion: «nuevo» muestra el formulario vacío, «editar» carga el camión y lo muestra (o avisa si no existe) y sin acción lista todos.

```

55 | protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
56 |     String accion = parametro(req, "accion");
57 |     try {
58 |         if ("guardar".equals(accion)) {
59 |             guardar(req, resp);
60 |         } else if ("eliminar".equals(accion)) {
61 |             eliminar(req, resp);
62 |         } else {
63 |             resp.sendError(HttpServletResponse.SC_BAD_REQUEST, "Acción desconocida");
64 |         }
65 |     } catch (DAOException e) {
66 |         throw new ServletException(e);
67 |     }
68 | }
69 |

```

Líneas 55 a 69: POST: despacha a guardar o eliminar; una acción desconocida responde 400 (solicitud incorrecta).

```

70 | private void guardar(HttpServletRequest req, HttpServletResponse resp)
71 |     throws DAOException, ServletException, IOException {
72 |     Long id = parametroLong(req, "id");
73 |     try {
74 |         Camion camion = new Camion(id, parametro(req, "marca"), parametro(req, "modelo"),
75 |             parametro(req, "dominio"), numero(req, "toneladasMaximas", "las toneladas máximas"),
76 |             numero(req, "capacidadTanqueLitros", "la capacidad del tanque"),
77 |             numero(req, "consumoLitrosPorKm", "el consumo por km"));
78 |         if (id == null) {
79 |             camionDAO.insertar(camion);
80 |             mensajeFlash(req, "success", "Camión " + camion.getDominio() + " creado correctamente.");
81 |         } else {
82 |             camionDAO.actualizar(camion);
83 |             mensajeFlash(req, "success", "Camión " + camion.getDominio() + " modificado correctamente.");
84 |         }
85 |         redirigir(req, resp, "/admin/camiones");

```

Líneas 70 a 85: Arma el Camion con los datos del formulario. Sin id es un alta (insertar); con id, una modificación (actualizar). Luego deja un mensaje y redirige.

```

86 |         } catch (ValidacionException e) {
87 |             // Se vuelve al formulario conservando lo que el usuario habia escrito
88 |             Map<String, String> valores = new HashMap<>();
89 |             for (String campo : new String[] {"id", "marca", "modelo", "dominio", "toneladasMaximas",
90 |                 "capacidadTanqueLitros", "consumoLitrosPorKm"}) {
91 |                 valores.put(campo, parametro(req, campo));
92 |             }
93 |             req.setAttribute("valores", valores);
94 |             mensajeAhora(req, "warning", e.getMessage());
95 |             vista(req, resp, "admin/camiones/form.jsp");
96 |         }
97 |     }
98 |

```

Líneas 86 a 98: Error de validación.

```

99 |     private void eliminar(HttpServletRequest req, HttpServletResponse resp) throws DAOException, IOException {
100 |         Long id = parametroLong(req, "id");
101 |         try {
102 |             if (id == null) {
103 |                 throw new ValidacionException("Falta indicar el camión a eliminar.");
104 |             }
105 |             camionDAO.eliminar(id);
106 |             mensajeFlash(req, "success", "Camión eliminado correctamente.");
107 |         } catch (ValidacionException e) {
108 |             mensajeFlash(req, "warning", e.getMessage());
109 |         }
110 |         redirigir(req, resp, "/admin/camiones");
111 |     }
112 |

```

Líneas 99 a 112: Baja: valida que venga el id, elimina y avisa; si el DAO rechaza (tiene viajes) muestra el motivo.

```

113 |     private Camion buscarCamion(HttpServletRequest req) throws DAOException {
114 |         Long id = parametroLong(req, "id");
115 |         return id == null ? null : camionDAO.buscarPorId(id);
116 |     }
117 |

```

Líneas 113 a 117: Auxiliar: busca por el id recibido.

```

118 |     private double numero(HttpServletRequest req, String campo, String descripcion) throws ValidacionException {
119 |         try {
120 |             // Se acepta la coma decimal, habitual en Argentina
121 |             return Double.parseDouble(parametro(req, campo).replace(',', '.'));
122 |         } catch (NumberFormatException e) {
123 |             throw new ValidacionException("Ingrese un número válido para " + descripcion + ".");
124 |         }
125 |     }
126 |

```

Líneas 118 a 126: Convierte texto a número aceptando coma decimal; si no es válido, lanza una ValidacionException con mensaje claro.

```

127 |     private Map<String, String> aMapa(Camion camion) {
128 |         Map<String, String> valores = new HashMap<>();
129 |         valores.put("id", String.valueOf(camion.getId()));
130 |         valores.put("marca", camion.getMarca());
131 |         valores.put("modelo", camion.getModelo());
132 |         valores.put("dominio", camion.getDominio());
133 |         valores.put("toneladasMaximas", String.valueOf(camion.getToneladasMaximas()));
134 |         valores.put("capacidadTanqueLitros", String.valueOf(camion.getCapacidadTanqueLitros()));
135 |         valores.put("consumoLitrosPorKm", String.valueOf(camion.getConsumoLitrosPorKm()));
136 |         return valores;
137 |     }
138 |

```

Líneas 127 a 138: Convierte el camión en un Map de textos que la JSP usa para rellenar el formulario de edición.

logistica-web / src / main / java / ar / edu / usal / logica / web / controlador / ChoferServlet.java

Para qué existe: Controlador del ABM de choferes (solo administrador), incluyendo los camiones autorizados.

Defensa: Mismo esquema que CamionServlet (GET consulta, POST modifica, PRG). La fecha, la categoría y los camiones seleccionados se convierten y validan en el servidor aunque el HTML ya tenga

validaciones: **nunca se confía en el navegador**. Al crear un chofer se guarda también su usuario y se informa cuál es.

```
1 | package ar.edu.usal.logistica.web.controlador;
2 |
```

Líneas 1 a 2: Paquete.

```
3 | import ar.edu.usal.logistica.dao.CamionDAO;
4 | import ar.edu.usal.logistica.dao.ChoferDAO;
5 | import ar.edu.usal.logistica.dao.DAOFactory;
6 | import ar.edu.usal.logistica.excepcion.DAOException;
7 | import ar.edu.usal.logistica.excepcion.ValidacionException;
8 | import ar.edu.usal.logistica.modelo.Camion;
9 | import ar.edu.usal.logistica.modelo.Categoria;
10 | import ar.edu.usal.logistica.modelo.Chofer;
11 |
12 | import jakarta.servlet.ServletException;
13 | import jakarta.servlet.annotation.WebServlet;
14 | import jakarta.servlet.http.HttpServletRequest;
15 | import jakarta.servlet.http.HttpServletResponse;
16 |
17 | import java.io.IOException;
18 | import java.time.LocalDate;
19 | import java.time.format.DateTimeParseException;
20 | import java.util.ArrayList;
21 | import java.util.HashMap;
22 | import java.util.HashSet;
23 | import java.util.List;
24 | import java.util.Map;
25 | import java.util.Set;
26 |
```

Líneas 3 a 26: Importaciones de las clases que usa este archivo: **CamionDAO, ChoferDAO, DAOFactory, DAOException, ValidacionException, Camion, Categoria, Chofer, ServletException,@WebServlet, HttpServletRequest, HttpServletResponse, IOException, LocalDate, DateTimeParseException, ArrayList, HashMap, HashSet, List, Map, Set**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```
27 | /**
28 |  * ABM de choferes (solo administrador), incluyendo los camiones que cada uno
29 |  * esta autorizado a manejar. GET consulta; POST guarda o elimina.
30 |  */
31 | @WebServlet(name = "ChoferServlet", urlPatterns = "/admin/choferes")
```

Líneas 27 a 31: Documentación.

```
32 | public class ChoferServlet extends BaseServlet {
33 |
34 |     private static final long serialVersionUID = 1L;
35 |
36 |     private final ChoferDAO choferDAO = DAOFactory.getInstance().getChoferDAO();
37 |     private final CamionDAO camionDAO = DAOFactory.getInstance().getCamionDAO();
38 |
39 |     @Override
```

Líneas 32 a 39: Dos DAO obtenidos de la fábrica: choferes y camiones.

```

40 | protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
41 |     String accion = parametro(req, "accion");
42 |     try {
43 |         if ("nuevo".equals(accion)) {
44 |             prepararFormulario(req, new HashMap<>(), new HashSet<>());
45 |             vista(req, resp, "admin/choferes/form.jsp");
46 |         } else if ("editar".equals(accion)) {
47 |             Long id = parametroLong(req, "id");
48 |             Chofer chofer = id == null ? null : choferDAO.buscarPorId(id);
49 |             if (chofer == null) {
50 |                 mensajeFlash(req, "error", "El chofer solicitado no existe.");
51 |                 redirigir(req, resp, "/admin/choferes");
52 |                 return;
53 |             }
54 |             Set<Long> seleccionados = new HashSet<>();
55 |             chofer.getCamionesAutorizados().forEach(c -> seleccionados.add(c.getId()));
56 |             prepararFormulario(req, aMapa(chofer), seleccionados);
57 |             vista(req, resp, "admin/choferes/form.jsp");
58 |         } else {
59 |             req.setAttribute("choferes", choferDAO.listarTodos());
60 |             vista(req, resp, "admin/choferes/lista.jsp");
61 |         }
62 |     } catch (DAOException e) {
63 |         throw new ServletException(e);
64 |     }
65 | }
66 |
67 | @Override

```

Líneas 40 a 67: GET: «nuevo» muestra el formulario vacío; «editar» busca el chofer, marca sus camiones autorizados y muestra el formulario; sin acción lista todos.

```

68 | protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
69 |     String accion = parametro(req, "accion");
70 |     try {
71 |         if ("guardar".equals(accion)) {
72 |             guardar(req, resp);
73 |         } else if ("eliminar".equals(accion)) {
74 |             eliminar(req, resp);
75 |         } else {
76 |             resp.sendError(HttpServletResponse.SC_BAD_REQUEST, "Acción desconocida");
77 |         }
78 |     } catch (DAOException e) {
79 |         throw new ServletException(e);
80 |     }
81 | }
82 |

```

Líneas 68 a 82: POST: despacha guardar o eliminar.

```

83 | private void guardar(HttpServletRequest req, HttpServletResponse resp)
84 |     throws DAOException, ServletException, IOException {
85 |     Long id = parametroLong(req, "id");
86 |     try {
87 |         Chofer chofer = new Chofer(id, parametro(req, "nombre"), parametro(req, "apellido"),
88 |             parametro(req, "dni"), fecha(req), categoria(req), parametro(req, "telefono"));
89 |         for (Camion camion : camionesSeleccionados(req)) {
90 |             chofer.autorizar(camion);
91 |         }

```

Líneas 83 a 91: Arma el Chofer con los datos del formulario (convirtiendo fecha y categoría) y le agrega cada camión seleccionado con **autorizar()**.

```

92 |         if (id == null) {
93 |             choferDAO.insertar(chofer, req.getParameter("password"));
94 |             mensajeFlash(req, "success", "Chofer " + chofer.getNombreCompleto()
95 |                 + " creado. Usuario de acceso: " + chofer.getDni() + ".");
96 |         } else {
97 |             choferDAO.actualizar(chofer);
98 |             mensajeFlash(req, "success", "Chofer " + chofer.getNombreCompleto() + " modificado correctamente.");
99 |         }
100 |         redirigir(req, resp, "/admin/choferes");

```

Líneas 92 a 100: Sin id → alta (con contraseña inicial); con id → modificación.

```

101 |         } catch (ValidacionException e) {
102 |             Map<String, String> valores = new HashMap<>();
103 |             for (String campo : new String[] { "id", "nombre", "apellido", "dni", "fechaNacimiento", "categoria",
104 |                 "telefono" }) {
105 |                 valores.put(campo, parametro(req, campo));
106 |             }
107 |             Set<Long> seleccionados = new HashSet<>();
108 |             String[] ids = req.getParameterValues("camionIds");
109 |             if (ids != null) {
110 |                 for (String valor : ids) {
111 |                     try {
112 |                         seleccionados.add(Long.valueOf(valor));
113 |                     } catch (NumberFormatException ignorado) {
114 |                         // Un id mal formado simplemente no se marca
115 |                     }
116 |                 }
117 |             }
118 |             prepararFormulario(req, valores, seleccionados);
119 |             mensajeAhora(req, "warning", e.getMessage());
120 |             vista(req, resp, "admin/choferes/form.jsp");
121 |         }
122 |     }
123 | }

```

Líneas 101 a 123: Si hay error de validación, reconstruye los valores ingresados y los camiones marcados y vuelve al formulario con el mensaje.

```

124 |     private void eliminar(HttpServletRequest req, HttpServletResponse resp) throws DAOException, IOException {
125 |         Long id = parametroLong(req, "id");
126 |         try {
127 |             if (id == null) {
128 |                 throw new ValidacionException("Falta indicar el chofer a eliminar.");
129 |             }
130 |             choferDAO.eliminar(id);
131 |             mensajeFlash(req, "success", "Chofer eliminado correctamente.");
132 |         } catch (ValidacionException e) {
133 |             mensajeFlash(req, "warning", e.getMessage());
134 |         }
135 |         redirigir(req, resp, "/admin/choferes");
136 |     }
137 | }

```

Líneas 124 a 137: Baja del chofer; si tiene viajes el DAO informa que no se puede.

```

138 |     private void prepararFormulario(HttpServletRequest req, Map<String, String> valores, Set<Long> seleccionados)
139 |         throws DAOException {
140 |         req.setAttribute("valores", valores);
141 |         req.setAttribute("camionesSeleccionados", seleccionados);
142 |         req.setAttribute("camiones", camionDAO.listarTodos());
143 |         req.setAttribute("categorias", Categoria.values());
144 |     }
145 | }

```

Líneas 138 a 145: Deja en la request todo lo que necesita la JSP: valores, camiones seleccionados, lista de camiones y categorías.

```

146 |     private LocalDate fecha(HttpServletRequest req) throws ValidacionException {
147 |         try {
148 |             return LocalDate.parse(parametro(req, "fechaNacimiento"));
149 |         } catch (DateTimeParseException e) {
150 |             throw new ValidacionException("La fecha de nacimiento no es válida.");
151 |         }
152 |     }
153 | }

```

Líneas 146 a 153: Convierte el texto yyyy-MM-dd del input date; si es inválido, ValidacionException.

```

154 |     private Categoria categoria(HttpServletRequest req) throws ValidacionException {
155 |         try {
156 |             return Categoria.valueOf(parametro(req, "categoria"));
157 |         } catch (IllegalArgumentException e) {
158 |             throw new ValidacionException("Seleccione una categoría válida.");
159 |         }
160 |     }
161 | }

```

Líneas 154 a 161: Convierte el texto al enum; si no es una categoría válida, ValidacionException.

```

162 | private List<Camion> camionesSeleccionados(HttpServletRequest req)
163 |     throws DAOException, ValidacionException {
164 |     List<Camion> camiones = new ArrayList<>();
165 |     String[] ids = req.getParameterValues("camionIds");
166 |     if (ids == null) {
167 |         return camiones;
168 |     }
169 |     for (String valor : ids) {
170 |         try {
171 |             Camion camion = camionDAO.buscarPorId(Long.parseLong(valor));
172 |             if (camion == null) {
173 |                 throw new ValidacionException("Alguno de los camiones seleccionados ya no existe.");
174 |             }
175 |             camiones.add(camion);
176 |         } catch (NumberFormatException e) {
177 |             throw new ValidacionException("Camión seleccionado inválido.");
178 |         }
179 |     }
180 |     return camiones;
181 | }
182 |

```

Líneas 162 a 182: Busca cada camión marcado por id en la base; si alguno no existe o el id es inválido, ValidacionException.

```

183 | private Map<String, String> aMapa(Chofer chofer) {
184 |     Map<String, String> valores = new HashMap<>();
185 |     valores.put("id", String.valueOf(chofer.getId()));
186 |     valores.put("nombre", chofer.getNombre());
187 |     valores.put("apellido", chofer.getApellido());
188 |     valores.put("dni", chofer.getDni());
189 |     valores.put("fechaNacimiento", chofer.getFechaNacimiento().toString());
190 |     valores.put("categoria", chofer.getCategoria().name());
191 |     valores.put("telefono", chofer.getTelefono());
192 |     return valores;
193 | }
194 | }

```

Líneas 183 a 194: Convierte el chofer en un Map de textos para el formulario de edición.

logistica-web / src / main / java / ar / edu / usal / logistica / web / controlador / ViajeServlet.java

Para qué existe: Controlador de la carga de viajes (solo administrador).

Defensa: Implementa el flujo exacto del enunciado: **1)** buscar el chofer por DNI (si no existe, se informa en pantalla); **2)** mostrar los camiones que puede manejar y que no están en viaje; **3)** elegir origen y destino; **4)** calcular tiempo y tanques; **5)** confirmar. Los pasos de consulta son GET y la confirmación es POST. **Al confirmar, el servidor vuelve a consultar el chofer, vuelve a verificar la disponibilidad del camión y recalcula todo:** lo que muestra la pantalla nunca se toma como cierto.

```

1 | package ar.edu.usal.logistica.web.controlador;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.CamionDAO;
4 | import ar.edu.usal.logistica.dao.ChoferDAO;
5 | import ar.edu.usal.logistica.dao.DAOFactory;
6 | import ar.edu.usal.logistica.dao.DistanciaDAO;
7 | import ar.edu.usal.logistica.dao.ViajeDAO;
8 | import ar.edu.usal.logistica.excepcion.ChoferNoEncontradoException;
9 | import ar.edu.usal.logistica.excepcion.DAOException;
10 | import ar.edu.usal.logistica.excepcion.ValidacionException;
11 | import ar.edu.usal.logistica.modelo.Camion;
12 | import ar.edu.usal.logistica.modelo.Chofer;
13 | import ar.edu.usal.logistica.modelo.Destino;
14 | import ar.edu.usal.logistica.modelo.Viaje;
15 |
16 | import jakarta.servlet.ServletException;
17 | import jakarta.servlet.annotation.WebServlet;
18 | import jakarta.servlet.http.HttpServletRequest;
19 | import jakarta.servlet.http.HttpServletResponse;
20 |
21 | import java.io.IOException;
22 | import java.net.URLEncoder;
23 | import java.nio.charset.StandardCharsets;
24 | import java.util.List;
25 |

```

Líneas 3 a 25: Importaciones de las clases que usa este archivo: **CamionDAO, ChoferDAO, DAOFactory, DistanciaDAO, ViajeDAO, ChoferNoEncontradoException, DAOException, ValidacionException, Camion, Chofer, Destino, Viaje, ServletException, WebServlet, HttpServletRequest, HttpServletResponse, IOException, URLEncoder, StandardCharsets, List**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

26 | /**
27 |  * Carga de viajes (solo administrador). El alta se hace en pasos, tal como
28 |  * describe el enunciado, y cada paso es una consulta (GET):
29 |  * 1) buscar el chofer por DNI; 2) elegir camion disponible, origen y destino;
30 |  * 3) ver el calculo de tiempo y tanques y confirmar (POST).
31 |  */
32 | @WebServlet(name = "ViajeServlet", urlPatterns = "/admin/viajes")

```

Líneas 26 a 32: Documentación del flujo en pasos.

```

33 | public class ViajeServlet extends BaseServlet {
34 |
35 |     private static final long serialVersionUID = 1L;
36 |
37 |     private final ViajeDAO viajeDAO = DAOFactory.getInstance().getViajeDAO();
38 |     private final ChoferDAO choferDAO = DAOFactory.getInstance().getChoferDAO();
39 |     private final CamionDAO camionDAO = DAOFactory.getInstance().getCamionDAO();
40 |     private final DistanciaDAO distanciaDAO = DAOFactory.getInstance().getDistanciaDAO();
41 |
42 |     @Override

```

Líneas 33 a 42: Cuatro DAO: viajes, choferes, camiones y distancias (todos de la fábrica).

```

43 |     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
44 |         try {
45 |             if ("nuevo".equals(parametro(req, "accion"))) {
46 |                 nuevo(req, resp);
47 |             } else {
48 |                 req.setAttribute("viajes", viajeDAO.listarTodos());
49 |                 vista(req, resp, "admin/viajes/lista.jsp");
50 |             }
51 |         } catch (DAOException e) {
52 |             throw new ServletException(e);
53 |         }
54 |     }
55 |
56 |     @Override

```

Líneas 43 a 56: GET: con accion=nuevo entra al flujo de carga; si no, lista los viajes.

```

57 | protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
58 |     if (!"guardar".equals(parametro(req, "accion"))) {
59 |         resp.sendError(HttpServletResponse.SC_BAD_REQUEST, "Acción desconocida");
60 |         return;
61 |     }
62 |     String dni = parametro(req, "dni");
63 |     String volver = "/admin/viajes?accion=nuevo&dni=" + URLEncoder.encode(dni, StandardCharsets.UTF_8);
64 |     try {
65 |         // Nada de lo que muestra la pantalla se da por cierto: se vuelve a consultar y a calcular todo

```

Líneas 57 a 65: POST: solo acepta accion=guardar. Calcula la URL de retorno (con el DNI codificado) por si hay que volver al formulario.

```

66 |         Chofer chofer = choferDAO.buscarPorDni(dni);
67 |         Viaje viaje = armarViaje(req, chofer);
68 |         viajeDAO.insertar(viaje);
69 |         mensajeFlash(req, "success", "Viaje cargado para " + chofer.getNombreCompleto() + ": "
70 |             + viaje.getOrigen().getNombre() + " → " + viaje.getDestino().getNombre() + ".");
71 |         redirigir(req, resp, "/admin/viajes");

```

Líneas 66 a 71: Recalcula todo en el servidor: busca el chofer, arma el viaje (validando camión y cálculo) y lo guarda.

```

72 |     } catch (ChoferNoEncontradoException e) {
73 |         mensajeFlash(req, "error", e.getMessage());
74 |         redirigir(req, resp, volver);
75 |     } catch (ValidacionException e) {
76 |         mensajeFlash(req, "warning", e.getMessage());
77 |         redirigir(req, resp, volver);
78 |     } catch (DAOException e) {
79 |         throw new ServletException(e);
80 |     }
81 | }
82 |

```

Líneas 72 a 82: Errores esperados → mensaje y vuelta al formulario; errores técnicos → página de error.

```

83 | private void nuevo(HttpServletRequest req, HttpServletResponse resp)
84 |     throws DAOException, ServletException, IOException {
85 |     String dni = parametro(req, "dni");
86 |     req.setAttribute("dni", dni);
87 |     req.setAttribute("destinos", Destino.values());
88 |     req.setAttribute("tabla", distanciaDAO.obtenerTabla());
89 |     req.setAttribute("origenElegido", parametro(req, "origen"));
90 |     req.setAttribute("destinoElegido", parametro(req, "destino"));
91 |     req.setAttribute("camionElegido", parametroLong(req, "camionId"));
92 |

```

Líneas 83 a 92: Prepara los datos del formulario: DNI, destinos, tabla de distancias y selecciones previas.

```

93 |     if (!dni.isEmpty()) {
94 |         try {
95 |             Chofer chofer = choferDAO.buscarPorDni(dni);
96 |             req.setAttribute("chofer", chofer);
97 |             List<Camion> disponibles = camionDAO.listarDisponiblesParaChofer(chofer.getId());
98 |             req.setAttribute("camiones", disponibles);
99 |             if (disponibles.isEmpty()) {
100 |                 mensajeAhora(req, "warning", "El chofer no tiene camiones disponibles: no está autorizado "
101 |                     + "a ninguno, o todos sus camiones están en viaje.");
102 |             } else if (parametroLong(req, "camionId") != null && !parametro(req, "origen").isEmpty()
103 |                 && !parametro(req, "destino").isEmpty()) {
104 |                 // Ya hay toda la informacion: se calcula y se muestra el resumen para confirmar
105 |                 req.setAttribute("viaje", armarViaje(req, chofer));
106 |             }

```

Líneas 93 a 106: Si se ingresó un DNI, busca el chofer y sus camiones disponibles. Si no hay disponibles avisa; si ya están camión, origen y destino, calcula y prepara el resumen para confirmar.

```

107 |     } catch (ChoferNoEncontradoException e) {
108 |         mensajeAhora(req, "error", e.getMessage());
109 |     } catch (ValidacionException e) {
110 |         mensajeAhora(req, "warning", e.getMessage());
111 |     }
112 | }

```

Líneas 107 a 112: Chofer inexistente: se informa en pantalla, como pide el enunciado.

```

113 |         vista(req, resp, "admin/viajes/nuevo.jsp");
114 |     }
115 |

```

Líneas 113 a 115: Muestra la vista con lo que corresponda al paso actual.

```

116 |     /** Arma el viaje validando que el camion este realmente disponible para ese chofer. */
117 |     private Viaje armarViaje(HttpServletRequest req, Chofer chofer) throws DAOException, ValidacionException {
118 |         Long camionId = parametroLong(req, "camionId");
119 |         Camion camion = null;
120 |         if (camionId != null) {
121 |             for (Camion c : camionDAO.listarDisponiblesParaChofer(chofer.getId())) {
122 |                 if (c.getId().equals(camionId)) {
123 |                     camion = c;
124 |                 }
125 |             }
126 |         }
127 |         if (camion == null) {
128 |             throw new ValidacionException("El camión elegido no está disponible para este chofer.");
129 |         }
130 |         Destino origen = destino(parametro(req, "origen"));
131 |         Destino destino = destino(parametro(req, "destino"));
132 |         int km = distanciaDAO.obtenerKm(origen, destino);
133 |         return Viaje.nuevo(chofer, camion, origen, destino, km);
134 |     }
135 |

```

Líneas 116 a 135: Verifica que el camión elegido esté **realmente** entre los disponibles para ese chofer, convierte origen y destino, consulta los km en la base y crea el viaje con Viaje.nuevo (que calcula tiempo y tanques).

```

136 |     private Destino destino(String codigo) throws ValidacionException {
137 |         try {
138 |             return Destino.valueOf(codigo);
139 |         } catch (IllegalArgumentException e) {
140 |             throw new ValidacionException("Seleccione un origen y un destino válidos.");
141 |         }
142 |     }
143 | }

```

Líneas 136 a 143: Convierte el texto al enum Destino; si no es válido lanza ValidacionException.

logistica-web / src / main / java / ar / edu / usal / logica / web / controlador / MisViajesServlet.java

Para qué existe: Controlador del perfil chofer: ver sus viajes, iniciarlos y marcarlos como realizados.

Defensa: El chofer solo ve y modifica **sus** viajes: el id sale de la Session, nunca de un parámetro; el DAO además lo incluye en el WHERE de cada UPDATE. GET lista; POST cambia el estado. Si un chofer forja una petición para un viaje ajeno, la base no modifica ninguna fila y se informa el error.

```

1 | package ar.edu.usal.logistica.web.controlador;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | import ar.edu.usal.logistica.dao.DAOFactory;
4 | import ar.edu.usal.logistica.dao.ViajeDAO;
5 | import ar.edu.usal.logistica.excepcion.DAOException;
6 | import ar.edu.usal.logistica.excepcion.ValidacionException;
7 |
8 | import jakarta.servlet.ServletException;
9 | import jakarta.servlet.annotation.WebServlet;
10 | import jakarta.servlet.http.HttpServletRequest;
11 | import jakarta.servlet.http.HttpServletResponse;
12 |
13 | import java.io.IOException;
14 |

```

Líneas 3 a 14: Importaciones de las clases que usa este archivo: **DAOFactory**, **ViajeDAO**, **DAOException**, **ValidacionException**, **ServletException**, **WebServlet**, **HttpServletRequest**, **HttpServletResponse**, **IOException**. Java necesita declararlas para poder nombrarlas sin su paquete completo.

```

15 | /**
16 |  * Pantalla del perfil chofer: solo ve SUS viajes y puede iniciarlos y
17 |  * marcarlos como realizados. El id del chofer sale siempre de la Session,
18 |  * nunca de un parametro, para que nadie pueda tocar viajes ajenos.
19 |  */
20 | @WebServlet(name = "MisViajesServlet", urlPatterns = "/chofer/viajes")

```

Líneas 15 a 20: Documentación.

```

21 | public class MisViajesServlet extends BaseServlet {
22 |
23 |     private static final long serialVersionUID = 1L;
24 |
25 |     private final ViajeDAO viajeDAO = DAOFactory.getInstance().getViajeDAO();
26 |
27 |     @Override

```

Líneas 21 a 27: DAO de viajes obtenido de la fábrica.

```

28 |     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
29 |         try {
30 |             req.setAttribute("viajes", viajeDAO.listarPorChofer(choferIdDeSesion(req)));
31 |             vista(req, resp, "chofer/mis-viajes.jsp");
32 |         } catch (DAOException e) {
33 |             throw new ServletException(e);
34 |         }
35 |     }
36 |
37 |     @Override

```

Líneas 28 a 37: GET: lista los viajes del chofer logueado.

```

38 |     protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
39 |         String accion = parametro(req, "accion");
40 |         Long viajeId = parametroLong(req, "viajeId");
41 |         try {
42 |             if (viajeId == null || !("iniciar".equals(accion) || "finalizar".equals(accion))) {
43 |                 throw new ValidacionException("Solicitud inválida.");
44 |             }
45 |             if ("iniciar".equals(accion)) {
46 |                 viajeDAO.iniciar(viajeId, choferIdDeSesion(req));
47 |                 mensajeFlash(req, "success", "Viaje iniciado. ¡Buen camino!");
48 |             } else {
49 |                 viajeDAO.finalizar(viajeId, choferIdDeSesion(req));
50 |                 mensajeFlash(req, "success", "Viaje marcado como realizado.");
51 |             }

```

Líneas 38 a 51: POST: valida acción y viaje; «iniciar» o «finalizar» según corresponda, siempre con el id del chofer de la Session.

```

52 |         } catch (ValidacionException e) {
53 |             mensajeFlash(req, "warning", e.getMessage());
54 |         } catch (DAOException e) {
55 |             throw new ServletException(e);
56 |         }
57 |         redirigir(req, resp, "/chofer/viajes");
58 |     }
59 | }

```

Líneas 52 a 59: Un error de regla (ya iniciado, no es suyo) se muestra como aviso; un error técnico va a la página de error. Después siempre redirige (PRG).

Proyecto MVC - vistas (JSP + JSTL + JS)

logistica-web / src / main / webapp / index.jsp

Para qué existe: Página de bienvenida (welcome-file de web.xml).

Defensa: Permite entrar con `http://host:puerto/logistica/` (path root con contexto). Redirige a `/inicio`; el filtro manda al login si no hay sesión.

```
1 | <%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

Línea 1: Importa JSTL core.

```
2 | <!-- Pagina de bienvenida: deriva al controlador de inicio (que a su vez pide login si hace falta) --%>
3 | <c:redirect url="/inicio"/>
```

Líneas 2 a 3: Comentario JSP (no llega al navegador) y **c:redirect** a `/inicio`.

logistica-web / src / main / webapp / WEB-INF / jsp / comun / taglibs.jspf

Para qué existe: Declaración única de las tres librerías JSTL, incluida al comienzo de cada JSP.

Defensa: Evita repetir las directivas en cada página y respeta que el taglib se declare antes de usarse. JSTL (punto 09) reemplaza a los scriptlets, que además están prohibidos en web.xml.

```
1 | <!-- Librerias JSTL usadas en todas las paginas (se incluye al principio de cada JSP) --%>
```

Línea 1: Comentario.

```
2 | <%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

Línea 2: **c:** core (if, choose, forEach, set, out, redirect).

```
3 | <%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
```

Línea 3: **fmt:** formato de números y locale.

```
4 | <%@ taglib prefix="fn" uri="jakarta.tags.functions" %>
```

Línea 4: **fn:** funciones (escapeXml).

logistica-web / src / main / webapp / WEB-INF / jsp / comun / encabezado.jspf

Para qué existe: Encabezado común: head HTML, barra de navegación según el perfil y aviso del servidor.

Defensa: Una sola plantilla para todas las pantallas (evita copiar y pegar). Toda salida de datos del usuario va con **c:out** o **fn:escapeXml** (escape de HTML): protege contra XSS. El botón «Cerrar sesión» es un formulario POST.

```
1 | <!-- Encabezado comun: <head>, barra de navegacion segun el perfil y aviso del servidor --%>
2 | <c:set var="ctx" value="${pageContext.request.contextPath}"/>
3 | <!-- Formato numerico argentino (coma decimal) fijo, sin depender del navegador --%>
```

Líneas 1 a 3: Comentario y variable **ctx** con el contexto de la aplicación (para armar URLs).

```
4 | <fmt:setLocale value="es_AR"/>
5 | <!-- El aviso puede venir de la request (forward) o de la Session (redirect) --%>
```

Líneas 4 a 5: Fija el formato numérico argentino (coma decimal), independiente del navegador.

```
6 | <c:set var="avisoTipo" value="${not empty alertaTipo ? alertaTipo : sessionScope.flashTipo}"/>
7 | <c:set var="avisoTexto" value="${not empty alertaTexto ? alertaTexto : sessionScope.flashTexto}"/>
8 | <c:remove var="flashTipo" scope="session"/>
9 | <c:remove var="flashTexto" scope="session"/>
```

Líneas 6 a 9: Toma el mensaje de la request (forward) o de la Session (redirect) y luego **borra** el de la Session para que se muestre una sola vez.

```

10 | <!DOCTYPE html>
11 | <html lang="es">
12 | <head>
13 |     <meta charset="UTF-8">
14 |     <meta name="viewport" content="width=device-width, initial-scale=1">
15 |     <title><c:out value="${tituloPagina}" /> - Logística</title>
16 |     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
17 |     <link href="${ctx}/css/app.css" rel="stylesheet">
18 | </head>

```

Líneas 10 a 18: Inicio del HTML: idioma, codificación, viewport responsive, título y estilos (Bootstrap desde CDN + CSS propio).

```

19 | <body>
20 | <c:if test="${not empty sessionScope.usuario}">
21 |     <nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
22 |         <div class="container">
23 |             <a class="navbar-brand" href="${ctx}/inicio">Logística <small>USAL Pilar</small></a>
24 |             <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#menu"
25 |                 aria-controls="menu" aria-expanded="false" aria-label="Menú">
26 |                 <span class="navbar-toggler-icon"></span>
27 |             </button>
28 |             <div class="collapse navbar-collapse" id="menu">
29 |                 <ul class="navbar-nav me-auto">

```

Líneas 19 a 29: Si hay usuario en Session, muestra la barra de navegación.

```

30 |                 <c:choose>
31 |                     <c:when test="${sessionScope.usuario.esAdmin()}">
32 |                         <li class="nav-item"><a class="nav-link" href="${ctx}/admin/viajes">Viajes</a></li>
33 |                         <li class="nav-item"><a class="nav-link" href="${ctx}/admin/choferes">Choferes</a></li>
34 |                         <li class="nav-item"><a class="nav-link" href="${ctx}/admin/camiones">Camiones</a></li>
35 |                     </c:when>
36 |                     <c:otherwise>
37 |                         <li class="nav-item"><a class="nav-link" href="${ctx}/chofer/viajes">Mis viajes</a></li>
38 |                     </c:otherwise>
39 |                 </c:choose>
40 |             </ul>

```

Líneas 30 a 40: Menú por perfil: el administrador ve Viajes, Choferes y Camiones; el chofer, solo «Mis viajes».

```

41 |                 <span class="navbar-text me-3">
42 |                     <c:out value="${sessionScope.usuario.username}" /> (<c:out value="${sessionScope.usuario.rol}" />)
43 |                 </span>
44 |                 <!-- Cerrar sesion modifica el estado: va por POST, nunca por un link GET -->
45 |                 <form action="${ctx}/logout" method="post" class="d-flex">
46 |                     <button type="submit" class="btn btn-outline-light btn-sm">Cerrar sesión</button>
47 |                 </form>
48 |             </div>
49 |         </div>
50 |     </nav>
51 | </c:if>

```

Líneas 41 a 51: Muestra usuario y rol (escapados con c:out) y el formulario POST de cierre de sesión.

```

52 | <main class="container pb-5">
53 |     <!-- fn:escapeXml evita que el texto rompa el atributo HTML (proteccion contra XSS) -->
54 |     <div id="alerta-servidor" class="alert alert-#{avisoTipo eq 'error' ? 'danger' : avisoTipo} d-none"
55 |         data-tipo="${fn:escapeXml(avisoTipo)}" data-texto="${fn:escapeXml(avisoTexto)}" role="alert">
56 |         <c:out value="${avisoTexto}" />
57 |     </div>

```

Líneas 52 a 57: Abre el contenido y define el div oculto con el aviso del servidor: tipo y texto en atributos data-, escapados con fn:escapeXml. El JS los muestra con SweetAlert.

logistica-web / src / main / webapp / WEB-INF / jsp / comun / pie.jspf

Para qué existe: Pie común.

Defensa: Carga los scripts al final (la página se dibuja primero). SweetAlert y el JS propio son opcionales: sin ellos todo funciona con los formularios y avisos de Bootstrap.

```

1 | <!-- Pie comun: cierra <main> y carga Bootstrap, SweetAlert y el JS propio -->

```

Línea 1: Comentario.

```
2 | </main>
```

Línea 2: Cierra el contenido principal.

```
3 | <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
4 | <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
5 | <script src="{ctx}/js/app.js"></script>
6 | </body>
7 | </html>
```

Líneas 3 a 7: Scripts: Bootstrap, SweetAlert2 y el app.js propio; cierra body y html.

logistica-web / src / main / webapp / WEB-INF / jsp / login.jsp

Para qué existe: Pantalla de inicio de sesión (la pantalla principal del enunciado).

Defensa: Formulario **POST** (las credenciales no viajan en la URL). Incluye la casilla «Recordarme» que activa la cookie. El usuario escrito se vuelve a mostrar con c:out (escapado) si falla el login.

```
1 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Iniciar sesión"/>
```

Líneas 1 a 3: Directiva de página (UTF-8) e importación de JSTL; define el título de la página.

```
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>
```

Línea 4: Incluye el encabezado común.

```
5 | <div class="card card-login shadow-sm mx-auto mt-5">
6 |   <div class="card-body p-4">
7 |     <h1 class="h4 text-center mb-1">Logística y Distribución</h1>
8 |     <p class="text-center text-muted">Iniciá sesión para continuar</p>
```

Líneas 5 a 8: Tarjeta centrada con el título.

```
9 |     <form action="{ctx}/login" method="post">
10 |       <div class="mb-3">
```

Líneas 9 a 10: Formulario que envía por POST a /login.

```
11 |         <label for="username" class="form-label">Usuario</label>
12 |         <input type="text" class="form-control" id="username" name="username" required maxlength="30"
13 |           autofocus autocomplete="username" value="<c:out value='${username}' />">
14 |       </div>
15 |     <div class="mb-3">
```

Líneas 11 a 15: Campo usuario: obligatorio, largo máximo 30, con foco automático.

```
16 |         <label for="password" class="form-label">Contraseña</label>
17 |         <input type="password" class="form-control" id="password" name="password" required
18 |           autocomplete="current-password">
19 |     </div>
```

Líneas 16 a 19: Campo contraseña (type=password oculta lo escrito).

```
20 |     <div class="form-check mb-3">
21 |       <input class="form-check-input" type="checkbox" id="recordar" name="recordar" checked>
22 |       <label class="form-check-label" for="recordar">Recordarme en este equipo</label>
23 |     </div>
```

Líneas 20 a 23: Casilla «Recordarme en este equipo» (marcada por defecto).

```
24 |       <button type="submit" class="btn btn-primary w-100">Ingresar</button>
25 |     </form>
26 |   </div>
27 | </div>
```

Líneas 24 a 27: Botón de ingreso y cierre de tarjeta.

```
28 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>
```

Línea 28: Incluye el pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / error.jsp

Para qué existe: Página de error única (403, 404 y errores inesperados).

Defensa: Nunca se muestra un stack trace al usuario (seguridad y presentación). El detalle técnico queda en el log del servidor.

```
1 | <%@ page contentType="text/html; charset=UTF-8" language="java" isErrorPage="true" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Error"/>
```

Líneas 1 a 3: Directiva marcada **isErrorPage** para acceder a los datos del error; define el título.

```
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>
```

Línea 4: Encabezado común.

```
5 | <div class="text-center mt-5">
6 |     <c:choose>
7 |         <c:when test="${pageContext.errorData.statusCode == 403}">
8 |             <h1 class="h3">Acceso denegado</h1>
9 |             <p class="text-muted">Tu perfil no tiene permiso para ver esta página.</p>
10 |         </c:when>
11 |         <c:when test="${pageContext.errorData.statusCode == 404}">
12 |             <h1 class="h3">Página no encontrada</h1>
13 |             <p class="text-muted">La dirección solicitada no existe.</p>
14 |         </c:when>
15 |         <c:otherwise>
16 |             <h1 class="h3">Ocurrió un error inesperado</h1>
17 |             <p class="text-muted">No pudimos completar la operación. Intentá de nuevo en unos minutos.</p>
18 |         </c:otherwise>
19 |     </c:choose>
```

Líneas 5 a 19: Según el código de estado muestra un mensaje distinto: acceso denegado, página no encontrada o error inesperado.

```
20 |     <a class="btn btn-primary" href="${ctx}/inicio">Volver al inicio</a>
21 | </div>
```

Líneas 20 a 21: Botón para volver al inicio.

```
22 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>
```

Línea 22: Pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / admin / camiones / lista.jsp

Para qué existe: Listado de camiones.

Defensa: Usa JSTL: **c:forEach** para recorrer, **c:out** para escapar, **fmt:formatNumber** para números, **c:if** para el mensaje de lista vacía. «Editar» es un enlace (GET, consulta); «Eliminar» es un formulario POST con confirmación (data-confirmar).

```
1 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Camiones"/>
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>
```

Líneas 1 a 4: Directiva, JSTL, título y encabezado.

```
5 | <div class="d-flex justify-content-between align-items-center mb-3">
6 |     <h1 class="h3 mb-0">Camiones</h1>
7 |     <a class="btn btn-primary" href="${ctx}/admin/camiones?accion=nuevo">Nuevo camión</a>
8 | </div>
9 | <div class="card shadow-sm">
10 |     <div class="table-responsive">
```

Líneas 5 a 10: Título y botón «Nuevo camión».

```

11 |         <table class="table table-hover align-middle mb-0">
12 |             <thead class="table-light">
13 |                 <tr>
14 |                     <th>Marca</th><th>Modelo</th><th>Dominio</th>
15 |                     <th class="text-end">Toneladas máx.</th>
16 |                     <th class="text-end">Tanque (l)</th>
17 |                     <th class="text-end">Consumo (l/km)</th>
18 |                     <th class="text-end">Acciones</th>
19 |                 </tr>
20 |             </thead>
21 |             <tbody>

```

Líneas 11 a 21: Tabla con sus columnas.

```

22 |         <c:forEach var="camion" items="${camiones}">
23 |             <tr>
24 |                 <td><c:out value="${camion.marca}"/></td>
25 |                 <td><c:out value="${camion.modelo}"/></td>
26 |                 <td><span class="badge text-bg-secondary"><c:out value="${camion.dominio}"/></span></td>
27 |                 <td class="text-end"><fmt:formatNumber value="${camion.toneladasMaximas}"
28 |                     <fmt:param name="maxFractionDigits" value="2"/></td>
29 |                 <td class="text-end"><fmt:formatNumber value="${camion.capacidadTanqueLitros}"
30 |                     <fmt:param name="maxFractionDigits" value="2"/></td>
31 |                 <td class="text-end"><fmt:formatNumber value="${camion.consumoLitrosPorKm}"
32 |                     <fmt:param name="maxFractionDigits" value="3"/></td>
33 |                 <td class="text-end">

```

Líneas 22 a 30: Una fila por camión, con los datos escapados y números formateados.

```

31 |                 <a class="btn btn-sm btn-outline-primary"
32 |                     href="${ctx}/admin/camiones?accion=editar&id=${camion.id}">Editar</a>

```

Líneas 31 a 32: Enlace de edición (GET con el id).

```

33 |             <form action="${ctx}/admin/camiones" method="post" class="d-inline"
34 |                 data-confirmar="¿Eliminar el camión ${fn:escapeXml(camion.dominio)}?">
35 |                 <input type="hidden" name="accion" value="eliminar">
36 |                 <input type="hidden" name="id" value="${camion.id}">
37 |                 <button type="submit" class="btn btn-sm btn-outline-danger">Eliminar</button>
38 |             </form>
39 |         </td>
40 |     </tr>
41 | </c:forEach>

```

Líneas 33 a 41: Formulario POST de baja; data-confirmar hace que el JS pida confirmación con SweetAlert.

```

42 |         <c:if test="${empty camiones}">
43 |             <tr><td colspan="7" class="text-center text-muted py-4">Todavía no hay camiones cargados.</td></tr>
44 |         </c:if>
45 |     </tbody>
46 | </table>
47 | </div>
48 | </div>

```

Líneas 42 a 48: Mensaje cuando no hay camiones.

```

49 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>

```

Línea 49: Pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / admin / camiones / form.jsp

Para qué existe: Formulario de alta y edición de camiones.

Defensa: El mismo formulario sirve para alta y edición (si hay id es edición). Validaciones HTML (required, min, max, step) como ayuda al usuario, **además** de las del servidor. Los valores se rellenan desde el Map «valores» con c:out.

```

1 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Camión"/>
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>

```

Líneas 1 a 4: Directiva, JSTL, título y encabezado.

```

5 | <h1 class="h3 mb-3"><c:out value="\${empty valores.id ? 'Nuevo camión' : 'Editar camión'}"/></h1>
6 | <div class="card shadow-sm">
7 |   <div class="card-body">

```

Líneas 5 a 7: El título cambia según sea alta o edición.

```

8 |         <form action="\${ctx}/admin/camiones" method="post" class="row g-3">
9 |             <input type="hidden" name="accion" value="guardar">
10 |            <input type="hidden" name="id" value="\<c:out value='\${valores.id}'/>">
11 |            <div class="col-md-6">

```

Líneas 8 a 11: Formulario POST con accion=guardar e id oculto (vacío en alta).

```

12 |                <label for="marca" class="form-label">Marca</label>
13 |                <input type="text" class="form-control" id="marca" name="marca" required maxlength="40"
14 |                    value="\<c:out value='\${valores.marca}'/>">
15 |            </div>
16 |            <div class="col-md-6">
17 |                <label for="modelo" class="form-label">Modelo</label>
18 |                <input type="text" class="form-control" id="modelo" name="modelo" required maxlength="40"
19 |                    value="\<c:out value='\${valores.modelo}'/>">
20 |            </div>
21 |            <div class="col-md-4">

```

Líneas 12 a 21: Marca y modelo.

```

22 |                <label for="dominio" class="form-label">Dominio (patente)</label>
23 |                <input type="text" class="form-control text-uppercase" id="dominio" name="dominio" required
24 |                    maxlength="10" placeholder="AB123CD" value="\<c:out value='\${valores.dominio}'/>">
25 |            </div>
26 |            <div class="col-md-4">

```

Líneas 22 a 26: Dominio (se normaliza en mayúsculas en el servidor).

```

27 |                <label for="toneladasMaximas" class="form-label">Toneladas máximas</label>
28 |                <input type="number" step="0.01" min="0.01" max="99" class="form-control" id="toneladasMaximas"
29 |                    name="toneladasMaximas" required value="\<c:out value='\${valores.toneladasMaximas}'/>">
30 |            </div>
31 |            <div class="col-md-4">

```

Líneas 27 a 31: Toneladas máximas.

```

32 |                <label for="capacidadTanqueLitros" class="form-label">Capacidad del tanque (litros)</label>
33 |                <input type="number" step="0.01" min="0.01" class="form-control" id="capacidadTanqueLitros"
34 |                    name="capacidadTanqueLitros" required value="\<c:out
35 |                        ↳ value='\${valores.capacidadTanqueLitros}'/>">
36 |            </div>
37 |            <div class="col-md-4">

```

Líneas 32 a 36: Capacidad del tanque en litros.

```

37 |                <label for="consumoLitrosPorKm" class="form-label">Consumo (litros por km)</label>
38 |                <input type="number" step="0.001" min="0.001" class="form-control" id="consumoLitrosPorKm"
39 |                    name="consumoLitrosPorKm" required value="\<c:out value='\${valores.consumoLitrosPorKm}'/>">
40 |            </div>

```

Líneas 37 a 40: Consumo en litros por km.

```

41 |            <div class="col-12 d-flex gap-2">
42 |                <button type="submit" class="btn btn-primary">Guardar</button>
43 |                <a class="btn btn-outline-secondary" href="\${ctx}/admin/camiones">Cancelar</a>
44 |            </div>
45 |        </form>
46 |    </div>
47 | </div>

```

Líneas 41 a 47: Botones Guardar y Cancelar.

```

48 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>

```

Línea 48: Pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / admin / choferes / lista.jsp

Para qué existe: Listado de choferes con sus camiones autorizados.

Defensa: Igual criterio que el listado de camiones. Muestra los camiones autorizados con un c:forEach anidado.

```

1 | <%@ page contentType="text/html;charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Choferes"/>
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>

```

Líneas 1 a 4: Directiva, JSTL, título y encabezado.

```

5 | <div class="d-flex justify-content-between align-items-center mb-3">
6 |     <h1 class="h3 mb-0">Choferes</h1>
7 |     <a class="btn btn-primary" href="${ctx}/admin/choferes?accion=nuevo">Nuevo chofer</a>
8 | </div>
9 | <div class="card shadow-sm">
10 |     <div class="table-responsive">

```

Líneas 5 a 10: Título y botón «Nuevo chofer».

```

11 |         <table class="table table-hover align-middle mb-0">
12 |             <thead class="table-light">
13 |                 <tr>
14 |                     <th>Apellido y nombre</th><th>DNI</th><th>Nacimiento</th><th>Categoría</th>
15 |                     <th>Celular</th><th>Camiones autorizados</th><th class="text-end">Acciones</th>
16 |                 </tr>
17 |             </thead>
18 |             <tbody>

```

Líneas 11 a 18: Tabla y encabezados de columnas.

```

19 |                 <c:forEach var="chofer" items="${choferes}">
20 |                     <tr>
21 |                         <td><c:out value="${chofer.nombreCompleto}"/></td>
22 |                         <td><c:out value="${chofer.dni}"/></td>
23 |                         <td><c:out value="${chofer.fechaNacimientoFormateada}"/></td>
24 |                         <td><span class="badge text-bg-info"><c:out value="${chofer.categoria}"/></span></td>
25 |                         <td><c:out value="${chofer.telefono}"/></td>
26 |                     </tr>

```

Líneas 19 a 26: Una fila por chofer: nombre, DNI, fecha, categoría, celular.

```

27 |                         <c:forEach var="camion" items="${chofer.camionesAutorizados}">
28 |                             <span class="badge text-bg-secondary"><c:out value="${camion.dominio}"/></span>
29 |                         </c:forEach>
30 |                         <c:if test="${empty chofer.camionesAutorizados}"><span
31 |                             class="text-muted">Ninguno</span></c:if>
32 |                     </td>

```

Líneas 27 a 32: Bucle anidado: un distintivo por camión autorizado (o «Ninguno»).

```

33 |                     <a class="btn btn-sm btn-outline-primary"
34 |                         href="${ctx}/admin/choferes?accion=editar&id=${chofer.id}">Editar</a>

```

Líneas 33 a 34: Editar (GET).

```

35 |                 <form action="${ctx}/admin/choferes" method="post" class="d-inline"
36 |                     data-confirmar="¿Eliminar al chofer ${fn:escapeXml(chofer.nombreCompleto)}? También se
37 |                     elimina su usuario.">
38 |                     <input type="hidden" name="accion" value="eliminar">
39 |                     <input type="hidden" name="id" value="${chofer.id}">
40 |                     <button type="submit" class="btn btn-sm btn-outline-danger">Eliminar</button>
41 |                 </form>
42 |             </tr>
43 |         </c:forEach>

```

Líneas 35 a 43: Eliminar (POST con confirmación).

```

44 |         <c:if test="${empty choferes}">
45 |             <tr><td colspan="7" class="text-center text-muted py-4">Todavía no hay choferes cargados.</td></tr>
46 |         </c:if>
47 |     </tbody>
48 | </table>
49 | </div>
50 | </div>

```

Líneas 44 a 50: Mensaje de lista vacía.

```

51 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>

```

Línea 51: Pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / admin / choferes / form.jsp

Para qué existe: Formulario de alta y edición de choferes.

Defensa: La contraseña inicial solo se pide en el alta. Los camiones se eligen con casillas; la categoría determina cuáles serán aceptados (lo valida el servidor). Patrones HTML (pattern) para DNI y celular ayudan al usuario pero no reemplazan la validación del servidor.

```
1 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Chofer"/>
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>
5 | <c:set var="esNuevo" value="${empty valores.id}"/>
```

Líneas 1 a 5: Directiva, JSTL, título, encabezado, y variable que indica si es alta.

```
6 | <h1 class="h3 mb-3"><c:out value="${esNuevo ? 'Nuevo chofer' : 'Editar chofer'}"/></h1>
7 | <div class="card shadow-sm">
8 |     <div class="card-body">
```

Líneas 6 a 8: Título según alta o edición.

```
9 |         <form action="${ctx}/admin/choferes" method="post" class="row g-3">
10 |             <input type="hidden" name="accion" value="guardar">
11 |             <input type="hidden" name="id" value="<c:out value='${valores.id}' />">
12 |         <div class="col-md-6">
```

Líneas 9 a 12: Formulario POST accion=guardar con id oculto.

```
13 |             <label for="nombre" class="form-label">Nombre</label>
14 |             <input type="text" class="form-control" id="nombre" name="nombre" required maxlength="50"
15 |                 value="<c:out value='${valores.nombre}' />">
16 |         </div>
17 |         <div class="col-md-6">
18 |             <label for="apellido" class="form-label">Apellido</label>
19 |             <input type="text" class="form-control" id="apellido" name="apellido" required maxlength="50"
20 |                 value="<c:out value='${valores.apellido}' />">
21 |         </div>
22 |         <div class="col-md-4">
```

Líneas 13 a 22: Nombre y apellido.

```
23 |             <label for="dni" class="form-label">DNI</label>
24 |             <input type="text" class="form-control" id="dni" name="dni" required pattern="\d{7,8}"
25 |                 maxlength="8" inputmode="numeric" title="7 u 8 dígitos" value="<c:out
26 |                     value='${valores.dni}' />">
27 |         </div>
28 |         <div class="col-md-4">
```

Líneas 23 a 27: DNI (7 u 8 dígitos).

```
28 |             <label for="fechaNacimiento" class="form-label">Fecha de nacimiento</label>
29 |             <input type="date" class="form-control" id="fechaNacimiento" name="fechaNacimiento" required
30 |                 value="<c:out value='${valores.fechaNacimiento}' />">
31 |         </div>
32 |         <div class="col-md-4">
```

Líneas 28 a 32: Fecha de nacimiento (input date).

```
33 |             <label for="telefono" class="form-label">Celular</label>
34 |             <input type="tel" class="form-control" id="telefono" name="telefono" required pattern="\d{8,15}"
35 |                 maxlength="15" inputmode="numeric" title="Solo números, entre 8 y 15 dígitos"
36 |                 value="<c:out value='${valores.telefono}' />">
37 |         </div>
38 |         <div class="col-md-6">
```

Líneas 33 a 38: Celular (solo números).

```

39 |         <label for="categoria" class="form-label">Categoría</label>
40 |         <select class="form-select" id="categoria" name="categoria" required>
41 |             <option value="">Seleccione...</option>
42 |             <c:forEach var="cat" items="${categorias}">
43 |                 <option value="${cat}" ${valores.categoria eq cat ? 'selected' : ''}>
44 |                     <c:out value="${cat.descripcion}" />
45 |                 </option>
46 |             </c:forEach>
47 |         </select>
48 |     </div>

```

Líneas 39 a 48: Lista desplegable generada con c:forEach; marca como seleccionada la actual.

```

49 |         <c:if test="${esNuevo}">
50 |             <div class="col-md-6">
51 |                 <label for="password" class="form-label">Contraseña inicial (usuario = DNI)</label>
52 |                 <input type="password" class="form-control" id="password" name="password" required minlength="6"
53 |                     autocomplete="new-password">
54 |             </div>
55 |         </c:if>
56 |     <div class="col-12">

```

Líneas 49 a 56: Contraseña inicial (solo alta).

```

57 |         <label class="form-label">Camiones que está autorizado a manejar</label>
58 |         <div class="form-text mb-2">Solo se aceptan camiones cuyas toneladas entren en la categoría
59 |             elegida.</div>
60 |         <div class="row">
61 |             <c:forEach var="camion" items="${camiones}">
62 |                 <div class="col-md-6">
63 |                     <div class="form-check">
64 |                         <input class="form-check-input" type="checkbox" name="camionIds"
65 |                             id="camion${camion.id}" value="${camion.id}"
66 |                             ${camionesSeleccionados.contains(camion.id) ? 'checked' : ''}>
67 |                         <label class="form-check-label" for="camion${camion.id}">
68 |                             <c:out value="${camion.descripcion}" />
69 |                         </label>
70 |                     </div>
71 |                 </div>
72 |             </c:forEach>
73 |             <c:if test="${empty camiones}">
74 |                 <p class="text-muted">No hay camiones cargados todavía.</p>
75 |             </c:if>
76 |         </div>

```

Líneas 57 a 76: Casillas de los camiones autorizados: marcadas las ya seleccionadas.

```

77 |         <div class="col-12 d-flex gap-2">
78 |             <button type="submit" class="btn btn-primary">Guardar</button>
79 |             <a class="btn btn-outline-secondary" href="${ctx}/admin/choferes">Cancelar</a>
80 |         </div>
81 |     </form>
82 | </div>
83 | </div>

```

Líneas 77 a 83: Botones Guardar y Cancelar.

```

84 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>

```

Línea 84: Pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / admin / viajes / lista.jsp

Para qué existe: Listado de todos los viajes (pantalla inicial del administrador).

Defensa: Muestra el resultado del cálculo guardado (km, días, tanques) y el estado con un color: verde finalizado, amarillo en curso, gris asignado (c:choose).

```

1 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Viajes" />
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>

```

Líneas 1 a 4: Directiva, JSTL, título y encabezado.

```

5 | <div class="d-flex justify-content-between align-items-center mb-3">
6 |   <h1 class="h3 mb-0">Viajes</h1>
7 |   <a class="btn btn-primary" href="{ctx}/admin/viajes?accion=nuevo">Cargar viaje</a>
8 | </div>
9 | <div class="card shadow-sm">
10 |   <div class="table-responsive">

```

Líneas 5 a 10: Título y botón «Cargar viaje».

```

11 |       <table class="table table-hover align-middle mb-0">
12 |         <thead class="table-light">
13 |           <tr>
14 |             <th>#</th><th>Chofer</th><th>Camión</th><th>Recorrido</th>
15 |             <th class="text-end">Km</th><th class="text-end">Días</th><th class="text-end">Tanques</th>
16 |             <th>Estado</th><th>Cargado</th>
17 |           </tr>
18 |         </thead>
19 |         <tbody>

```

Líneas 11 a 19: Tabla de viajes y sus encabezados.

```

20 |           <c:forEach var="viaje" items="{${viajes}}">
21 |             <tr>
22 |               <td>${viaje.id}</td>
23 |               <td><c:out value="{${viaje.chofer.nombreCompleto}}"/></td>
24 |               <td><c:out value="{${viaje.camion.dominio}}"/></td>
25 |               <td><c:out value="{${viaje.origen.nombre}}"/> → <c:out value="{${viaje.destino.nombre}}"/></td>

```

Líneas 20 a 25: Una fila por viaje: id, chofer, camión, recorrido.

```

26 |               <td class="text-end">${viaje.estimacion.distanciaKm()}</td>
27 |               <td class="text-end">${viaje.estimacion.dias()}</td>
28 |               <td class="text-end">${viaje.estimacion.tanques()}</td>
29 |             </td>

```

Líneas 26 a 29: Km, días y tanques (los accesores de EstimacionViaje se llaman como métodos en EL).

```

30 |               <c:choose>
31 |                 <c:when test="{${viaje.estado eq 'FINALIZADO'}}"><span class="badge
32 |                   ↳ text-bg-success"></c:when>
33 |                 <c:when test="{${viaje.estado eq 'EN_CURSO'}}"><span class="badge text-bg-warning"></c:when>
34 |                 <c:otherwise><span class="badge text-bg-secondary"></c:otherwise>
35 |               </c:choose>
36 |               <c:out value="{${viaje.estado.descripcion}}"/></span>

```

Líneas 30 a 36: Color del estado.

```

37 |               <td><c:out value="{${viaje.fechaCargaFormateada}}"/></td>
38 |             </tr>
39 |           </c:forEach>

```

Líneas 37 a 39: Fecha de carga.

```

40 |           <c:if test="{${empty viajes}}">
41 |             <tr><td colspan="9" class="text-center text-muted py-4">Todavía no se cargaron viajes.</td></tr>
42 |           </c:if>
43 |         </tbody>
44 |       </table>
45 |     </div>
46 | </div>

```

Líneas 40 a 46: Mensaje de lista vacía.

```

47 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>

```

Línea 47: Pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / admin / viajes / nuevo.jsp

Para qué existe: Pantalla de carga de viajes en tres pasos.

Defensa: Un solo JSP que muestra cada paso según los datos presentes. Pasos 1 y 2 son **GET** (solo consultan); el paso 3 confirma por **POST** (modifica). La confirmación reenvía solo ids y códigos; el servidor recalcula todo. También muestra la tabla de distancias del enunciado, leída de la base.

```

1 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Cargar viaje"/>
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>

```

Líneas 1 a 4: Directiva, JSTL, título y encabezado.

```

5 | <h1 class="h3 mb-3">Cargar viaje</h1>
6 |
7 | <!-- PASO 1: buscar el chofer por DNI (consulta: GET) -->

```

Líneas 5 a 7: Título.

```

8 | <div class="card shadow-sm mb-3">
9 |   <div class="card-header">1. Chofer</div>
10 |   <div class="card-body">

```

Líneas 8 a 10: Paso 1.

```

11 |         <form action="${ctx}/admin/viajes" method="get" class="row g-2">
12 |           <input type="hidden" name="accion" value="nuevo">
13 |           <div class="col-sm-6 col-md-4">
14 |             <input type="text" class="form-control" name="dni" required pattern="\d{7,8}" maxlength="8"
15 |               inputmode="numeric" placeholder="DNI del chofer" aria-label="DNI del chofer"
16 |               value="<c:out value='${dni}' />">
17 |           </div>
18 |           <div class="col-auto"><button type="submit" class="btn btn-primary">Buscar</button></div>
19 |         </form>

```

Líneas 11 a 19: Formulario de búsqueda por DNI (GET). El campo solo acepta 7 u 8 dígitos.

```

20 |         <c:if test="${not empty chofer}">
21 |           <p class="mt-3 mb-0">
22 |             <strong><c:out value='${chofer.nombreCompleto}' /></strong> ·
23 |             <c:out value='${chofer.categoria.descripcion}' /> · Cel. <c:out value='${chofer.telefono}' />
24 |           </p>
25 |         </c:if>
26 |       </div>
27 | </div>
28 |

```

Líneas 20 a 28: Si se encontró el chofer muestra su nombre, categoría y celular.

```

29 | <c:if test="${not empty chofer and not empty camiones}">
30 |   <!-- PASO 2: camion disponible, origen y destino; "Calcular" sigue siendo una consulta -->
31 |   <div class="card shadow-sm mb-3">
32 |     <div class="card-header">2. Camión, origen y destino</div>
33 |     <div class="card-body">
34 |       <form action="${ctx}/admin/viajes" method="get" class="row g-3">
35 |         <input type="hidden" name="accion" value="nuevo">
36 |         <input type="hidden" name="dni" value="<c:out value='${chofer.dni}' />">
37 |         <div class="col-md-4">

```

Líneas 29 a 37: Paso 2: solo aparece si hay chofer con camiones disponibles.

```

38 |           <label for="camionId" class="form-label">Camión disponible</label>
39 |           <select class="form-select" id="camionId" name="camionId" required>
40 |             <option value="">Selecione...</option>
41 |             <c:forEach var="camion" items="${camiones}">
42 |               <option value="${camion.id}" ${camionElegido eq camion.id ? 'selected' : ''}>
43 |                 <c:out value='${camion.descripcion}' />
44 |               </option>
45 |             </c:forEach>
46 |           </select>
47 |         </div>
48 |         <div class="col-md-4">

```

Líneas 38 a 48: Lista de camiones disponibles de ese chofer.

```

49 |           <label for="origen" class="form-label">Origen</label>
50 |           <select class="form-select" id="origen" name="origen" required>
51 |             <option value="">Selecione...</option>
52 |             <c:forEach var="d" items="${destinos}">
53 |               <option value="${d}" ${origenElegido eq d ? 'selected' : ''}><c:out
54 |                 value='${d.nombre}' /></option>
55 |             </c:forEach>
56 |           </select>
57 |         </div>
58 |         <div class="col-md-4">

```

Líneas 49 a 57: Lista de orígenes (enum Destino).

```

58 |         <label for="destino" class="form-label">Destino</label>
59 |         <select class="form-select" id="destino" name="destino" required>
60 |             <option value="">Selecione...</option>
61 |             <c:forEach var="d" items="${destinos}">
62 |                 <option value="{d}" ${destinoElegido eq d ? 'selected' : ''}><c:out
63 |                     ↳ value="{d.nombre}"</option>
64 |             </c:forEach>
65 |         </select>
66 |     </div>
67 |     <div class="col-12"><button type="submit" class="btn btn-primary">Calcular viaje</button></div>
68 | </form>
69 | </div>
70 | </c:if>
71 |

```

Líneas 58 a 71: Lista de destinos.

```

72 | <c:if test="${not empty viaje}">
73 |     <!-- PASO 3: resumen con tiempo y tanques; confirmar es una modificacion: POST -->
74 |     <div class="card shadow-sm border-success mb-3">
75 |         <div class="card-header text-bg-success">3. Resumen del viaje</div>
76 |         <div class="card-body">

```

Líneas 72 a 76: Paso 3: solo aparece si ya se calculó el viaje.

```

77 |         <dl class="row resumen-viaje mb-3">
78 |             <dt class="col-sm-4">Recorrido</dt>
79 |             <dd class="col-sm-8"><c:out value="{viaje.origen.nombre}" /> → <c:out
80 |                 ↳ value="{viaje.destino.nombre}"</dd>
81 |             <dt class="col-sm-4">Distancia</dt>
82 |             <dd class="col-sm-8">${viaje.estimacion.distanciaKm()} km</dd>
83 |             <dt class="col-sm-4">Tiempo de viaje</dt>
84 |             <dd class="col-sm-8">${viaje.estimacion.dias()} día(s) (200 km por día)</dd>
85 |             <dt class="col-sm-4">Combustible</dt>
86 |             <dd class="col-sm-8"><fmt:formatNumber value="{viaje.estimacion.litrosTotales()}"
87 |                 ↳ maxFractionDigits="1" /> litros</dd>
88 |             <dt class="col-sm-4">Tanques a llenar</dt>
89 |             <dd class="col-sm-8">${viaje.estimacion.tanques()}
90 |                 (tanque de <fmt:formatNumber value="{viaje.camion.capacidadTanqueLitros}"
91 |                     ↳ maxFractionDigits="0" /> l)</dd>
92 |         </dl>

```

Líneas 77 a 89: Resumen del cálculo: recorrido, distancia, días, litros y tanques.

```

90 |         <form action="{ctx}/admin/viajes" method="post" class="d-flex gap-2">
91 |             data-confirmar="¿Confirmar la carga del viaje?">
92 |             <input type="hidden" name="accion" value="guardar">
93 |             <input type="hidden" name="dni" value="{c:out value='{chofer.dni}' />">
94 |             <input type="hidden" name="camionId" value="{viaje.camion.id}">
95 |             <input type="hidden" name="origen" value="{viaje.origen}">
96 |             <input type="hidden" name="destino" value="{viaje.destino}">
97 |             <button type="submit" class="btn btn-success">Confirmar y cargar viaje</button>
98 |             <a class="btn btn-outline-secondary" href="{ctx}/admin/viajes">Cancelar</a>
99 |         </form>
100 |     </div>
101 | </div>
102 | </c:if>
103 |

```

Líneas 90 a 103: Formulario de confirmación por POST con campos ocultos (DNI, camión, origen, destino) y confirmación con SweetAlert.

```

104 | <!-- Tabla de distancias del enunciado (consultada a la base con DistanciaDAO) -->
105 | <div class="card shadow-sm">
106 |   <div class="card-header">Tabla de distancias (km)</div>
107 |   <div class="table-responsive">
108 |     <table class="table table-sm table-bordered tabla-distancias mb-0">
109 |       <thead class="table-light">
110 |         <tr>
111 |           <th></th>
112 |           <c:forEach var="d" items="${destinos}"><th><c:out value="${d.nombre}"/></th></c:forEach>
113 |         </tr>
114 |       </thead>
115 |       <tbody>
116 |         <c:forEach var="fila" items="${destinos}">
117 |           <tr>
118 |             <th class="table-light"><c:out value="${fila.nombre}"/></th>
119 |             <c:forEach var="col" items="${destinos}">
120 |               <c:choose>
121 |                 <c:when test="${fila eq col}"><td class="misma"></td></c:when>
122 |                 <c:otherwise><td>${tabla[fila][col]}</td></c:otherwise>
123 |               </c:choose>
124 |             </c:forEach>
125 |           </tr>
126 |         </c:forEach>
127 |       </tbody>
128 |     </table>
129 |   </div>
130 | </div>

```

Líneas 104 a 130: Tabla de distancias: recorre filas y columnas de destinos y consulta tabla[fila][columna]; la diagonal queda gris.

```

131 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>

```

Línea 131: Pie común.

logistica-web / src / main / webapp / WEB-INF / jsp / chofer / mis-viajes.jsp

Para qué existe: Pantalla del perfil chofer.

Defensa: Muestra tarjetas con los viajes del chofer y solo los botones que corresponden al estado: «Iniciar» si está asignado y «Marcar como realizado» si está en curso (puedeIniciarse()/puedeFinalizarse()) del modelo: la vista no repite reglas). Ambos botones son POST con confirmación.

```

1 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 | <%@ include file="/WEB-INF/jsp/comun/taglibs.jspf" %>
3 | <c:set var="tituloPagina" value="Mis viajes"/>
4 | <%@ include file="/WEB-INF/jsp/comun/encabezado.jspf" %>

```

Líneas 1 a 4: Directiva, JSTL, título y encabezado.

```

5 | <h1 class="h3 mb-3">Mis viajes</h1>
6 | <div class="row g-3">

```

Líneas 5 a 6: Título y contenedor de tarjetas.

```

7 |   <c:forEach var="viaje" items="${viajes}">
8 |     <div class="col-md-6">
9 |       <div class="card shadow-sm h-100">
10 |         <div class="card-body">
11 |           <div class="d-flex justify-content-between">
12 |             <h2 class="h5">
13 |               <c:out value="${viaje.origen.nombre}"/> → <c:out value="${viaje.destino.nombre}"/>
14 |             </h2>

```

Líneas 7 a 14: Una tarjeta por viaje con el recorrido.

```

15 |         <c:choose>
16 |             <c:when test="${viaje.estado eq 'FINALIZADO'}"><span class="badge text-bg-success
17 |                 ↳ align-self-start"></c:when>
18 |             <c:when test="${viaje.estado eq 'EN_CURSO'}"><span class="badge text-bg-warning
19 |                 ↳ align-self-start"></c:when>
20 |             <c:otherwise><span class="badge text-bg-secondary align-self-start"></c:otherwise>
21 |         </c:choose>
22 |     <c:out value="${viaje.estado.descripcion}"/></span>
23 | </div>

```

Líneas 15 a 21: Color del estado.

```

22 |         <ul class="list-unstyled small mb-3">
23 |             <li>Camión: <c:out value="${viaje.camion.descripcion}"/></li>
24 |             <li>Distancia: ${viaje.estimacion.distanciaKm()} km · ${viaje.estimacion.dias()} día(s)</li>
25 |             <li>Tanques a llenar: ${viaje.estimacion.tanques()}</li>
26 |             <li>Iniciado: <c:out value="${viaje.fechaInicioFormateada}"/> ·
27 |                 Finalizado: <c:out value="${viaje.fechaFinFormateada}"/></li>
28 |         </ul>

```

Líneas 22 a 28: Datos del viaje: camión, distancia y días, tanques, fechas.

```

29 |         <c:if test="${viaje.puedeIniciarse()}">
30 |             <form action="${ctx}/chofer/viajes" method="post" data-confirmar="¿Iniciar este viaje ahora?">
31 |                 <input type="hidden" name="accion" value="iniciar">
32 |                 <input type="hidden" name="viajeId" value="${viaje.id}">
33 |                 <button type="submit" class="btn btn-primary">Iniciar viaje</button>
34 |             </form>
35 |         </c:if>

```

Líneas 29 a 35: Botón «Iniciar viaje» (POST).

```

36 |         <c:if test="${viaje.puedeFinalizarse()}">
37 |             <form action="${ctx}/chofer/viajes" method="post" data-confirmar="¿Marcar el viaje como
38 |                 ↳ realizado?">
39 |                 <input type="hidden" name="accion" value="finalizar">
40 |                 <input type="hidden" name="viajeId" value="${viaje.id}">
41 |                 <button type="submit" class="btn btn-success">Marcar como realizado</button>
42 |             </form>
43 |         </c:if>
44 |     </div>
45 | </div>
46 | </c:forEach>

```

Líneas 36 a 46: Botón «Marcar como realizado» (POST).

```

47 |         <c:if test="${empty viajes}">
48 |             <div class="col-12"><p class="text-center text-muted py-4">No tenés viajes asignados.</p></div>
49 |         </c:if>
50 |     </div>

```

Líneas 47 a 50: Mensaje cuando no tiene viajes.

```

51 | <%@ include file="/WEB-INF/jsp/comun/pie.jspf" %>

```

Línea 51: Pie común.

logistica-web / src / main / webapp / js / app.js

Para qué existe: JavaScript común a todas las páginas.

Defensa: JS y SweetAlert son opcionales en el enunciado; la aplicación funciona sin ellos (degradación elegante: sin SweetAlert se usa el aviso de Bootstrap y window.confirm). Se usa dataset para leer el mensaje del servidor, sin construir HTML con texto del usuario (evita XSS).

```

1 | // Comportamiento comun a todas las paginas (JavaScript + SweetAlert, ambos opcionales).
2 | (function () {
3 |     'use strict';
4 |
5 |     var iconos = { success: 'success', error: 'error', warning: 'warning', info: 'info' };
6 |
7 |     // Muestra el mensaje que dejo el servidor (div#alerta-servidor) como ventana SweetAlert

```

Líneas 1 a 7: Comentario y función autoejecutable con modo estricto (no contamina el ámbito global). Tabla de íconos por tipo de aviso.

```

8 |     function mostrarAlerta() {
9 |         var alerta = document.getElementById('alerta-servidor');
10 |         if (!alerta || !alerta.dataset.texto) {
11 |             return;
12 |         }
13 |         var tipo = alerta.dataset.tipo || 'info';
14 |         if (window.Swal) {
15 |             Swal.fire({ icon: iconos[tipo] || 'info', text: alerta.dataset.texto, confirmButtonText: 'Aceptar' });
16 |         } else {
17 |             alerta.classList.remove('d-none'); // sin SweetAlert queda visible el aviso de Bootstrap
18 |         }
19 |     }
20 |
21 |     // Pide confirmacion antes de enviar los formularios marcados con data-confirmar

```

Líneas 8 a 21: Muestra el mensaje del servidor como ventana SweetAlert con el ícono correspondiente; si SweetAlert no cargó, deja visible el aviso de Bootstrap.

```

22 |     function confirmarFormularios() {
23 |         document.querySelectorAll('form[data-confirmar]').forEach(function (form) {
24 |             form.addEventListener('submit', function (evento) {
25 |                 if (form.dataset.confirmado === 'si') {
26 |                     return;
27 |                 }
28 |                 evento.preventDefault();
29 |                 var mensaje = form.dataset.confirmar;
30 |                 if (window.Swal) {
31 |                     Swal.fire({
32 |                         icon: 'question', text: mensaje, showCancelButton: true,
33 |                         confirmButtonText: 'Sí, continuar', cancelButtonText: 'Cancelar'
34 |                     }).then(function (resultado) {
35 |                         if (resultado.isConfirmed) {
36 |                             form.dataset.confirmado = 'si';
37 |                             form.submit();
38 |                         }
39 |                     });
40 |                 } else if (window.confirm(mensaje)) {
41 |                     form.dataset.confirmado = 'si';
42 |                     form.submit();
43 |                 }
44 |             });
45 |         });
46 |     }
47 |

```

Líneas 22 a 47: Para cada formulario con data-confirmar intercepta el envío y pide confirmación (SweetAlert o confirm); si confirma, marca el formulario y lo envía.

```

48 |     document.addEventListener('DOMContentLoaded', function () {
49 |         mostrarAlerta();
50 |         confirmarFormularios();
51 |     });
52 | })();

```

Líneas 48 a 52: Ejecuta ambas funciones cuando la página terminó de cargar.

logistica-web / src / main / webapp / css / app.css

Para qué existe: Estilos propios mínimos (el resto es Bootstrap).

Defensa: Se usa Bootstrap para un diseño responsive sin escribir CSS extenso; solo se agregan detalles puntuales.

```

1 | body { background: #f4f6f9; }
2 | .navbar-brand small { font-weight: 400; opacity: .7; }

```

Líneas 1 a 2: Fondo gris claro y estilo del nombre de la aplicación.

```

3 | .card-login { max-width: 420px; }

```

Línea 3: Ancho máximo de la tarjeta de login.

```

4 | .tabla-distancias th, .tabla-distancias td { text-align: center; font-size: .85rem; white-space: nowrap; }
5 | .tabla-distancias td.misma { background: #e9ecef; }
6 | .resumen-viaje dt { font-weight: 600; }

```

Líneas 4 a 6: Estilo de la tabla de distancias (centrada, letra chica, diagonal gris) y del resumen de viaje.

Parte IV - Defensa oral: preguntas frecuentes

¿Por qué separar en dos proyectos (DAO y MVC)?

Es lo que pide el enunciado y además baja el acoplamiento: el proyecto DAO no conoce servlets ni JSP, se puede probar solo (tiene tests unitarios) y se podría reutilizar con otra interfaz. La dependencia va en un solo sentido: web → dao.

¿Por qué una interfaz DAO y una Factory si solo hay una implementación?

Porque los controladores dependen de una abstracción. Si se cambia JDBC por otra tecnología (JPA, otra base) se crea otra fábrica y los controladores no cambian. También permite reemplazar el DAO por un falso en pruebas.

¿Por qué el Singleton usa una clase interna (holder)?

Porque es la forma más simple de tener inicialización perezosa y segura entre hilos sin synchronized: la JVM garantiza que la clase Holder se carga una sola vez, cuando se usa por primera vez. Un servidor web atiende muchos hilos a la vez, así que la seguridad entre hilos es importante.

¿Por qué el Singleton entrega conexiones nuevas en lugar de compartir una?

Una Connection JDBC no está pensada para usarse desde varios hilos a la vez, y compartirla mezclaría transacciones de distintos usuarios. Se abre una por operación y se cierra con try-with-resources. En producción se usaría un pool de conexiones (por ejemplo un DataSource de Tomcat) sin cambiar los DAO: es una mejora conocida.

¿Cuándo usaste Statement, PreparedStatement y CallableStatement y por qué?

Statement solo en consultas fijas sin datos del usuario (listar camiones, choferes y distancias). **PreparedStatement** en todo lo que recibe valores (alta, baja, modificación, login, búsquedas), porque los parámetros ? evitan la **inyección SQL**. **CallableStatement** para invocar los stored procedures sp_camiones_disponibles (regla de disponibilidad en un único lugar) y sp_finalizar_viaje (con parámetro OUT que informa cuántas filas cambió).

¿Qué transacciones hay y por qué?

Alta de chofer: inserta en chofer, chofer_camion y usuario; si falla cualquiera, rollback y no queda un chofer sin usuario. **Modificación de chofer:** actualiza, borra y reinserta autorizaciones. **Alta de viaje:** bloquea el camión con SELECT FOR UPDATE, verifica que siga libre e inserta; así dos administradores no pueden asignar el mismo camión a la vez. **Iniciar viaje:** verifica que no haya otro en curso y actualiza. **Guardar token:** limpia vencidos e inserta. El patrón es siempre setAutoCommit(false), commit al final, rollback en el catch, y volver a autocommit y cerrar en el finally.

¿Cómo se calcula el tiempo y la cantidad de tanques?

Tiempo = techo(km / 200). Ejemplo: CABA-Córdoba = 646 km → 3,23 → **4 días**. Tanques = techo((km × litros por km) / capacidad del tanque). Ejemplo: con un camión de 0,4 l/km y tanque de 600 l, 646 km consumen 258,4 l → 0,43 → **1 tanque**. Se redondea hacia arriba porque un viaje no termina a mitad de día y hay que cargar el último tanque aunque se use parcialmente. El cálculo está en CalculadoraViaje, tiene tests, y se hace en el servidor.

¿Cómo se sabe qué camiones puede manejar un chofer y cuáles están disponibles?

El procedure sp_camiones_disponibles une chofer_camion (autorización), categoría (las toneladas del camión no superan las de la categoría) y descarta con NOT EXISTS los camiones que tengan un viaje ASIGNADO o EN_CURSO. Además Chofer.puedeManejar() valida lo mismo en el modelo, y la base bloquea la doble asignación con la transacción.

¿Cómo funciona la sesión recordada y qué seguridad tiene?

La cookie no guarda la contraseña sino un token aleatorio de 256 bits que está registrado en la base con vencimiento. Es HttpOnly (JavaScript no la lee), SameSite=Lax y Secure en HTTPS. Al cerrar sesión se borra el token de la base, se invalida la Session y se expiran ambas cookies; aunque alguien copiara el token, ya no sirve.

¿Cómo se guardan las contraseñas?

Nunca en texto plano: PBKDF2-HMAC-SHA256 con sal aleatoria por usuario y 65.536 iteraciones; se guarda «iteraciones\$sal\$hash». La comparación es en tiempo constante. El login responde con el mismo mensaje si falla usuario o contraseña.

¿Por qué la baja es un POST y no un DELETE?

Los formularios HTML solo envían GET o POST. Se usa POST con accion=eliminar y, como toda modificación, va seguido de un redirect (PRG). Las consultas son GET porque son seguras e idempotentes: se pueden recargar, guardar en favoritos y no cambian datos.

¿Qué es el patrón Post/Redirect/Get y por qué lo usan?

Después de un POST exitoso el servidor responde con un redirect y el navegador hace un GET. Si el usuario recarga (F5), repite el GET y no el POST: no se duplican altas ni viajes.

¿Por qué los servlets Login/Logout están en web.xml y los demás con anotaciones?

El enunciado pide usar ambas formas. Login, Logout y el filtro de seguridad son infraestructura y quedan visibles en un solo archivo; los controladores de negocio usan @WebServlet, que deja la URL junto al código. web.xml además define sesión, páginas de error, welcome-file y prohíbe scriptlets.

¿Cómo evitas XSS e inyección SQL?

SQL: PreparedStatement con parámetros. XSS: toda salida de datos va con c:out o fn:escapeXml, y el mensaje del servidor se pasa al JavaScript como atributo data- escapado (no se concatena HTML).

¿Qué pasa si un chofer intenta entrar a /admin o tocar un viaje ajeno?

El filtro devuelve 403 para /admin si no es ADMIN. Para viajes, el id del chofer sale de la Session y el UPDATE incluye «AND chofer_id = ?»: si el viaje es ajeno no se modifica ninguna fila y se informa el error.

¿Cómo están organizados los comentarios y la legibilidad (lo que evalúa el profesor)?

Cada clase tiene su Javadoc con el propósito, los métodos de negocio están comentados con la regla que implementan, los nombres son descriptivos en español del dominio, no hay números mágicos (constantes), cada método hace una sola cosa y no hay duplicación importante (BaseServlet, Mapeadores, JdbcUtil).

Supuestos y limitaciones (dicho con honestidad)

- **Toneladas por categoría** (A=10, B=20, C=30, D=40): el enunciado no las define; son un supuesto, fácil de cambiar en el enum Categoria y en la tabla categoria.
- **Tiempo de viaje** se expresa en días de 200 km, tal como indica el enunciado; no se modelan horas de descanso ni feriados.
- **Usuario de chofer = DNI**; la contraseña inicial la define el administrador. No hay pantalla de cambio de contraseña.
- **Sin protección CSRF** con token: se mitiga con cookies SameSite=Lax y porque las acciones que modifican son POST, pero un sistema real debería agregar un token CSRF en cada formulario.
- **Sin pool de conexiones**: se abre una conexión por operación. Para producción convendría un DataSource con pool.
- **Bootstrap y SweetAlert se cargan desde CDN**: requieren Internet; sin ellos la aplicación funciona igual, con estilos y avisos más simples.
- **Base probada**: el script está escrito para MySQL 8. En el entorno donde se desarrolló no había un servidor MySQL, por lo que todas las pruebas manuales se hicieron contra MariaDB 10.11 (compatible) con el mismo driver de MySQL. Se recomienda ejecutar el script en MySQL 8 antes de entregar.
- Se probaron manualmente con un Tomcat embebido: login correcto e incorrecto, sesión recordada, logout, ABM de camiones y choferes (incluyendo duplicados, categoría insuficiente y rollback), carga de viaje completa, camión ocupado, iniciar y finalizar, acceso de chofer a /admin (403) y chofer intentando tocar un viaje ajeno. No se verificó visualmente el aspecto en un navegador.

Cómo ejecutar el proyecto

- Ejecutar **db/01_ddl_logistica.sql** y luego **db/02_datos_iniciales.sql** en MySQL 8.
- Ajustar usuario y clave de la base en **logistica-dao/src/main/resources/db.properties** (o variables DB_USER y DB_PASSWORD).
- **mvn clean package** en la raíz: genera **logistica-web/target/logistica.war**.
- Copiar el WAR a la carpeta webapps de **Tomcat 10.1** y abrir **http://localhost:8080/logistica/**.

- Usuarios de prueba: admin / admin123; 30111222 / chofer123; 28555666 / chofer123.